

OpenMP: Além das Diretivas de Compilação

Rogério Aparecido Gonçalves (rogerioag@utfpr.edu.br)

2028-06-07

Sumário

Prefácio	3
Bem-vindo	3
Licença	3
1 Introdução à Computação Paralela	4
1.1 Introdução	4
1.2 Suporte do Hardware	4
1.2.1 Arquiteturas Superescalares	5
1.2.2 Arquiteturas SMT	6
1.2.3 Arquiteturas de Múltiplos Núcleos com SMT	9
1.3 Suporte do Sistema Operacional	10
1.4 Prática	17
1.5 Considerações Finais	18
1.6 <i>Word Cloud</i>	18
Referências	18
2 Multithreading com Pthreads	23
2.1 Introdução	23
3 Introdução	25
3.1 Introdução	25
3.2 Threads: Exemplo de uso - Processador de texto	26
3.3 Threads: Exemplo de uso - Servidor Web	27
3.4 Threads: Implementação	27
3.5 Threads: Implementação	29
3.6 Threads: Nível de Usuário	29
3.7 Threads: Nível de Núcleo	29
3.8 Threads: Implementação Híbrida	30
3.9 Threads: Modelos de mapeamento	31
3.10 Threads: Tratamento de sinais	34
3.11 Threads: Operações	34
3.12 POSIX	35
3.13 POSIX e Pthreads	35
3.14 Threads no Linux	35

4	Implementação de Threads	37
4.1	Implementação de Threads em Linguagens de Programação	37
4.2	Exemplos	38
4.3	Implementação de Paralelismo com Cilk	45
4.4	Implementação de Paralelismo com OpenMP	46
5	Prática: Programação com Threads no Linux	48
5.1	Introdução	48
5.2	POSIX e Threads	48
5.3	Threads no Linux	48
6	Biblioteca pthread	49
6.1	Criar novas threads	49
6.2	Finalizar threads	49
6.3	Aguarda término de threads	49
6.4	Exemplo: Criação e finalização de threads	49
6.5	Cancelamento de threads	50
6.6	Exemplo: Cancelamento de threads	50
6.7	Paralelismo com Threads	51
7	Atividades	53
7.1	Atividades	53
8	Prática: Soma de Vetores Sequencial	54
9	Prática: Soma de Vetores Paralelo: Versão estática	55
10	Prática: Soma de Vetores Paralelo: Versão dinâmica	57
11	Relatório	60
12	Leitura Recomendada	61
12.1	Leitura Recomendada	61
13	Referências	62
13.1	Referências	62
13.2	Leitura Recomendada	62
14	Próximas Aulas	63
14.1	Próximas Aulas	63
14.2	Livros Texto	63
14.3	<i>Word Cloud</i>	66
15	Referências	67
15.1	Referências	67

15.2 William	67
15.3 Padrão POSIX	67
16 Implementação de Threads	69
16.1 Implementação de Threads em Linguagens de Programação	69
16.2 Implementação de Paralelismo com Cilk	73
16.2.1 Primitivas principais da API Pthreads	74
16.3 Suporte do Hardware e Sistema Operacional	75
16.4 Bibliotecas e Implementações	76
16.5 Exemplos Práticos com Pthreads	76
16.5.1 Exemplo 1 – Hello World com threads	76
16.5.2 Exemplo 2 – Passagem de parâmetros	77
16.5.3 Exemplo 3 – Execução paralela de múltiplas threads	78
16.5.4 Exemplo 4 – Uso de mutex (trava)	79
16.5.5 Exemplo 5 – Variáveis condicionais	80
16.6 Modelo de Programação e Execução	82
16.7 Considerações Finais	83
16.7.1 Prós e Contras	83
16.8 Referências	85
17 Introdução ao OpenMP: Diretivas de Compilação, Variáveis de Ambiente e Funções de Biblioteca	88
17.1 Introdução	88
17.2 Padrão OpenMP, compiladores, bibliotecas e Implementações	89
17.3 Diretivas de Compilação	89
17.4 Objetivos	91
18 Abordagem do OpenMP	92
18.1 Vantagens do uso de OpenMP	92
18.2 Padrão OpenMP	92
18.3 OpenMP	93
18.4 Expansão das Diretivas de Compilação	94
18.5 Sintaxe dos Elementos de OpenMP para C/C++.	94
18.6 Implementações OpenMP	94
19 Diretivas de Compilação	95
19.1 Diretivas de Compilação: Construtor Paralelo	95
20 Variáveis de Ambiente	97
20.1 Variáveis de Ambiente do OpenMP	97
21 Diretivas de Compilação	99
21.1	99
21.2 Estudo das Diretivas de Compilação	99

21.3 Regiões paralelas: construtor <code>parallel</code>	99
21.4 Outras Flags Úteis	102
21.5 Considerações Finais	104
21.6 <i>Word Cloud</i>	105
Referências	106
22 Diretivas de Compilação para Regiões Paralelas	108
22.1 Introdução	108
22.2 Diretivas de Compilação para Regiões paralelas: construtor <code>parallel</code>	108
22.3 Cláusulas do Construtor <i>parallel</i>	111
22.4 Cláusula <i>num_threads</i> do Construtor <i>parallel</i>	114
22.5 Cláusula <i>if</i> do Construtor <i>parallel</i>	116
22.6 Cláusulas <i>default</i> , <i>private</i> , <i>shared</i> do Construtor <i>parallel</i>	118
22.7 Cláusula <i>reduction</i> do Construtor <i>parallel</i>	119
22.8 Prática: Exercício de Implementação	120
22.9 Relatório	120
22.10 Considerações Finais	120
22.11 <i>Word Cloud</i>	121
Referências	122
Referências	124

Prefácio

Bem-vindo

Este livro reúne o material das aulas da disciplina **PPGCC18 – Computação Paralela e Distribuída**. Nele apresentamos capítulos com assuntos importantes na Computação Paralela, como **Pthreads**, **Vetorização**, **OpenMP** e **CUDA**, **OpenACC** e **OpenMPI**.

Licença

Este material é **livre para o uso** e está sobre a licença [Creative Commons Attribution-NonCommercial-NoDerivs 4.0](https://creativecommons.org/licenses/by-nc-nd/4.0/).

Este é um livro produzido com a ferramenta Quarto.

Para aprender mais sobre livros Quarto visite <https://quarto.org/docs/books>.

1 Introdução à Computação Paralela

Resumo: Neste capítulo são apresentados conceitos, técnicas e mecanismos de suporte do *hardware* e do Sistema Operacional à Computação Paralela. Falamos um pouco sobre as técnicas de melhorias de desempenho ligadas à arquitetura dos processadores até chegarmos ao *multicore*. Do suporte do Sistema Operacional à implementação de *threads*.

1.1 Introdução

As melhorias de desempenho no decorrer das últimas décadas passaram pelo aumento da velocidade de *clock*, otimizações do conjunto de instruções, hierarquia de memória e níveis de *cache* e otimizações do código com compilação.

Nos dias atuais, as frentes de pesquisa e desenvolvimento buscam soluções voltadas para a computação sustentável, com técnicas e mecanismos de gerenciamento de energia. Plataformas modernas cada vez heterogêneas, com dispositivos de computação cada vez mais distintos, como Unidades de Processamento Gráfico (GPUs), arranjos de coprocessadores e FPGAs. O que nos leva à *Computação Paralela e Heterogênea*.

A obtenção de melhorias de desempenho passam atualmente pelo uso de *diretivas de compilação*, para anotações em código legado e ferramentas como compiladores paralelizantes que com base em diretivas de compilação ou pelo uso de técnicas de paralelização automática, geram versões paralelas dos códigos.

Nesse esforço, fabricantes fornecem bibliotecas otimizadas para suas plataformas ou dispositivos que permitem uma modernização do código, pela substituição de trechos de código legado ou versões ingênuas de algoritmos, por chamadas a versões otimizadas.

1.2 Suporte do Hardware

Tecnologias mais rápidas possibilitam projetos mais eficientes. Partindo-se de um modelo básico de computação foram sendo adicionados aos projetos arquiteturais recursos para a melhoria de desempenho. Entre esses recursos, estão Hierarquia de Memória (Cache) visando mitigar a diferença de tecnologias na construção de CPUs e Memórias. A divisão do ciclo de execução de

uma instrução em estágios de um *pipeline*, que levou a técnicas de previsão de desvios para evitar penalidades de busca de instruções em caminhos errados.

As Arquiteturas Superescalares buscam o *Paralelismo em Nível de Instrução (ILP)* com escalonamento dinâmico de instruções (Tomasulo 1967) e o mecanismos para a *execução fora de ordem*. Porém, ainda que processem múltiplas instruções ao mesmo tempo, elas ainda sofrem com ineficiência e subutilização de recursos. O número de instruções executadas por ciclo (IPC) não é muito alto, isto é, *hardware* fica ocioso.

Para solucionar isso e maximizar o desempenho, surgiu a **execução simultânea de diferentes tarefas** (como o *Simultaneous Multithreading - SMT*), uma abordagem que otimiza o uso do processador ao aproveitar tanto o paralelismo das instruções quanto o das próprias tarefas, *Paralelismo em Nível de Tarefas (TLP)*. A execução de múltiplas tarefas diferentes, isto é, aplicações diferentes, garante uma ocupação mais efetiva dos recursos do processador que ficariam ociosos, podem ser utilizados pelas instruções de outras tarefas.

A multiplicidade de núcleos trazida pelos processadores *multicore* garante um paralelismo em núcleos de processamento diferentes.

1.2.1 Arquiteturas Superescalares

As arquiteturas superescalares expandiram a capacidade em largura através de *pipelines* distintos para cada unidade funcional, fazendo com que múltiplas instruções sejam buscadas, decodificadas e despachadas para a execução. Mesmo as arquiteturas superescalares, ainda não atingem níveis de paralelismo desejados e acabam sendo subutilizadas ou utilizadas ineficientemente.

Diante da necessidade de melhoria no desempenho e da utilização mais eficiente dos recursos, surgiu a idéia de remessa e execução de instruções de tarefas distintas, caracterizando a execução simultânea de tarefas com o objetivo de intensificar a utilização de recursos e explorar o grau de paralelismo existente entre essas instruções e ainda o grau de paralelismo existente entre tarefas.

O conceito de superescalar é implementado na maioria dos processadores modernos, dando-lhes a capacidade de explorar o paralelismo no nível de instruções (ILP). Em uma arquitetura superescalar típica, o processador busca e decodifica várias instruções tentando garantir um fluxo contínuo de instruções prevendo antecipadamente os resultados de instruções de desvio. As instruções são analisadas e as dependências de dados e de controle são reduzidas, e instruções são despachadas e distribuídas para as estações de reserva associadas e na sequência são emitidas para a execução em suas respectivas unidades funcionais (Smith and Sohi 1995).

A execução em paralelo baseia-se na disponibilidade de operandos ao invés de restringir-se à sequência original do programa, proporcionando um escalonamento dinâmico de instruções, no qual os operandos pendentes em outras instruções são atualizados à medida em que são calculados, permitindo que a execução das instruções que aguardam esses operandos possa então prosseguir. A arquitetura utiliza-se de uma estrutura de reordenação, um *buffer* ou uma

fila, para que após a execução e obtenção dos resultados, o estado do programa seja finalizado na mesma ordem estabelecida pelo código, mantendo assim a semântica do código original.

A execução fora de ordem é obtida sobre as instruções que estão com os operandos prontos, não respeitando a ordem original do código, mas sim o fluxo de dados. Esse paralelismo é implícito no código e ocorre de forma transparente ao usuário, não sendo necessário recompilar as aplicações existentes para a arquitetura alvo.

A execução de instruções em paralelo envolve questões relacionadas à dependência de controle e de dados entre as instruções, ao hardware para suportar a execução de múltiplas instruções em paralelo, às estratégias para determinar quando uma instrução está pronta e à passagem de valores entre instruções que estão em execução.

Instruções de desvio condicional podem levar o fluxo de execução a um caminho tomado pela decisão das técnicas de previsão de desvios, de forma que as instruções executadas desse caminho são executadas em um contexto especulativo, podendo mais tarde serem descartadas ou efetivadas, dependendo da conformidade da previsão. As operações realizadas especulativamente são efetivadas no estado real da máquina somente quando as previsões de desvios são confirmadas. O controle da visibilidade desse novo estado e da manutenção da aparência de execução sequencial sem prejuízos para a semântica do código original é uma tarefa complexa, exigindo para isso um hardware também complexo.

As técnicas de previsão de desvio tentam prever o caminho a ser seguido pelo fluxo de execução, procurando minimizar os erros de forma que sempre instruções de um caminho correto sejam buscadas, decodificadas e executadas.

1.2.2 Arquiteturas SMT

A denominação *Simultaneous Multithreading* (SMT) teve origem na publicação de (Tullsen et al. 1995), embora o conceito tenha sido apresentado por idéias semelhantes em trabalhos anteriores como a arquitetura *Simultaneous Instruction Issuing* (Hirata et al. 1992) e a arquitetura chamada *Multistreamed Superscalar* (Yamamoto et al. 1994), ambos com a capacidade de executar instruções provenientes de diferentes fluxos simultaneamente.

As arquiteturas SMT têm como objetivo a execução simultânea de tarefas, remetendo à execução instruções de diferentes tarefas durante o mesmo ciclo do processador. Cada tarefa detém, durante a execução, o uso de alguns recursos que constituem o seu contexto de execução, compartilhando diversos outros com outras tarefas em execução. O contexto de cada tarefa é formado pelo conteúdo dos registradores que estão sendo utilizados, pelo seu contador de programa e pelos recursos que estão sob seu domínio.

Através do embaralhamento das instruções das diversas tarefas em execução simultânea, a arquitetura explora o paralelismo em nível de tarefas (TLP), em adição ao ILP, o qual é típico das arquiteturas superescalares convencionais. Esta característica proporciona melhor

aproveitamento dos recursos existentes, pois aqueles que estariam ociosos com a execução de uma única tarefa poderão ser aproveitados pela execução simultânea de outras tarefas.

A execução de tarefas simultâneas garante um paralelismo maior em relação à execução convencional, permitindo uma exploração maior dos recursos do *hardware*. A unidade de busca, entre outros recursos é compartilhada pelas diversas tarefas, permite que as instruções das tarefas sejam buscadas e inseridas no caminho de execução, onde a coexistência de vários contextos permite aproveitar melhor as oportunidades de execução enquanto as tarefas concorrentes alternam seus momentos de *cpu-bound* e *io-bound*. A Figure 1.1 apresenta o projeto arquitetural de uma arquitetura SMT.

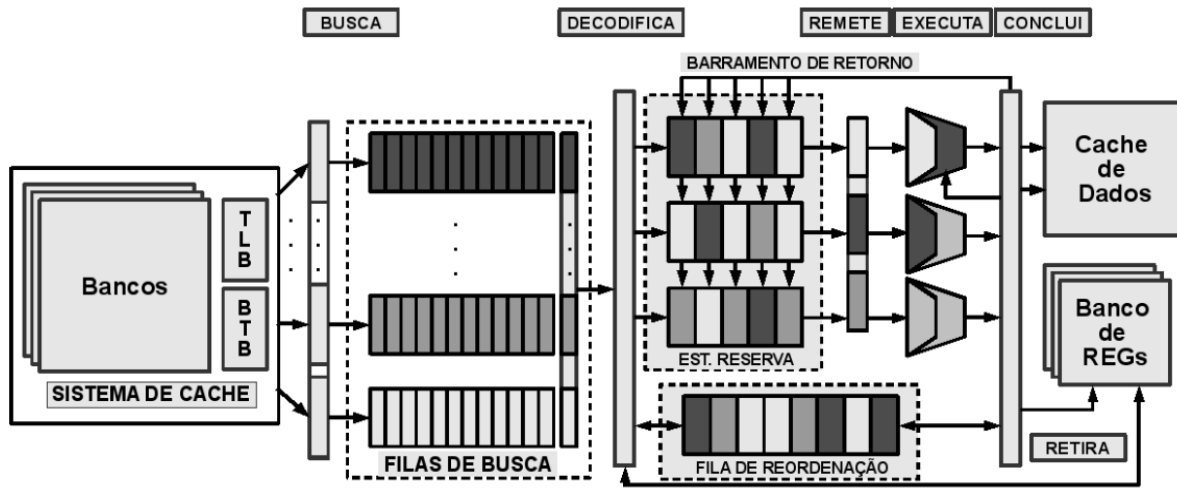


Figure 1.1: SMT

A Figure 1.2 apresenta a ideia geral da utilização de recursos na execução em um núcleo SMT, fazendo um comparativo com a utilização dos recursos na execução em um processador de núcleo simples e em um processador de núcleo duplo. Os recursos são representados pelos quadrados, as linhas representam os recursos que podem ser utilizados em paralelo e as colunas mostram a utilização efetiva de cada recurso no decorrer do tempo.

O compartilhamento de recursos e estruturas internas do processador pode ser feito de diversas maneiras e da forma que melhor se adequar ao projeto arquitetural. Por exemplo, a implementação da Intel, denominada tecnologia *Hyper-Threading*, implementa um *pipeline* para a execução simultânea de duas tarefas através da duplicação do estado arquitetural, como sendo dois processadores lógicos, porém a execução das instruções é simultânea sobre os recursos compartilhados do mesmo processador físico (Marr et al. 2002), sendo vistos pelo SO como sendo dois processadores físicos.

A tecnologia *Hyper-Threading* (HT) da Intel (Marr et al. 2002) executa duas tarefas simultaneamente, enquanto que na proposta inicial do conceito SMT (Tullsen et al. 1995) foram

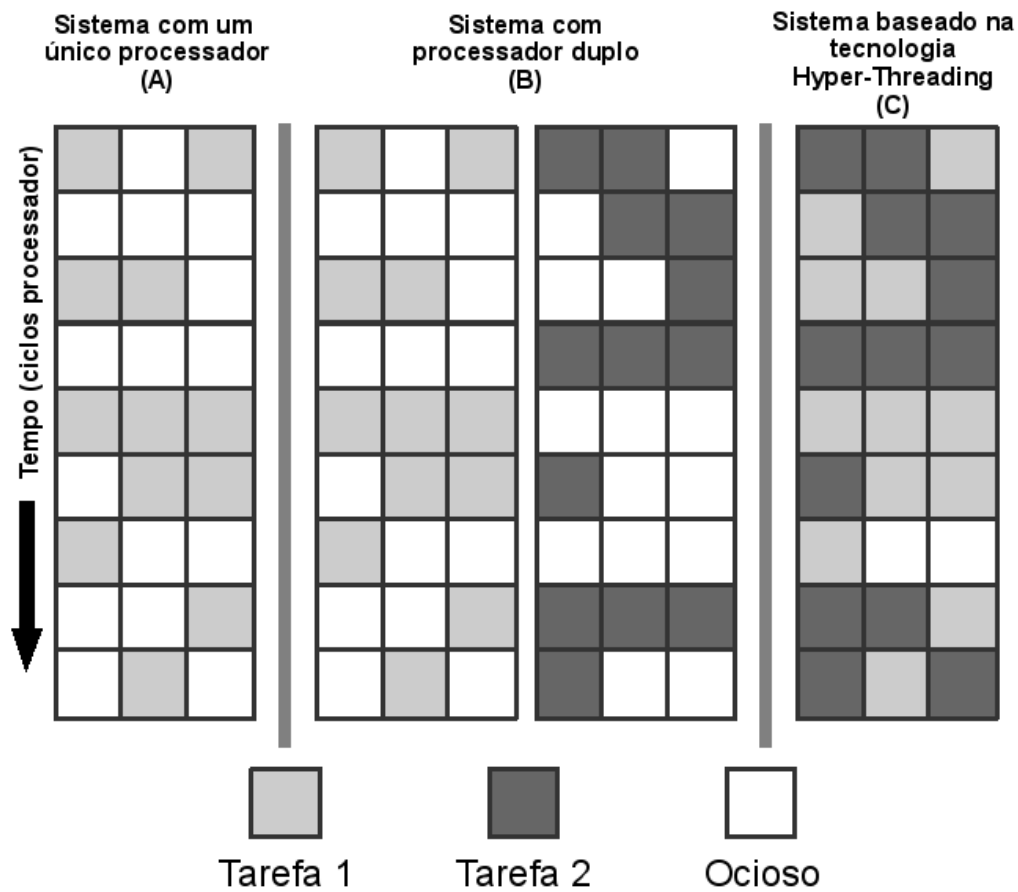


Figure 1.2: Comparativo de Execução dos Modelos Arquiteturais

realizados experimentos com até 8 tarefas, o que leva a crer que é alta a complexidade envolvida no controle e no compartilhamento de recursos.

A tecnologia HT faz com que um único núcleo físico seja visto como múltiplos núcleos lógicos, pois há uma cópia do estado arquitetural para cada processador lógico que compartilha os mesmos recursos físicos durante a execução. Esses núcleos lógicos são percebidos pelo Sistema Operacional (SO) como se fossem núcleos físicos, podendo escalonar processos ou tarefas para eles como se estivessem em um sistema multiprocessado.

A primeira implementação da HT foi feita para a família de processadores Intel *Xeon*, com dois processadores lógicos por processador físico. Sua implementação custou menos de 5% no aumento na área do *chip* e nos requisitos de consumo de potência, fornecendo uma melhoria considerável no desempenho.

O estado arquitetural mantido por cada processador lógico é formado pelos registradores de propósito geral e de controle, por registradores relacionados ao controlador de interrupção e alguns registradores de estado. Todos os outros recursos do núcleo físico são compartilhados, tais como *caches*, unidades de execução, previsores de desvios, lógica de controle e barramentos.

Cada processador lógico tem seu próprio controlador de interrupção (APIC), favorecendo a idéia de que para o SO é como se fosse um sistema com dois processadores físicos. Assim, interrupções enviadas via software (*trap*) ou via hardware são tratadas somente pelo processador lógico ao qual são endereçadas.

Um processador lógico pode ter seu funcionamento suspenso temporariamente enquanto falhas na *cache* e erros de previsão de desvios são tratados ou enquanto aguarda por resultados de instruções executadas anteriormente. Porém, devido ao particionamento dos recursos ser estático, cada tarefa tem seu conjunto particular de recursos. Em um cenário de execução com uma única tarefa, os recursos que seriam compartilhados, provavelmente serão subutilizados momentaneamente, mas este fato não poderá prejudicar as demais tarefas que vierem a ser executadas simultaneamente. Um dos objetivos de SMT é que contextos com uma única tarefa ativa tenham o mesmo desempenho com a tecnologia HT, como teriam em um processador sem essa tecnologia.

1.2.3 Arquiteturas de Múltiplos Núcleos com SMT

Com o surgimento de arquiteturas de múltiplos núcleos em um único processador físico ou *Chip Multiprocessors* (CMP) (Olukotun et al. 1996), houve certa redução nas pesquisas em SMT. Entretanto, devido aos limites de desempenho alcançado por esta nova abordagem, o desejo atual é que os processadores passem a combinar as duas estratégias em uma única arquitetura. Com isso, surgem as arquiteturas *Chip Multi-Threaded* (CMT) (Spracklen and Abraham 2005) como sendo processadores que suportam a execução simultânea de muitas tarefas via combinação do suporte a múltiplos núcleos com SMT. A aplicação de SMT em cada núcleo é feita para os mesmos fins que em processadores de núcleo único, melhor aproveitando

os recursos e as oportunidades de execução durante a latência de operações de memória. A Figure 1.3 apresenta o diagrama de um processador multinúcleos.

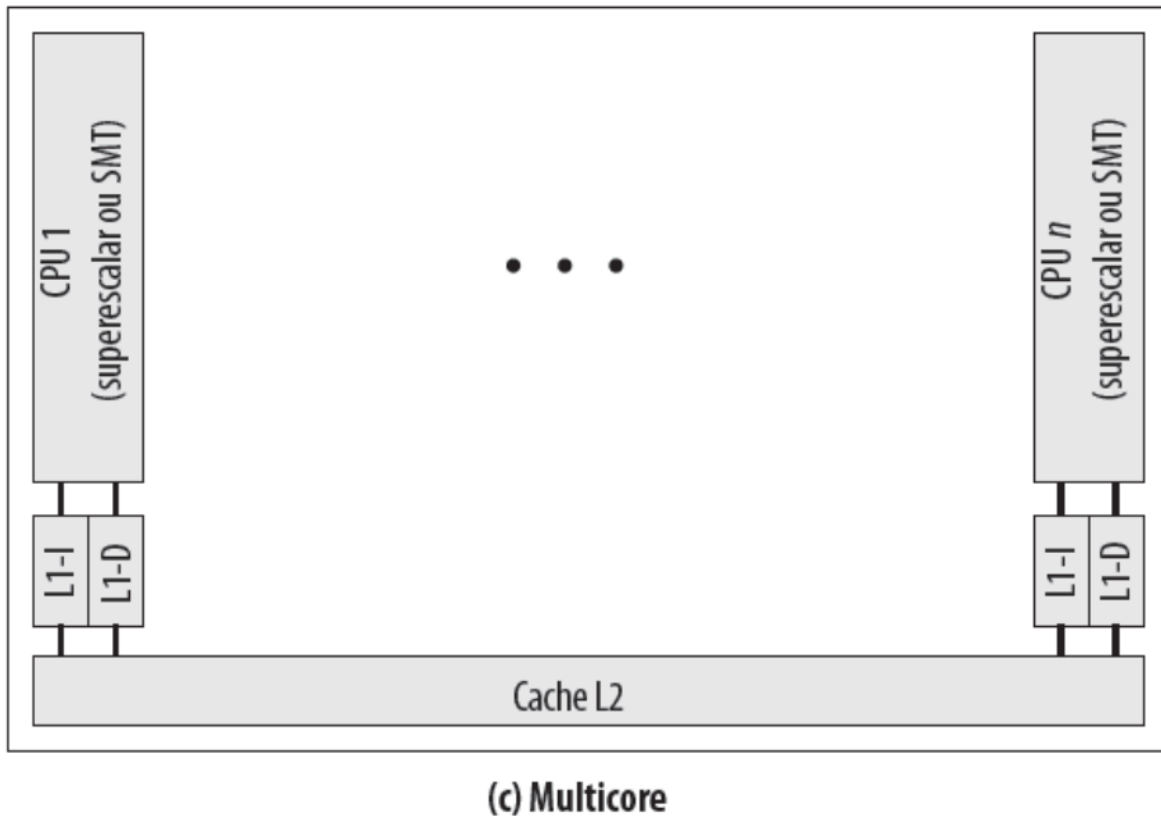


Figure 1.3: Multicore

A Figure 1.4 apresenta algumas imagens de *chips* de processadores que conhecidos.

1.3 Suporte do Sistema Operacional

O *Sistema Operacional (SO)* é responsável pelo gerenciamento de recursos disponíveis na máquina, gerenciamento e escalonamento dos processos.

Estamos em um ambiente multiprogramado e multiprocessado, mas nem sempre foi assim. Lá no começo, podíamos encontrar a seguinte situação: *uma máquina com apenas um processador, com apenas um núcleo, o que conhecemos por single core*. Além desses recursos de *hardware* restrito, se comparado com a situação atual, poderia também ter um Sistema Operacional monoprogramado, isto é, que executa apenas um programa por vez, exemplo, MS-DOS.

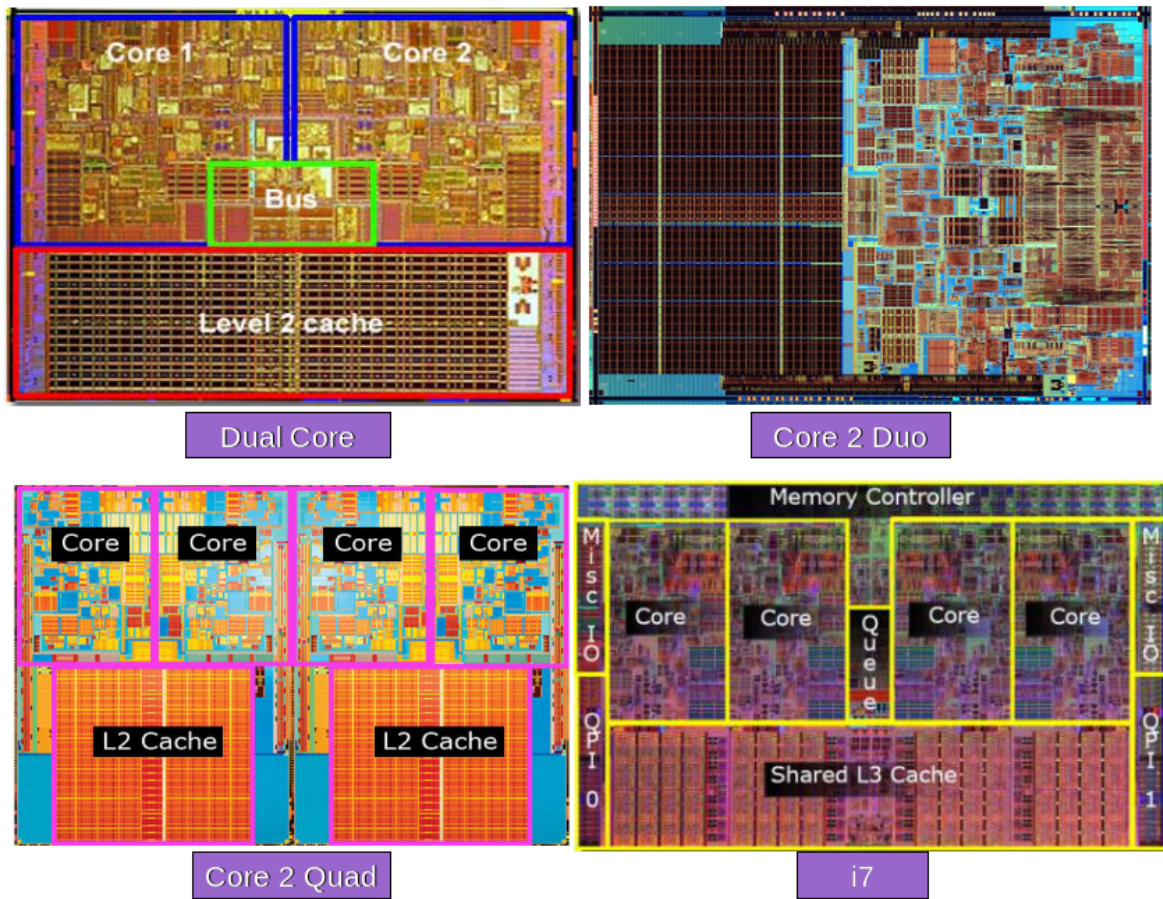


Figure 1.4: Multicore

Em um ambiente mais comum de hoje, de *hardware* com múltiplos elementos de processamento, de Sistema Operacional capaz de gerenciar a execução de múltiplos processos, temos a possibilidade de executar diversas aplicações ao mesmo tempo: *browser*, *player* de músicas, editor de textos, que antes tinham uma execução intercalada, através do compartilhamento de tempo e de uso de CPU.

Quando executamos um programa qualquer em uma máquina, um **processo** será criado. Então processos são programas executando em um computador. Cada processo tem um identificador, um *pid* e vai executar o código associado à sua imagem/segmento de código.

O Código 1.1 apresenta um exemplo de código que recupera os identificadores relacionados ao processo.

Código 1.1 exemplo-pid-ppid.c Recuperandos os identificadores do processo

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <pthread.h>

int main() {
    printf("Main PID: %d, PPID: %d, Main TID: %ld, TID: %ld\n",
        getpid(), getppid(), syscall(SYS_gettid), pthread_self());

    return 0;
}
```

getpid(): Retorna o PID do processo.

getppid(): Retorna o ID do processo pai.

syscall(SYS_gettid): Retorna o ID da *Thread* (TID) no *kernel* do Linux, que identifica unicamente uma *thread*.

pthread_self(): Retorna POSIX *thread* ID (*pthread_t*), convertido para *unsigned long* com (*unsigned long*)*pthread_self()*.

Todos os processos interagem com o Sistema Operacional por meio de chamadas ao sistema. Quando abrimos um terminal (*prompt* de comando) e executamos um comando qualquer, como por exemplo, o *pwd*. O processo do programa *terminal* é duplicado através da chamada de sistema *fork* e depois executa uma outra chamada de sistema da família *execv* para trocar a imagem do código do processo clonado, para que execute o código do programa desejado.

```
$ pwd
/home/rag
$
```

A chamada `fork` cria um processo filho que diferencia-se a partir do processo pai somente em seus PID e PPID. Arquivos abertos, sinais pendentes e todo o espaço de endereçamento do processo pai não são herdados pelo filho. Em caso de sucesso, o PID do processo filho é devolvido para o pai, e um 0 é retornado para o filho. Caso contrário, um `-1` será retornado no contexto pai, e nenhum processo filho será criado, e um erro será selecionado adequadamente. O Código 1.2 apresenta um exemplo do uso da chamada de sistema `fork` para a criação de um processo.

Código 1.2 `exemplo-fork.c` Exemplo do uso de FORK

```
#include <stdio.h>
#include <stdlib.h>

int main( ){
    int pai;
    int valor = 0;

    pai = fork( );
    if (pai){ /* este trecho é executado pelo pai */
        printf("Eu Sou o Processo Pai %d \n", pai);
        valor = 5;
        printf("Valor: %d \n", valor);
    }
    else { /* este trecho é executado pelo filho */
        printf("Eu Sou o Processo Filho %d \n", pai);
        valor = 10;
        printf("Valor: %d \n", valor);
    }
    exit(0);
}
```

A família de funções `exec` substituem a imagem do processo atual por uma imagem de código do novo processo que será executado. O Código 1.3 e o Código 1.4 ilustram o funcionamento da chamada `exec`.

A execução do exemplo do Código 1.2 no terminal vai apresentar o resultado que aparece na Figure 1.5.

Código 1.3 exemplo-exec-comment.c Exemplo do uso da chamada EXEC

```
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("Antes do exec\n");
    // execl("/bin/ls", "/bin/ls", "-r", "-t", "-l", (char *) 0);
    printf("Depois do exec\n");

    return(0);
}
```

i Nota

Note que a linha que contém a chamada à função `execl(...)` está comentada, assim as mensagens “*Antes do exec*” e “*Depois do exec*” serão escritas na tela.

Sem o comentário na linha do `execl(...)` conforme apresentado no Código 1.4, o que aconteceu? A mensagem “*Depois do exec*” não foi escrita na tela.

Código 1.4 exemplo-exec.c Exemplo do uso da chamada EXEC

```
#include <unistd.h>
#include <stdio.h>

int main() {
    printf("Antes do exec\n");
    execl("/bin/ls", "/bin/ls", "-r", "-t", "-l", (char *) 0);
    printf("Depois do exec\n");

    return(0);
}
```

A execução exemplo do Código 1.4 no terminal vai apresentar o resultado que aparece na Figure 1.6.

A segunda mensagem não foi escrita, pois no momento que a chamada `execl` é executada, a imagem de código do processo é trocada pelo código do comando `ls`, isto é, o código do processo original que imprimia as duas mensagens em torno da chamada `execl` deixa de existir.

```
[rag@backporting exemplo-exec]$ gcc exemplo-exec-comment.c -o exemplo-  
exec-comment.exe  
[rag@backporting exemplo-exec]$ ./exemplo-exec-comment.exe  
Antes do exec  
Depois do exec  
[rag@backporting exemplo-exec]$ █
```

Figure 1.5: Execução do exec

```
[rag@backporting exemplo-exec]$ gcc exemplo-exec.c -o exemplo-exec.exe  
[rag@backporting exemplo-exec]$ ./exemplo-exec.exe  
Antes do exec  
total 24  
-rw-r--r-- 1 rag rag 197 ago 20 10:10 exemplo-exec-coment.c  
-rw-r--r-- 1 rag rag 193 ago 20 10:12 exemplo-exec.c  
-rwxr-xr-x 1 rag rag 16024 ago 20 10:58 exemplo-exec.exe  
[rag@backporting exemplo-exec]$ █
```

Figure 1.6: Execução do exec

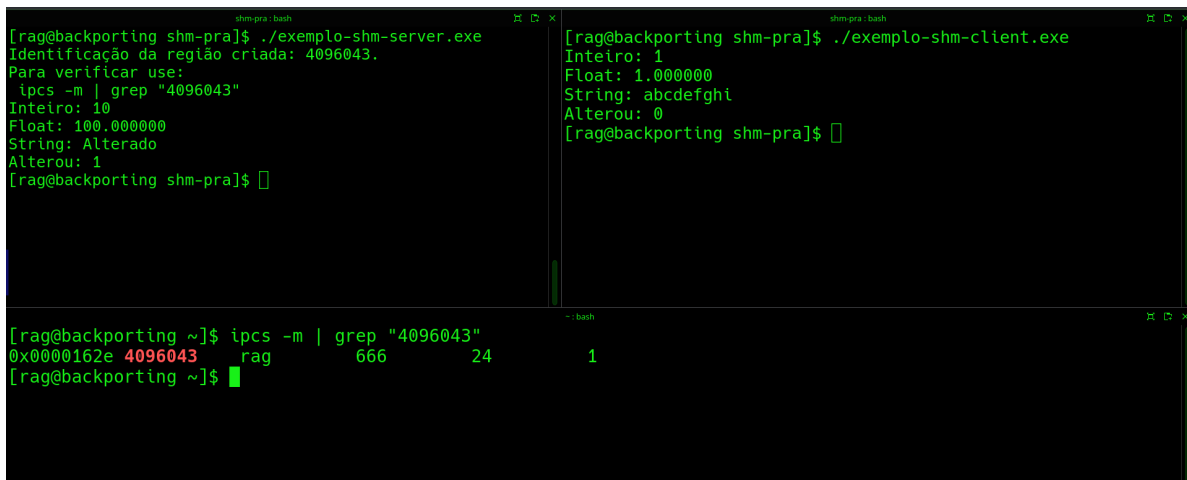
Além da criação de processos, outros recursos podem ser obtidos do gerenciamento feito pelo SO, utilizando chamadas de sistema. Para alocar memória, sua aplicação faz uma chamada à função `malloc(...)` da biblioteca, que por sua vez faz uma chamada de sistema para alocação de memória no segmento de dados, a **heap**. Para alocações cotínuas e pequenas a chamada `brk` / `sbrk` são utilizadas e deslocam o fim do segmento de dados do processo, já para alocações grandes, a chamada `mmap` é usada. Essa chamada cria um novo mapeamento de memória virtual independente no espaço de endereçamento do processo. No Linux, o limite padrão para disparar o `mmap` geralmente é de **128 KB** (configurável via `M_MMAP_THRESHOLD`). As regiões de memória alocadas são liberadas pela chamada à função `free()`.

Vimos que na criação de um novo processo, os processos pai e filho, são uma cópia do código, das variáveis, do espaço de endereçamento do momento da chamada `fork`. E que a partir da próxima linha cada processo segue um caminho independente. Através do chaveamento feito com o valor da variável `pai` que recebe o resultado da chamada `fork` podemos separar o que é executado pelo processo `pai` e o que será executado pelo `filho`.

Essa independência dos espaços de endereçamento garante a segurança do sistema e o isolamento entre os processos.

O que dificulta a execuções para a resolução cooperativa de problemas. Não há compartilhamento natural de dados entre os processos, é necessário utilizar algum mecanismo de comunicação entre processos (IPC – *Inter Processs Communication*). Os recursos disponíveis que podemos citar são: *pipe*, *fifo*, *shm*, *troca de mensagens*, *sockets* cada mecanismo com sua finalidade e suas limitações.

A Figure 1.7 apresenta o resultado da execução do exemplo de compartilhamento de memória.



```
[rag@backporting shm-pra]$ ./exemplo-shm-server.exe
Identificação da região criada: 4096043.
Para verificar use:
  ipcs -m | grep "4096043"
Inteiro: 10
Float: 100.000000
String: Alterado
Alterou: 1
[rag@backporting shm-pra]$

[rag@backporting shm-pra]$ ./exemplo-shm-client.exe
Inteiro: 1
Float: 1.000000
String: abcdefghi
Alterou: 0
[rag@backporting shm-pra]$

[rag@backporting ~]$ ipcs -m | grep "4096043"
0x0000162e 4096043    rag          666         24          1
[rag@backporting ~]$
```

Figure 1.7: Execução do SHM

No arquivo `<sys/shm.h>` não tem limite fixo do tamanho dos dados que podem ser compartilhados. O limite máximo depende dos parâmetros do kernel (como `SHMMAX` e `SHMALL` no

Linux) e da arquitetura do processador (32 ou 64 bits), podendo alcançar gigabytes ou frações significativas da memória RAM física e virtual disponíveis.

O `SHMMAX` (`/proc/sys/kernel/shmmax`) define o tamanho máximo (em bytes) permitido para um único segmento de memória compartilhada. Em sistemas modernos de 64 bits, o padrão costuma ser muito grande (próximo ao limite de um *unsigned long*). E o `SHMALL` (`/proc/sys/kernel/shmall`) é o limite total de páginas de memória compartilhada que podem ser alocadas simultaneamente no sistema inteiro. Para verificar qual o valor desse parâmetro basta executar os comandos:

```
$ cat /proc/sys/kernel/shmmax
18446744073692774399
$ cat /proc/sys/kernel/shmall
18446744073692774399

$ sysctl kernel.shmmax
kernel.shmmax = 18446744073692774399

$ sysctl kernel.shmall
kernel.shmall = 18446744073692774399
```

Em sistemas de 32 bits, o espaço de endereçamento por processo restringe o tamanho prático de um segmento a menos de 4 GB, enquanto sistemas de 64 bits suportam segmentos massivos limitados apenas pela RAM e swap. Caso seja necessário alterar esses limites ou verificar a configuração atual no seu sistema, o conteúdo do arquivo `/proc/sys/kernel/shmmax` pode ser alterado temporariamente pelo comando:

```
sudo sysctl -w kernel.shmmax=VALOR
```

Ou pode editar diretamente o arquivo `/etc/sysctl.conf` incluindo no final ou alterando valores existentes para

as variáveis `kernel.shmmax = VALOR_EM_BYTES` e `kernel.shmall = VALOR_EM_PAGINAS`. Para aplicar as mudanças sem a necessidade de reiniciar a máquina, o comando `sudo sysctl -p` pode ser executado.

1.4 Prática

1. Com base nos exemplos de `fork` e memória compartilhada `shm` altere o código do exemplo `vectoradd` para que soma de vetores seja feita por mais processos.

1.5 Considerações Finais

Neste Capítulo revisamos conceitos sobre suporte do *hardware* e do Sistema Operacional ao paralelismo. A criação de múltiplos processos e os mecanismos de comunicação entre processos. Em específico o compartilhamento de memória entre os processos independentes para a resolução de problemas cooperativos.

1.6 Word Cloud



Referências

Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. 2007. *Using OpenMP: Portable Shared Memory Parallel Programming (Scientific and Engineering Computation)*. The MIT Press.

- Clang, Site. 2013. “Clang Site @ONLINE.” <http://clang.llvm.org/>.
- GCC. 2020. *GCC, the GNU Compiler Collection*. <http://gcc.gnu.org/onlinedocs>.
- GNU Libgomp. 2015. *GNU libgomp, GNU Offloading and Multi Processing Runtime Library documentation (Online manual)*. <https://gcc.gnu.org/onlinedocs/libgomp/>.
- Hirata, Hiroaki, Kozo Kimura, Satoshi Nagamine, et al. 1992. “An Elementary Processor Architecture with Simultaneous Instruction Issuing from Multiple Threads.” *Proceedings of the 19th Annual International Symposium on Computer Architecture* (New York, NY, USA), ISCA '92, 136–45. <https://doi.org/10.1145/139669.139710>.
- Intel. 2016. *Intel® OpenMP* Runtime Library Interface*. Intel. https://www.openmprtl.org/sites/default/files/resources/libomp_20160322_manual.pdf.
- Intel. 2019. *Intel® c++ Compiler 19.1 Developer Guide and Reference*. <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>.
- Marr, Deborah T., Frank Binns, David L. Hill, et al. 2002. “Hyper-Threading Technology Architecture and Microarchitecture.” *Intel Technology Journal* 6 (1): 4–15.
- Olukotun, Kunle, Basem A. Nayfeh, Lance Hammond, Ken Wilson, and Kunyung Chang. 1996. “The Case for a Single-Chip Multiprocessor.” *SIGPLAN Not.* (New York, NY, USA) 31 (9): 2–11. <https://doi.org/10.1145/248209.237140>.
- OpenMP API Site. 2020. *The OpenMP(r) API Specification for Parallel Programming*. <http://openmp.org>.
- OpenMP-ARB. 2011. *OpenMP Application Program Interface Version 3.1*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP3.1.pdf>.
- OpenMP-ARB. 2013. *OpenMP Application Program Interface Version 4.0*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>.
- OpenMP-ARB. 2014. *OpenMP Application Program Interface Examples 4.0.1*. OpenMP Architecture Review Board (ARB). http://openmp.org/mp-documents/OpenMP_Examples_4.0.1.pdf.
- OpenMP-ARB. 2015. *OpenMP Application Program Interface Version 4.5*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/openmp-4.5.pdf>.
- Smith, J. E., and G. S. Sohi. 1995. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE* 83 (12): 1609–24. <https://doi.org/10.1109/5.476078>.

- Spracklen, Lawrence, and Santosh G. Abraham. 2005. “Chip Multithreading: Opportunities and Challenges.” *Proceedings of the 11th International Symposium on High-Performance Computer Architecture* (USA), HPCA '05, 248–52. <https://doi.org/10.1109/HPCA.2005.10>.
- Standard, OpenACC. 2012. “OpenACC Directives for Accelerators Site @ONLINE.” March. <http://www.openacc-standard.org/>.
- Tomasulo, R. M. 1967. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units.” *IBM Journal of Research and Development* 11 (1): 25–33. <https://doi.org/10.1147/rd.111.0025>.
- Tullsen, Dean M., Susan J. Eggers, and Henry M. Levy. 1995. “Simultaneous Multithreading: Maximizing on-Chip Parallelism.” *SIGARCH Comput. Archit. News* (New York, NY, USA) 23 (2): 392–403. <https://doi.org/10.1145/225830.224449>.
- Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky. 1994. “Performance Estimation of Multistreamed, Superscalar Processors.” *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences* 1: 195–204. <https://doi.org/10.1109/HICSS.1994.323172>.

```
/*
 * Fonte: http://www.cs.cf.ac.uk/Dave/C/node27.html
 */
/*
 * shm-client - client program to demonstrate shared memory.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>

#define SHMSZ (sizeof(tshare))

typedef struct tshare {
    int ii;
    float ff;
    char str[10];
    int alterou;
} tshare;

int main() {
    int shmid;
    key_t key;
    tshare *shm, *s;

    /* É preciso pegar o segmento nomeado como "5678", criado pelo servidor. */
    key = 5678;

    /* Localizando o segmento (locate). */
    if ((shmid = shmget(key, SHMSZ, 0666)) < 0) {
        perror("Erro ao tentar acessar o segmento de shm (shmget).");
        exit(1);
    }

    /* Acoplamento do segmento ao espaço de endereços de dados. */
    if ((shm = shmat(shmid, NULL, 0)) == (void *)-1) {
        perror(
            "Erro ao acoplar o segmento ao espaço de dados do programa (shmat).");
        exit(1);
    }

    /* Lendo o que o servidor colocou na região de memória. */
    s = shm;
    printf("Inteiro: %d\n", s->ii);
    printf("Float: %f\n", s->ff);
    printf("String: %s\n", s->str);
    printf("Alterou: %d\n", s->alterou);

    /* Modificando o primeiro caracter do segmento para '*', indicando que os
```

Código 1.6 exemplo-shm-server.c Exemplo do uso Memória Compartilhada

```
/*
 * Fonte: http://www.cs.cf.ac.uk/Dave/C/node27.html
 */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/types.h>
#include <unistd.h>

/* Tamanho do segmento compartilhado. */
#define SHMSZ (sizeof(tshare))

typedef struct tshare {
    int ii;
    float ff;
    char str[10];
    int alterou;
} tshare;

int main() {

    int shmid;
    key_t key;
    tshare *shm, *s;

    /* Nome do segmento de memória compartilhada "5678". */
    key = 5678;

    /* Cria o segmento compartilhado. */
    if ((shmid = shmget(key, SHMSZ, IPC_CREAT | 0666)) < 0) {
        perror("Erro ao tentar criar o segmento de shm (shmget).");
        exit(1);
    }

    printf("Identificação da região criada: %d. \nPara verificar use:\n ipcs -m "
           "| grep \"%d\"\\n",
           shmid, shmid);

    /* Acoplamento do segmento criado ao espaço de dados. */
    if ((shm = shmat(shmid, NULL, 0)) == (void *)-1) {
        perror(
            "Erro ao acoplar o segmento ao 25 espaço de dados do programa (shmat).");
        exit(1);
    }

    /* Colocando dados para o outro processo ler. */
    s = shm;
```

2 Multithreading com Pthreads

Resumo: Este capítulo introduz os conceitos fundamentais da computação paralela e da programação com threads utilizando a biblioteca Pthreads, padrão POSIX. Serão apresentados os princípios de execução concorrente, criação e gerenciamento de threads, sincronização com mutexes, e boas práticas para evitar condições de corrida e deadlocks.

2.1 Introdução

- **Thread:** parte de um programa. Processo Leve. Tarefa.
- Cada processo pode possuir uma ou múltiplas *threads*.

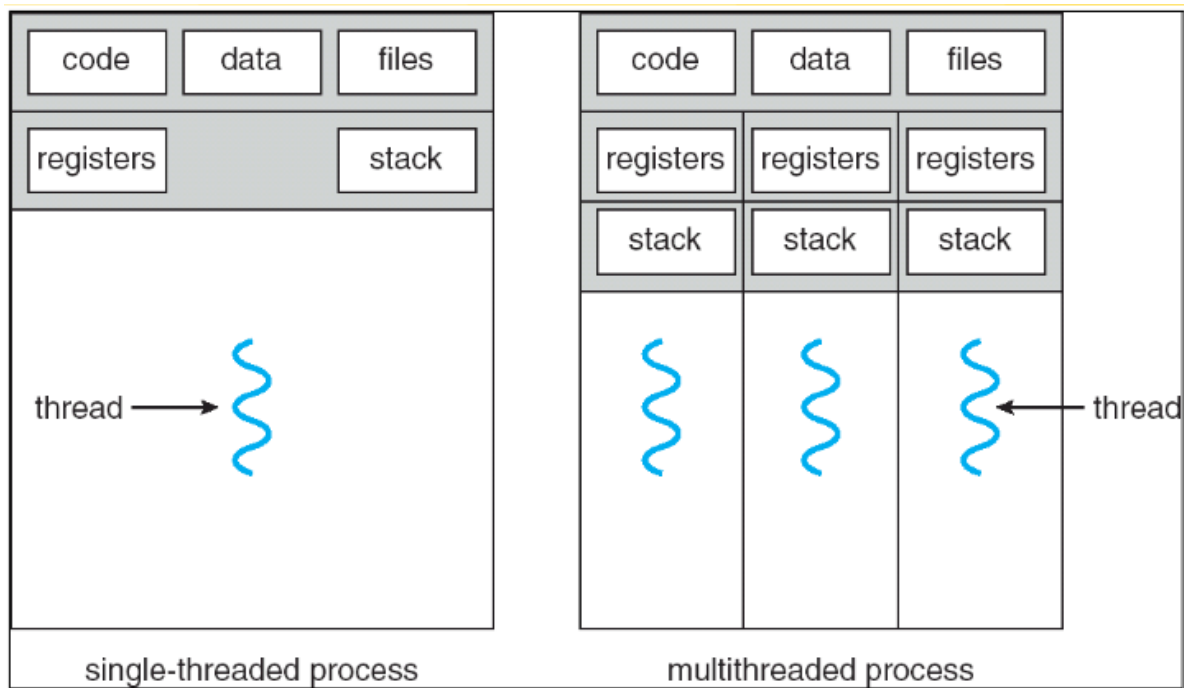


Figure 2.1: Processos e Threads

- **Exemplos:**
 - Um editor de textos, várias tarefas (*typing, spell checking, printing...*)
 - Um servidor web, cada requisição atendida por uma *thread*.
- **Em um computador convencional (*single core*):**
 - Cada processo gera pelo menos uma *thread* (fluxo de execução principal).
 - Um processo é escalonado para execução, logo uma *thread* está em execução por vez.
 - Quando uma *thread* pára (por exemplo, devido a um ***page fault***:
 - O estado da *thread* é guardado (registradores, ...)
 - * O estado de outra *thread* em espera é carregado no processador e inicia-se sua execução.
 - * O que é chamado de troca de contexto (***context switching***).
 - Para o usuário parece que todas as *threads* executam simultaneamente.
- Em um computador com processador *multicore* tem-se a possibilidade de paralelismo físico.
- **Concorrência:** Múltiplas tarefas estão logicamente ativas em um mesmo momento.
- **Paralelismo:** Múltiplas tarefas estão realmente ativas em um mesmo momento.

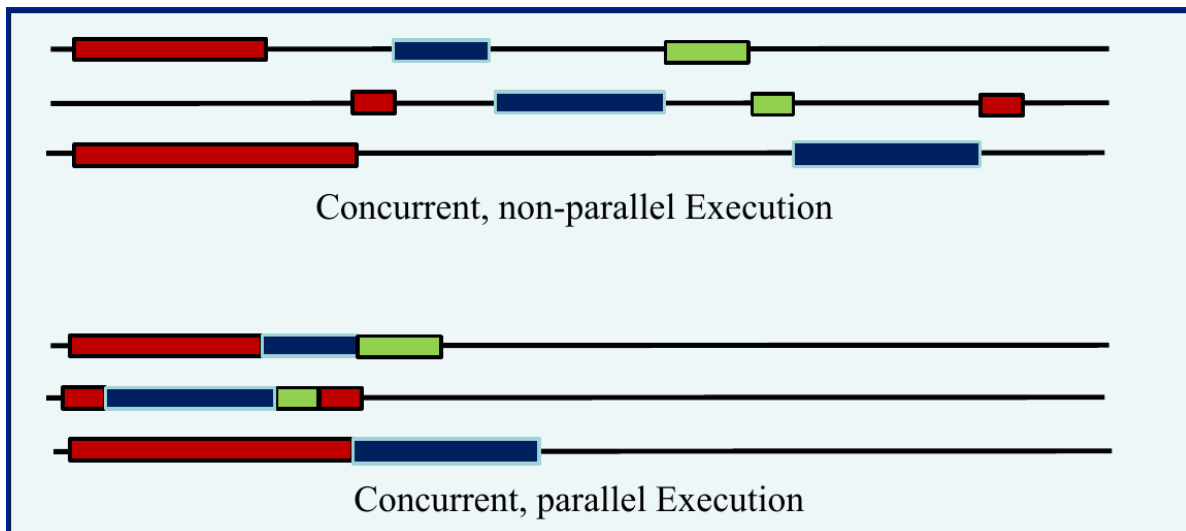


Figure 2.2: Concorrência e Paralelismo

3 Introdução

3.1 Introdução

[block]{Threads}

- O que é uma **thread**?
 - um fluxo em execução.
 - um processo leve (*lightweight process*).
 - unidade básica que o SO aloca tempo de processador¹.
- Por que usar threads?
 - Dividir tarefas de uma aplicação (ex: navegador Web).
 - Compartilhamento de espaço de endereçamento.
 - Tempo de criação e finalização.
 - Concorrência e paralelismo.

[/block]

¹<https://docs.microsoft.com/en-us/windows/win32/procthread/processes-and-threads>

3.2 Threads: Exemplo de uso - Processador de texto

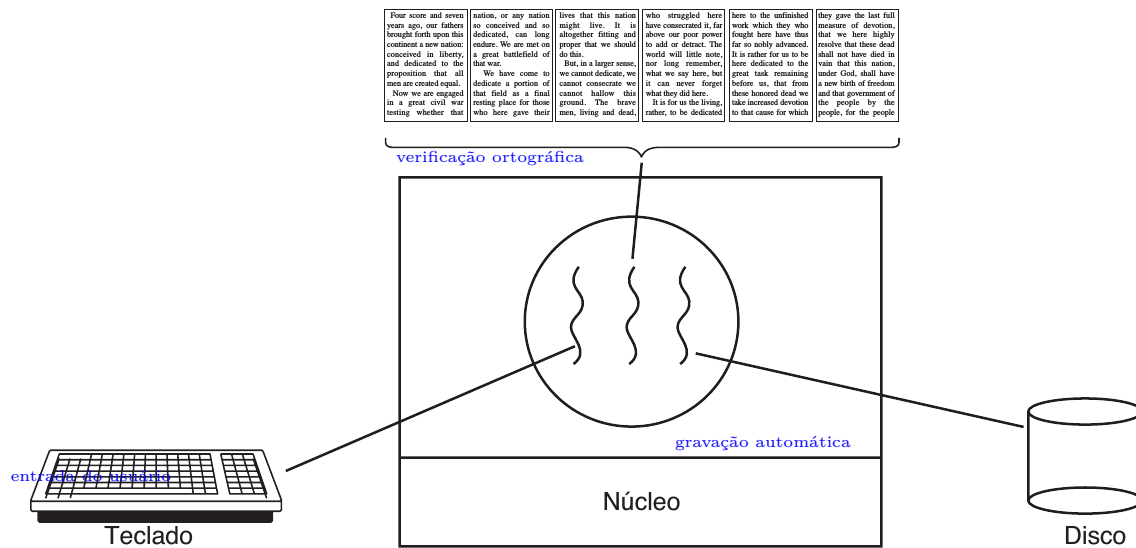


Figure 3.1: Processador de texto com três threads (Tanenbaum?).

3.3 Threads: Exemplo de uso - Servidor Web

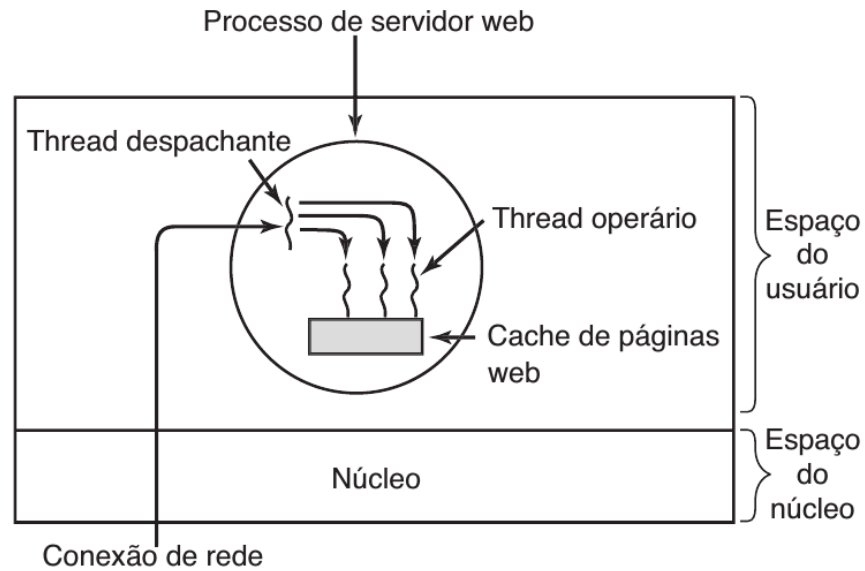


Figure 3.2: Servidor Web atendendo múltiplas requisições simultâneas (**Tanenbaum?**).

3.4 Threads: Implementação

- Modelos clássicos:

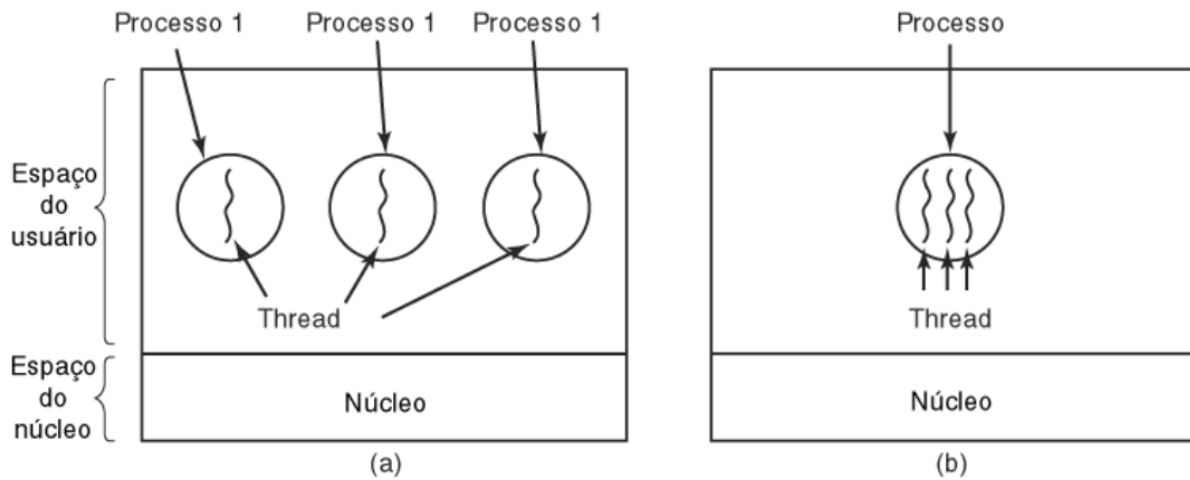


Figure 3.3: (a) Processo com uma thread. (b) Processo com três threads (**Tanenbaum?**).

- Quais recursos as threads compartilham em um processo?

```

[columns]
[column=.45]
[block]{Compartilhado entre threads}

    • Espaço de endereçamento.
    • Variáveis globais.
    • Arquivos abertos.
    • Processos filhos.
    • Alarmes pendentes.
    • Sinais e tratadores de sinais.
    • Estatísticas de execução.
    • Permissões de acesso.

[/block]
[/column]
[column=.45]
[block]{Exclusivo por thread}

    • Thread ID
    • Contador de programa.
    • Registradores.
    • Pilha (variáveis locais e endereços de retorno).
    • Estado.
    • Prioridade.

[/block]
[/column]
[/columns]
[framebreak]

```

- Modelos clássicos - Detalhes:

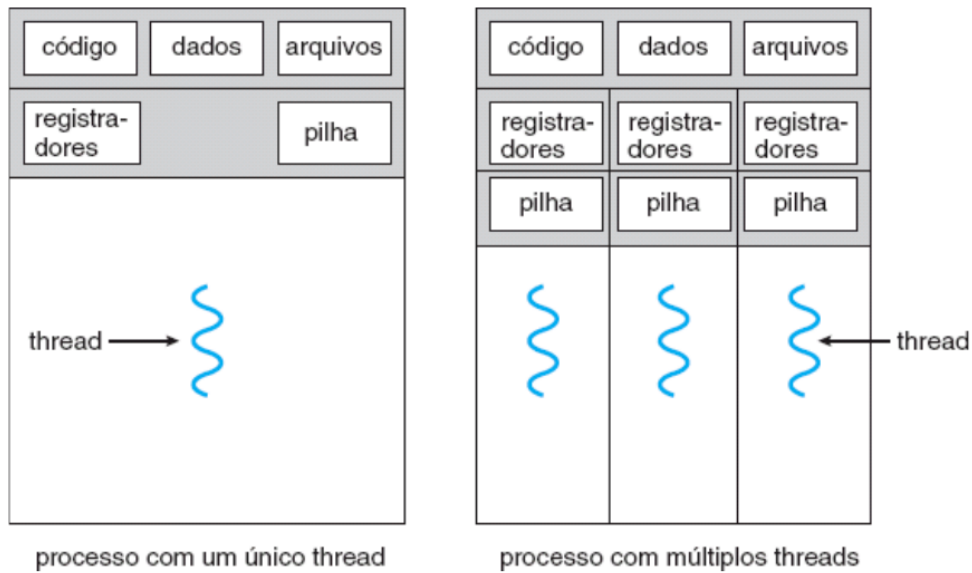


Figure 3.4: (a) Processo com uma thread. (b) Processo com três threads (**Silberschatz?**).

- Cada processo tem uma *thread* principal.
- Se um processo for *multithread* terá a *thread* principal e as *threads* que criar.

```

[framebreak]
[block]{Tipos de Implementação de Threads}

```

- Threads do Usuário (*User Level Threads*): Gerenciamento de thread realizado por biblioteca em nível de usuário.
- Threads do Núcleo (*Kernel Level Threads*): Gerenciamento de thread realizado diretamente pelo núcleo do SO. *Threads* admitidos diretamente pelo *kernel*.

[/block]

3.5 Threads: Implementação

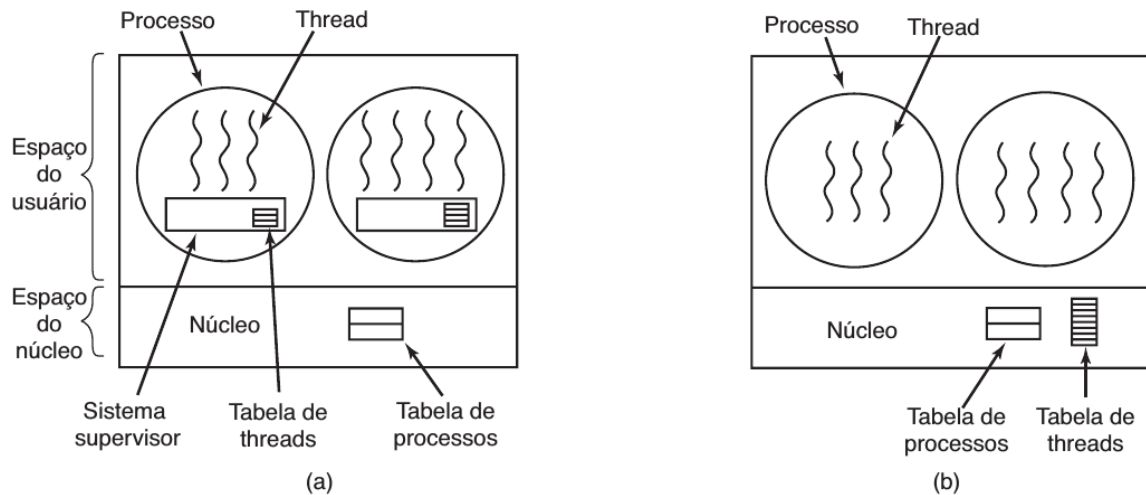


Figure 3.5: (a) Threads em nível de usuário. (b) Threads em nível de núcleo (**Tanenbaum?**).

3.6 Threads: Nível de Usuário

- Executam operações de suporte a threads no espaço do usuário.
- SO mapeia threads de um processo *multithread* para um único contexto de execução.
- **Vantagens:**
 - Escalonamento controlado pela biblioteca possibilita otimizar o desempenho.
 - Chaveamento de contexto em nível de usuário. A sincronização é realizada fora do núcleo, e isso evita chaveamento de contexto.
 - Portabilidade maior.
- **Desvantagens:**
 - Núcleo considera o processo *multithread* como um único thread.
 - Operações de E/S podem bloquear todas as *threads*. Desempenho pode ficar abaixo do ideal.
 - Escalonamento limitado a um único núcleo.

3.7 Threads: Nível de Núcleo

- Mapeia cada thread para o seu próprio contexto de execução.
- O núcleo reconhece cada thread individualmente.
- **Vantagens:**
 - paralelismo.
 - divisão justa do processador.
- **Desvantagens:**
 - custo do chaveamento de contexto.

- menor portabilidade. Sobrecarga decorrente do chaveamento de contexto e menor portabilidade em virtude de as APIs serem específicas ao sistema operacional.
- Os threads de núcleo nem sempre são a solução ideal para as aplicações.

3.8 Threads: Implementação Híbrida

- Consiste em mapear números arbitrários de *threads* de usuário para threads de núcleo.
- Multiplexação de threads de usuário sobre threads de núcleo.

[columns]

[column=0.5]

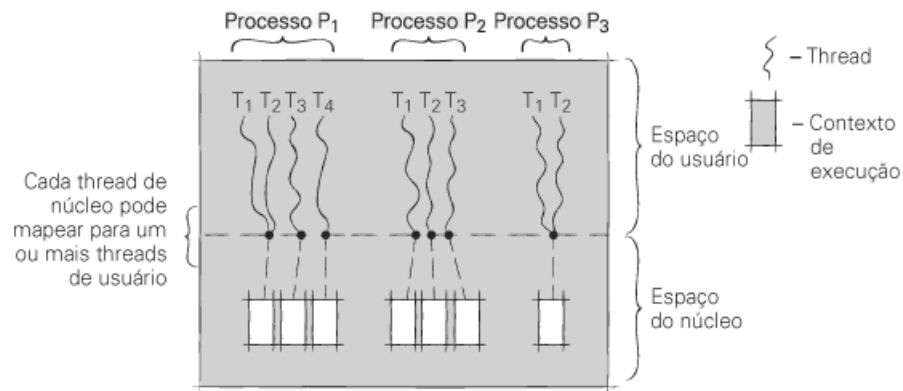


Figure 3.6: Mapeamento de Threads

[column=0.5]

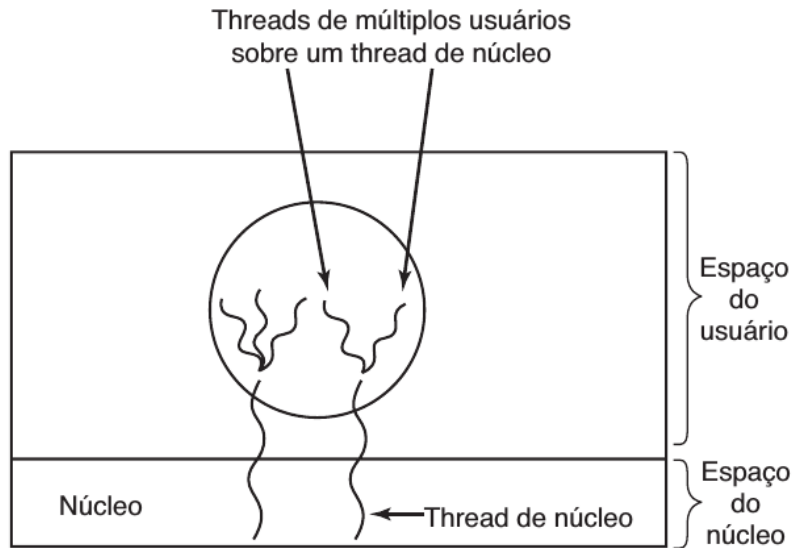


Figure 3.7: Threads de um processo mapeadas em duas threads de núcleo (Tanenbaum?).

[/columns]

[framebreak]

- Necessitam de comunicação para manter o número apropriado de threads de núcleo alocados à aplicação.
- O mecanismo de **ativações do escalonador** oferece **upcalls** – um mecanismo de comunicação do núcleo para a biblioteca de threads.

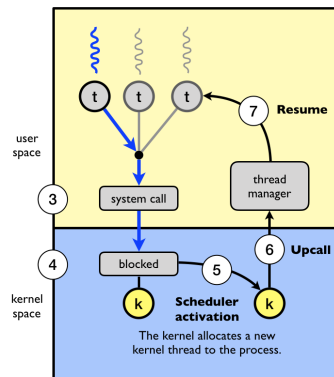


Figure 3.8: Ativação de escalonador via upcalls².

3.9 Threads: Modelos de mapeamento

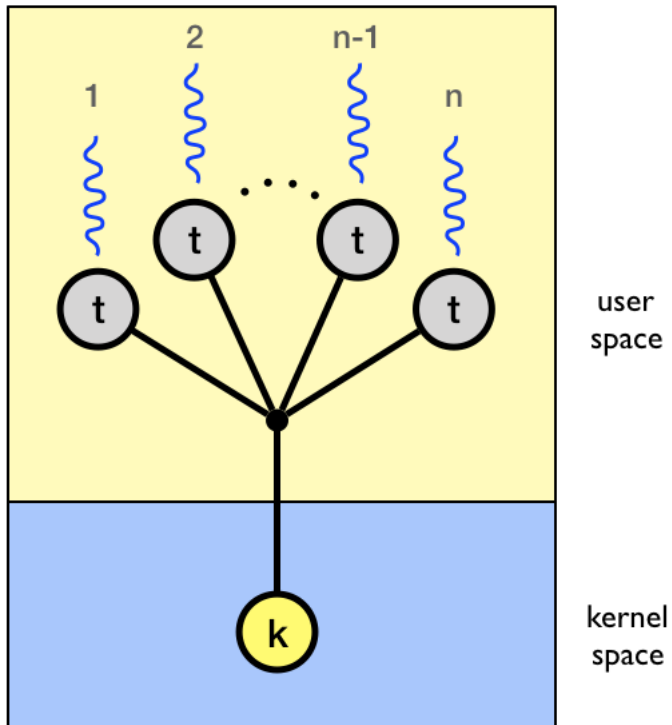
- **Muitos-para-um:** Solaris Green Threads, GNU Portable Threads.

²[OpenMP-ARB (2013)][OpenMP-ARB 2014]

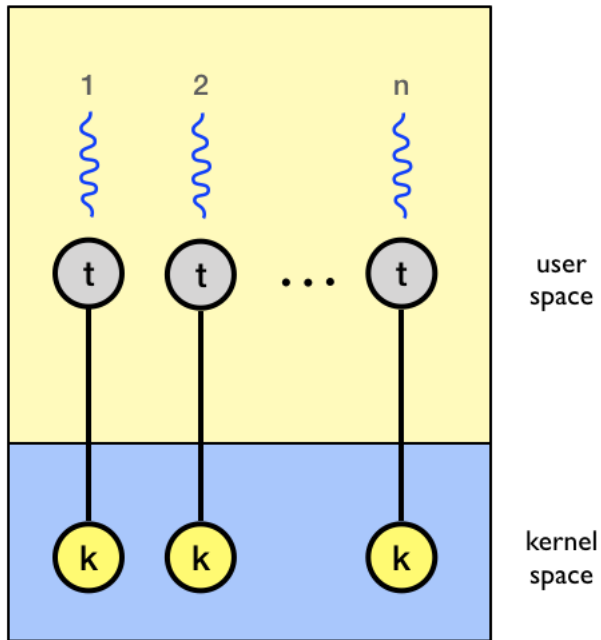
- **Um-para-um:** Windows NT/XP/2000, Linux, Solaris 9 em diante
- **Muitos-para-muitos (dois níveis):** Solaris <= 8, Windows NT/2000 *fiber*.
- **Modelo de dois níveis:** Semelhante a M:M, exceto por permitir que um *thread* do usuário seja ligado ao thread do kernel. IRIX, HP-UX, Tru64 UNIX, Solaris 8 e mais antigos.

[columns]

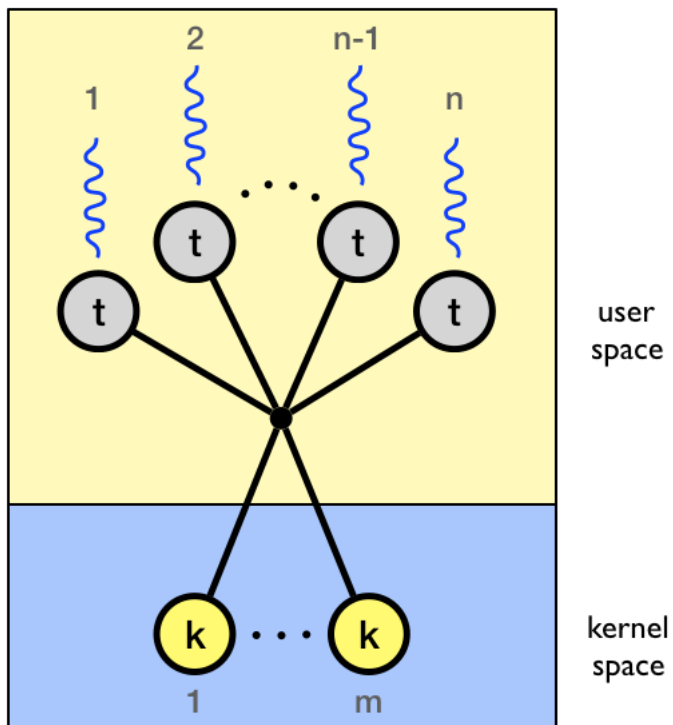
[column=.20]



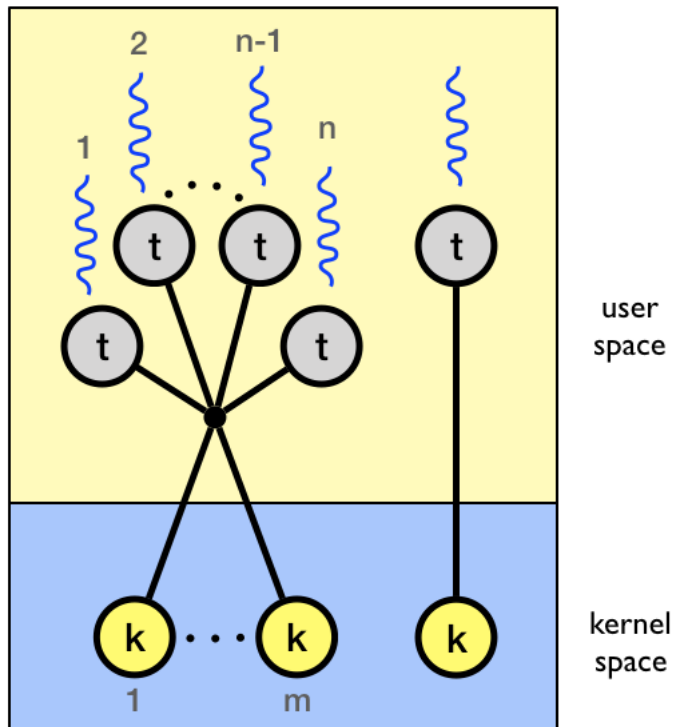
[column=.20]



[column=.20]



[column=.20]



[/columns]

Fonte: <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>

3.10 Threads: Tratamento de sinais

- Sinais são usados em sistemas UNIX para notificar um processo de que ocorreu um evento em particular;
- Opções para tratamento de sinais com threads:
 - Entregar o sinal a thread de destino no processo.
 - Entregar o sinal a todas threads no processo.
 - Entregar o sinal a threads específicas no processo.
 - Atribuir uma área específica para receber todos os sinais para o processo.
- As threads normalmente estão associadas a um conjunto de sinais pendentes que são emitidos quando são executadas.
- As threads podem mascarar todos os sinais, exceto aqueles que deseja receber.

3.11 Threads: Operações

[block]{Operações com threads}

- criação e finalização.
- sincronização (*join*, *blocking*).
- escalonamento (*{priority, yield}*).
- sinalização (síncrona ou assíncrona - processo externo).
- cancelamento (assíncrono, adiado).
- suspensão (parada) e ativação (retomada).
- reuso (*pools de thread*)

[/block]

3.12 POSIX

- Uma API padrão POSIX (IEEE 1003.1c) para criação e sincronismo de thread.
- A API especifica o comportamento da biblioteca de threads, a implementação fica para o desenvolvimento da biblioteca.
- Comum em sistemas operacionais UNIX (Solaris, Linux, Mac OS X).

3.13 POSIX e Pthreads

- Os threads que usam a API de thread POSIX são chamados de Pthreads.
- A especificação POSIX determina que os registradores do processador, a pilha e a máscara de sinal sejam mantidos individualmente para cada thread.
- A especificação POSIX especifica como os sistemas operacionais devem emitir sinais a Pthreads, além de especificar diversos modos de cancelamento de thread.

3.14 Threads no Linux

- Linux se refere a eles como tarefas ao invés de threads.
 - O Linux aloca o mesmo tipo de descritor para processos e tarefas.
 - Para criar tarefas-filhas, o Linux usa a chamada fork, baseada no Unix.
 - Para habilitar os threads, o Linux oferece uma versão modificada, denominada clone.
- A criação de thread é feita por meio da chamada do sistema clone()
 - Clone aceita argumentos que determinam os recursos que devem ser compartilhados com a tarefa-filha.
 - clone() permite que uma tarefa filha compartilhe o espaço de endereços da tarefa pai (processo)

[framebreak]

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

Figure 3.9: FLAGS

[framebreak]

- Diagrama de transição de estado de tarefa do Linux.

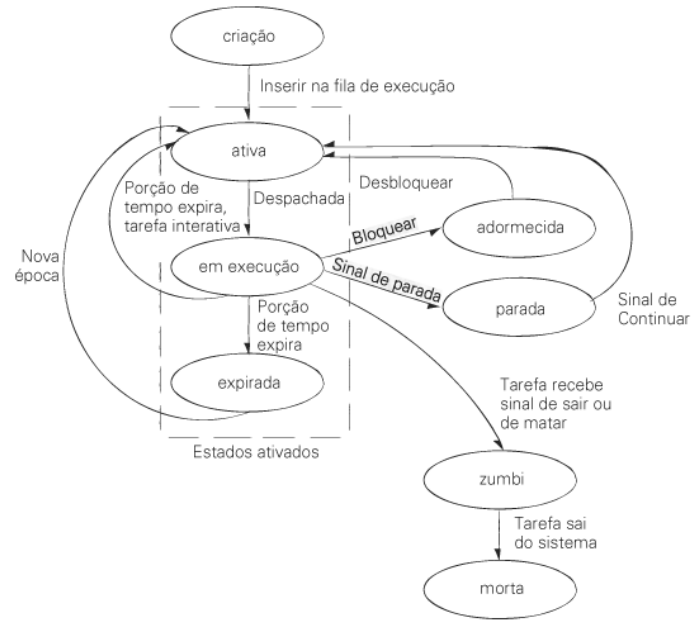


Figure 3.10: Thread Status

4 Implementação de Threads

4.1 Implementação de Threads em Linguagens de Programação

- Basicamente em todas as linguagens que suportam *threads* a implementação segue a ideia de se ter uma função que é o código a ser executado pela *threads* quando ela for criada.
 - Na linguagem C, podemos utilizar a biblioteca *Pthreads* do **GNU/Linux**.
 - Implementa a API padrão POSIX (IEEE 1003.1c) para criação e sincronismo de *threads*.
 - A API especifica o comportamento da biblioteca de threads, a implementação fica para o desenvolvimento da biblioteca.
 - A especificação POSIX determina que os registradores do processador, a pilha e a máscara de sinal sejam mantidos individualmente para cada *thread*.
 - Detalhando como os sistemas operacionais devem emitir sinais a *Pthreads*, além de especificar diversos modos de cancelamento de *thread*.
- Básico de Pthreads:

[exampleblock]{Criar uma nova thread}

```
int pthread_create(pthread_t * thread, const pthread_attr_t * attr,
void * (*start_routine)(void *), void *arg);
```

[/exampleblock]

pthread_create(&thread, attr, funcao, arg): Inicializa e executa a thread. Recebe a variável da thread, atributos (passar NULL usa o padrão), a função executável e o argumento a ser transmitido para a função.

Retorno: - 0 se conseguiu criar a thread. - Ou um código de erro. - Veja: http://linux.die.net/man/3/pthread_create

[framebreak]

[exampleblock]{Aguardar pelo término de outra thread}

```
int pthread_join(pthread_t th, void **thread_return);
```

pthread_join(thread, retval): Suspende a thread principal até que a thread especificada termine. Essencial para evitar que a função main() encerre antes do término dos processos filhos.

A função pthread_join retorna um valor do tipo int: 0 se a operação for bem-sucedida ou um código de erro (valor positivo diferente de zero) caso falhe.

Valores de Retorno da Função

0: Sucesso. A thread aguardada foi finalizada e seus recursos foram liberados.

EDEADLK: Deadlock detectado (por exemplo, se a thread tentar aguardar a si mesma).

EINVAL: A thread não é associável (joinable) — ocorre se ela foi criada como detached, se já passou por pthread_detach ou se outra thread já fez o join nela.

ESRCH: Nenhuma thread com o ID fornecido foi encontrada.

Diferença importante: O retorno da função pthread_join indica apenas se o processo de sincronização funcionou. Se você precisa obter o valor retornado pela thread finalizada (enviado via return ou pthread_exit), esse valor é gravado no ponteiro informado no segundo argumento (void **retval).

- A thread principal que cria outras threads deve aguardar o término das threads filhas.

- Pois pode acontecer da thread principal terminar antes.
- Veja: http://linux.die.net/man/3/pthread_join

[/exampleblock]

[block]{Terminar a *thread*}

```
void pthread_exit(void *retval);
```

`pthread_exit(retval)`: Usada para finalizar manualmente a thread corrente devolvendo um valor opcional.

[/block]

- Veja: http://linux.die.net/man/3/pthread_exit

[alertblock]{Cancelar a execução da thread}

```
int pthread_cancel(pthread_t thread);
```

[/alertblock]

- Veja: http://linux.die.net/man/3/pthread_cancel

[alertblock]{Enviar um sinal para a *thread*}

```
#include <signal.h>
```

```
int pthread_kill(pthread_t thread, int sig);
```

- Veja: http://linux.die.net/man/3/pthread_kill

[/alertblock]

- **Tutoriais:** <http://www.yolinux.com/TUTORIALS/LinuxTutorialPosixThreads.html> , <https://computing.llnl.gov/tutorials/threads>

[framebreak]

4.2 Exemplos

O Código 4.1 apresenta um código que recupera os identificadores (ids) do processo e das *threads* relacionadas a ele. A *thread* principal de execução do código do próprio processo e a *thread* criada com a chamada à função `pthread_create(...)`.

Para compilar o código dos exemplos, podemos utilizar o compilador `gcc` ou o `clang`, o programa deve ser compilado com a *flag* `-pthread` ou `-lpthread`, conforme apresentado no Código 4.2.

No processo de compilação podemos utilizar o automatizador `make` que irá executar os alvos definidos no arquivo `Makefile`, quando este estiver disponível no diretório de cada exemplo. O Código 4.3 apresenta os comandos para esse exemplo.

O Código 4.4 apresenta a compilação utilizando o `make`. Executar o `make` por padrão irá executar o comando de compilação que irá gerar o arquivo do executável do exemplo.

Mais sobre `Makefile` pode ser visto no site makefiletutorial.com.

A execução do binário do exemplo pode ser vista no Código 4.5, são listados os identificadores do processo criado para gerenciar nosso programa (PID), o identificador do processo pai (terminal, bash) que criou o processo do nosso programa (PPID) e os identificadores da *thread* no `_kernel` do `Linux` e na biblioteca `pthread`.

Código 4.1 exemplo-pid-ppid-threadid.c Recuperando os IDs

```
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <pthread.h>

void* thread_func(void* arg) {
    printf("Thread[%lu]:    Thread. PID: %d, PPID: %d, Kernel TID: %ld, Pthread TID: %ld\n",
        return NULL;
}

int main() {
    pthread_t tid;
    printf("Thread[%lu]: Principal. PID: %d, PPID: %d, Kernel TID: %ld, Pthread TID: %ld\n",

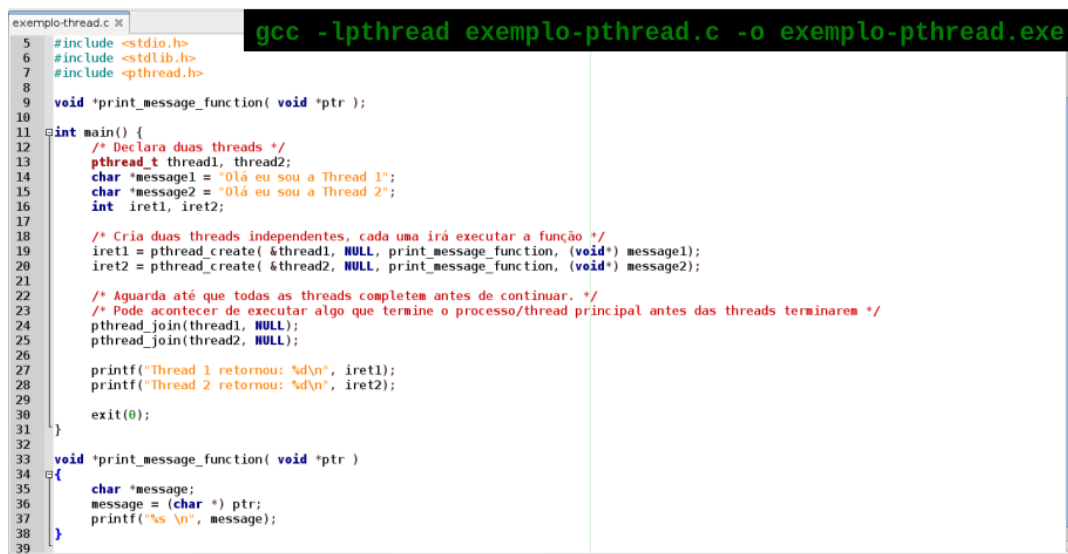
    pthread_create(&tid, NULL, thread_func, NULL);
    pthread_join(tid, NULL);

    return 0;
}
```

-
- Na linguagem C com *Pthreads*:

Código 4.2 Terminal Comandos de compilação e execução do programa.

```
[exemplo-pid-ppid-threadid]$ clang -lpthread exemplo-pid-ppid-threadid.c -o exemplo-pid-ppid-threadid-clang.exe
[exemplo-pid-ppid-threadid]$ ./exemplo-pid-ppid-threadid-clang.exe
Thread[140689466234688]: Principal. PID: 145090, PPID: 136279, Kernel TID: 145090, Pthread TID: 140689466234688
Thread[140689463572160]: Thread. PID: 145090, PPID: 136279, Kernel TID: 145091, Pthread TID: 140689463572160
[exemplo-pid-ppid-threadid]$ gcc -lpthread exemplo-pid-ppid-threadid.c -o exemplo-pid-ppid-threadid-gcc.exe
[exemplo-pid-ppid-threadid]$ ./exemplo-pid-ppid-threadid-gcc.exe
Thread[140337874589504]: Principal. PID: 145136, PPID: 136279, Kernel TID: 145136, Pthread TID: 140337874589504
Thread[140337871845056]: Thread. PID: 145136, PPID: 136279, Kernel TID: 145137, Pthread TID: 140337871845056
[exemplo-pid-ppid-threadid]$
```



```
exemplo-thread.c
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <pthread.h>
8
9 void *print_message_function( void *ptr );
10
11 int main() {
12     /* Declara duas threads */
13     pthread_t thread1, thread2;
14     char *message1 = "Olá eu sou a Thread 1";
15     char *message2 = "Olá eu sou a Thread 2";
16     int  iret1, iret2;
17
18     /* Cria duas threads independentes, cada uma irá executar a função */
19     iret1 = pthread_create( &thread1, NULL, print_message_function, (void*) message1);
20     iret2 = pthread_create( &thread2, NULL, print_message_function, (void*) message2);
21
22     /* Aguarda até que todas as threads completem antes de continuar. */
23     /* Pode acontecer de executar algo que termine o processo/thread principal antes das threads terminarem */
24     pthread_join(thread1, NULL);
25     pthread_join(thread2, NULL);
26
27     printf("Thread 1 retornou: %d\n", iret1);
28     printf("Thread 2 retornou: %d\n", iret2);
29
30     exit(0);
31 }
32
33 void *print_message_function( void *ptr )
34 {
35     char *message;
36     message = (char *) ptr;
37     printf("%s \n", message);
38 }
39
```

```
gcc -lpthread exemplo-pthread.c -o exemplo-pthread.exe
```

Figure 4.1: Exemplo de Thread no Linux

Código 4.3 Makefile Makefile

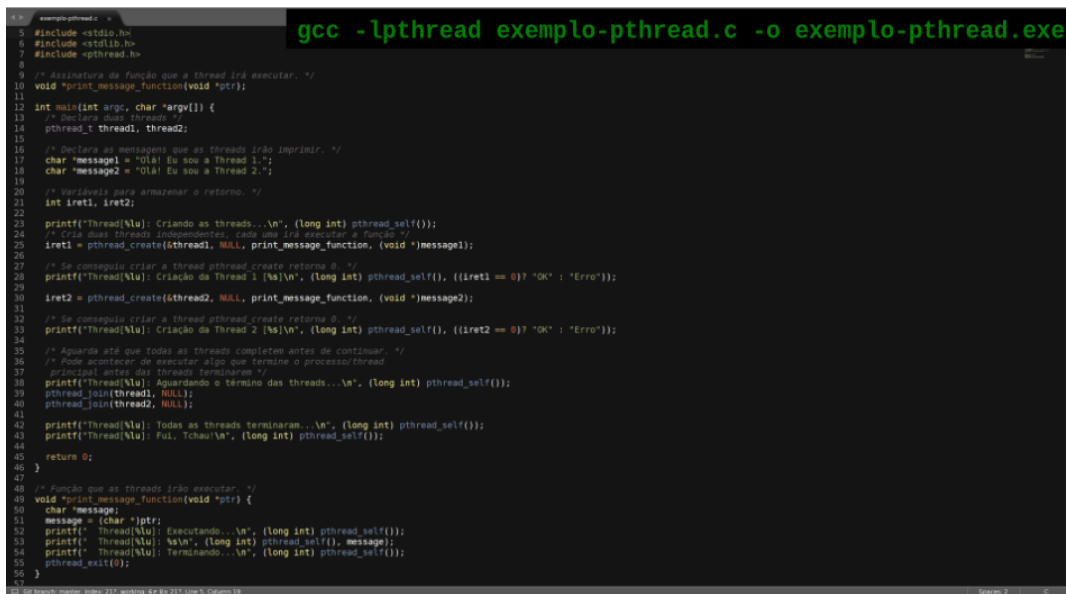
```
# Variáveis.
CC = gcc
CFLAGS = -Wall -lpthread
TARGET = exemplo-pid-ppid-threadid
EXT = exe

# Alvo padrão executado ao digitar apenas 'make'.
all: $(TARGET)

# Regra para gerar o executável final.
$(TARGET): $(TARGET).o
    $(CC) $(CFLAGS) $~ -o $(TARGET).$(EXT)

# Regra para compilar o arquivo objeto.o.
$(TARGET).o: $(TARGET).c
    $(CC) $(CFLAGS) -c $~

# Alvo para limpar arquivos gerados.
clean:
    rm -f *.o $(TARGET).$(EXT)
```



```
gcc -lpthread exemplo-pthread.c -o exemplo-pthread.exe

5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <pthread.h>
8
9 /* Assinatura da função que a thread irá executar. */
10 void *print_message_function(void *ptr);
11
12 int main(int argc, char *argv[]) {
13     /* Declara duas threads */
14     pthread_t thread1, thread2;
15
16     /* Declara as mensagens que as threads irão imprimir. */
17     char *message1 = "Olá! Eu sou a Thread 1.";
18     char *message2 = "Olá! Eu sou a Thread 2.";
19
20     /* Variáveis para armazenar o retorno. */
21     int iret1, iret2;
22
23     printf("Thread\n"); Criando as threads...\n", (long int) pthread_self());
24     /* Cria duas threads independentes, cada uma irá executar a função */
25     iret1 = pthread_create(&thread1, NULL, print_message_function, (void *)message1);
26
27     /* Se conseguiu criar a thread pthread_create retorna 0. */
28     printf("Thread\n"); Criação da Thread 1 [%s]\n", (long int) pthread_self(), ((iret1 == 0)? "OK" : "Erro"));
29
30     iret2 = pthread_create(&thread2, NULL, print_message_function, (void *)message2);
31
32     /* Se conseguiu criar a thread pthread_create retorna 0. */
33     printf("Thread\n"); Criação da Thread 2 [%s]\n", (long int) pthread_self(), ((iret2 == 0)? "OK" : "Erro"));
34
35     /* Aguarda até que todas as threads completem antes de continuar. */
36     /* Não acontece de executar algo que termine o processo/thread
37     principal antes das threads terminarem */
38     printf("Thread\n"); Aguardando o término das threads...\n", (long int) pthread_self());
39     pthread_join(thread1, NULL);
40     pthread_join(thread2, NULL);
41
42     printf("Thread\n"); Todas as threads terminaram...\n", (long int) pthread_self());
43     printf("Thread\n"); Fui. Tchau!\n", (long int) pthread_self());
44
45     return 0;
46 }
47
48 /* Função que as threads irão executar. */
49 void *print_message_function(void *ptr) {
50     char *message;
51     message = (char *)ptr;
52     printf(" Thread\n"); Executando...\n", (long int) pthread_self());
53     printf(" Thread\n"); %s\n", (long int) pthread_self(), message);
54     printf(" Thread\n"); Terminando...\n", (long int) pthread_self());
55     pthread_exit(0);
56 }
57
```

Figure 4.2: Exemplo de Thread no Linux

Código 4.4 Terminal Compilando com o ‘make’.

```
[exemplo-pid-ppid-threadid]$ ls
exemplo-pid-ppid-threadid.c  Makefile
[exemplo-pid-ppid-threadid]$ make
gcc -Wall -lpthread -c exemplo-pid-ppid-threadid.c
gcc -Wall -lpthread exemplo-pid-ppid-threadid.o -o exemplo-pid-ppid-threadid.exe
[exemplo-pid-ppid-threadid]$ ls
exemplo-pid-ppid-threadid.c  exemplo-pid-ppid-threadid.exe  exemplo-pid-ppid-threadid.o  Makefile
[exemplo-pid-ppid-threadid]$
```

Código 4.5 Terminal Executando o Exemplo

```
[exemplo-pid-ppid-threadid]$ ./exemplo-pid-ppid-threadid.exe
Thread[140546732336960]: Principal. PID: 136568, PPID: 136279, Kernel TID: 136568, Pthread TID: 136568
Thread[140546729309888]: Thread. PID: 136568, PPID: 136279, Kernel TID: 136569, Pthread TID: 136569
[exemplo-pid-ppid-threadid]$ ps -aux | grep bash
rag      136279  0.1  0.0 12336 10660 pts/3    Ss   09:13   0:00 /bin/bash
rag      136574  0.0  0.0  7096  4504 pts/3    S+   09:13   0:00 grep --color=auto bash
[rag@backporting exemplo-pid-ppid-threadid]$
```

[block]{Na linguagem C com Pthreads}

```
#include <pthread.h>

void *function( void *ptr ) {
    char *message;
    message = (char *) ptr;
    printf("%s \n", message);
}

int main() {
    /* Declara a thread */
    pthread_t thread1;
    char *message1 = "Ola eu sou a Thread 1";
    int iret1;

    /* Cria a thread para executar a funcao. */
    iret1 = pthread_create( &thread1, NULL, function, (void*) message1);

    /* Aguarda ate que todas as threads completem antes de continuar. */
    pthread_join(thread1, NULL);

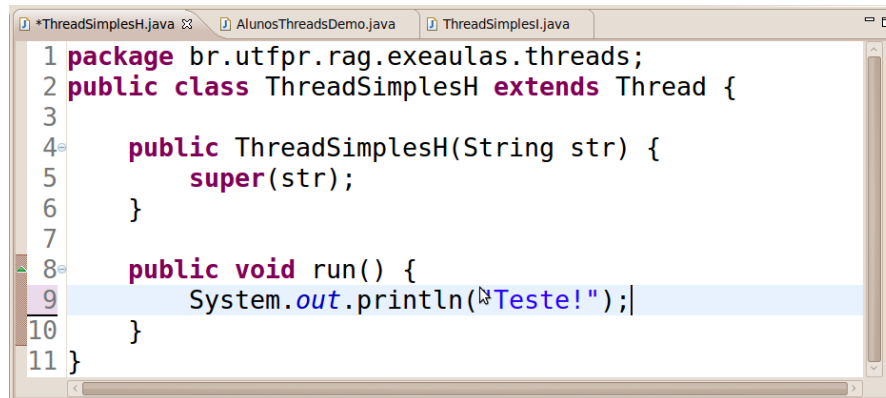
    printf("Thread 1 retornou: %d\n", iret1);

    exit(0);
}
```

[/block]

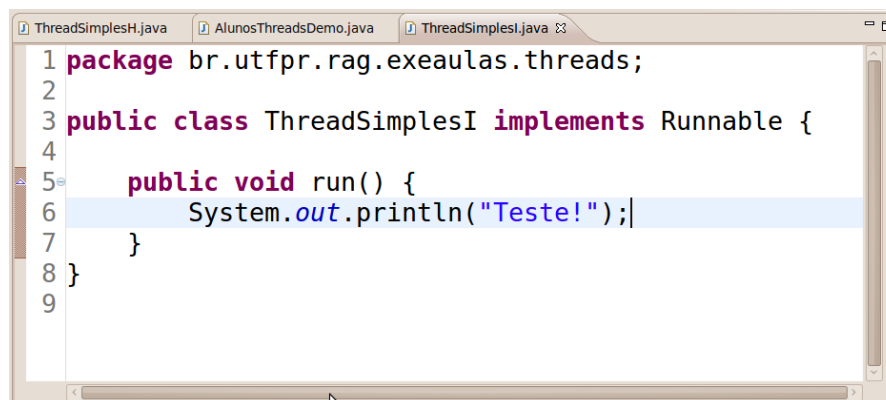
[framebreak]

- Em Java existem pelo menos duas formas de criarmos *threads*.



```
1 package br.utfpr.rag.exeaulas.threads;
2 public class ThreadSimpleH extends Thread {
3
4     public ThreadSimpleH(String str) {
5         super(str);
6     }
7
8     public void run() {
9         System.out.println("Teste!");
10    }
11 }
```

Figure 4.3: Criação por Herança



```
1 package br.utfpr.rag.exeaulas.threads;
2
3 public class ThreadSimpleI implements Runnable {
4
5     public void run() {
6         System.out.println("Teste!");
7     }
8 }
9
```

Figure 4.4: Implementação por Interface

[framebreak]

- Em *Python* também podemos criar *threads*:

[columns]

[column=0.5]

```
from threading import Thread
from time import sleep

class Hello(Thread):
    def run(self):
        sleep(3);
        print("Hello World")

thread = Hello()
thread.start()
```

Figure 4.5: Por Herança

[column=0.5]

```

from threading import Thread
from time import sleep

class Hello:
    def foo(self):
        sleep(3);
        print("Hello World")

hello = Hello()
thread = Thread(target=hello.foo)
thread.start()

```

Figure 4.6: Por Alvo

[/columns]

4.3 Implementação de Paralelismo com Cilk

- Paralelismo com a Extensão *Cilk*.

[columns]

[column=0.5]

```

#include <stdio.h>
#include <cilk/cilk.h>

static void hello(){
    int i=0;
    for(i=0;i<1000000;i++)
        printf("");
    printf("Hello ");
}

static void world(){
    int i=0;

```

```

    for(i=0;i<1000000;i++)
        printf("");
        printf("world! ");
}

int main(){
    cilk_spawn hello();
    cilk_spawn world();
    cilk_sync;
    printf("Done! ");
}

```

[column=0.5]

```

#include <stdio.h>
#include <pthread.h>

int main(){
    int sum = 0;
    int i = 0;
    pthread_mutex_t m;
    pthread_mutex_init(&m, NULL);

    cilk_for (i = 0; i <= 1000; i++){
        // lock.
        pthread_mutex_lock(&m);
        sum += i;
        // unlock.
        pthread_mutex_unlock(&m);
    }
    printf("%d\n",sum);
}

```

[/columns]

4.4 Implementação de Paralelismo com OpenMP

- Paralelismo com a Extensão OpenMP:

```

/* Compile:
 * gcc -fopenmp example-parallel-num_threads.c -o
 * example-parallel-num_threads.exe
 * or
 * make omp-parallel-num_threads
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#ifdef _OPENMP
#include <omp.h>
#else
#define omp_get_thread_num() 0
#define omp_get_num_threads() 1
#define omp_get_num_procs() \
    (system("cat /proc/cpuinfo | grep 'processor' | wc -l"))
#endif

int main() {
    int id, i;

    printf("Thread[%d] [%lu]: Before parallel region.\n", omp_get_thread_num(), (long int) pthread_self());

    #pragma omp parallel num_threads(4) default(none) private(id)
    {

```

```
// All threads executes this code.
id = omp_get_thread_num();

#pragma omp for
for(i=0; i<16; i++){
    printf("Thread[%d] [%lu]: Working in %lu loop iteration.\n", id, (long int) pthread_self(), i);
}

printf("Thread[%d] [%lu]: After parallel region...\n", omp_get_thread_num(), (long int) pthread_self());

return 0;
}
```

5 Prática: Programação com Threads no Linux

5.1 Introdução

[alertblock]{Objetivos}

- Compreender algumas das principais funções da biblioteca **pthread**.
- Criar threads no SO GNU/Linux.
- Desenvolver aplicações simples multithreads.

[/alertblock]

5.2 POSIX e Threads

- POSIX (IEEE 1003.1c) especifica uma API para criação e sincronismo de threads.
- A API especifica o comportamento da biblioteca de threads, a implementação fica para o desenvolvimento da biblioteca.
- Comum em sistemas operacionais UNIX (Solaris, Linux, Mac OS X).
- Os threads que usam a API de thread POSIX são chamados de **Pthreads**.

5.3 Threads no Linux

- Aloca o mesmo bloco de controle para threads e processos.
- Faz uso da chamada de sistema **clone**.
- As threads compartilham o espaço de endereço do processo.
- Os estados que as threads assumem são os mesmos dos processos.
- As Pthreads são implementadas como nível núcleo.

6 Biblioteca pthread

Identificadores da Thread.

<https://courses.cs.vt.edu/cs3214/spring2022/questions/threadid>

6.1 Criar novas threads

```
int pthread_create (pthread_t *thread,
                   const pthread_attr_t *attr,
                   void * (*start_routine)(void *),
                   void *arg);
```

- *thread*: identificador da thread.
- *attr*: atributos de configuração da thread. Se NULL, usa os padrões.
- *start_routine*: função com o código da thread.
- *arg*: parâmetro único para passagem de valores para a thread.
- *returns*: 0 em caso de sucesso ou um código de erro.

6.2 Finalizar threads

```
void pthread_exit (void *retval);
```

- *retval*: valor de retorno para a thread que aguarda (*join*).

6.3 Aguarda término de threads

```
int pthread_join (pthread_t thread,
                  void **retval);
```

- *thread*: identificador da thread a ser aguardada.
- *retval*: valor devolvido pela função *pthread_exit*.
- *returns*: 0 em caso de sucesso ou um código de erro.

6.4 Exemplo: Criação e finalização de threads

```
#include <stdio.h>
#include <stdlib.h> // exit
#include <unistd.h> // sleep
#include <pthread.h> // pthread_t, pthread_create(), pthread_join(), pthread_exit()

/** funcao que implementa o fluxo da thread */
void *print_message (void *ptr){
    char *message;
    message = (char *) ptr; //casting - coersao explicita
    sleep(2);
    printf("%s \n", message);

    //pthread_exit(NULL);
}
```

```

    return NULL;
}

/* fluxo principal - thread 'main' */
int main() {
    /* Declara duas threads */
    pthread_t thread1, thread2;
    //pthread_t thread[2];

    char *msg1 = "Eu sou a Thread 1.";
    char *msg2 = "Eu sou a Thread 2.";

    int iret1, iret2;

    /* Cria duas threads independentes, cada uma executa a funcao */
    iret1 = pthread_create( &thread1, NULL, print_message, (void*) msg1);
    iret2 = pthread_create( &thread2, NULL, print_message, (void*) msg2);

    printf("Thread 1 foi criada com sucesso?: %d\n", iret1);
    printf("Thread 2 foi criada com sucesso?: %d\n", iret2);

    /* Aguarda threads finalizarem antes de continuar. */
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("MAIN - FIM\n");

    //pthread_exit(0);
    exit(0);
}

```

6.5 Cancelamento de threads

```
int pthread_cancel (pthread_t thread);
```

- *thread*: identificador da thread.
- *returns*: 0 em caso de sucesso ou um código de erro.

Cria pontos de cancelamento de threads:

```
void pthread_testcancel(void);
```

Ativa ou desativa o cancelamento de threads

```
int pthread_setcancelstate (int state,
                           int *oldstate);
```

- *state*: modifica o estado para *PTHREAD_CANCEL_ENABLE* ou *PTHREAD_CANCEL_DISABLE*.
- *oldstate*: armazena o estado antigo.
- *returns*: 0 em caso de sucesso ou um código de erro.

Modifica o tipo do cancelamento de threads

```
int pthread_setcanceltype (int type,
                           int *oldtype);
```

- *type*: modifica o tipo de cancelamento para *PTHREAD_CANCEL_DEFERRED* ou *PTHREAD_CANCEL_ASYNCHRONOUS*.
- *oldstate*: armazena o antigo tipo.
- *returns*: 0 em caso de sucesso ou um código de erro.

6.6 Exemplo: Cancelamento de threads

```

#include <pthread.h> // pthread_testcancel(), pthread_setcancelstate(), pthread_cancel()
#include <stdio.h>
#include <unistd.h>

/* tarefa para ser executada como thread e testar o cancelamento */
void *task_do_nothing (void *parm){
    int i = 1;
    printf("THREAD - thread executada.\n");
    //pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);
    /* altera o tipo de cancelamento para assincrono - padrao deferred (adiado) */
    //pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);

    for (;;) {
        printf("THREAD - thread em laço %d\n", i); //ponto de cancelamento

        /* testa o cancelamento */
        //pthread_testcancel();

        sleep(1);
        if (i == 5) {
            /* Como a opção de cancelamento foi desabilitada, ativar agora */
            printf("THREAD - ativa o estado de cancelamento\n");
            pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
        }
    } // for

    return NULL;
}

/* fluxo principal (thread principal) */
int main(int argc, char **argv){
    pthread_t thread;
    int status = 0;

    printf("MAIN - Iniciando a thread principal (main)\n");

    /* cria a thread com atributos padraes */
    printf("MAIN - Cria a thread com atributos padraes\n");
    status = pthread_create(&thread, NULL, task_do_nothing, NULL);

    /* aguarda um tempo antes de solicitar o cancelamento */
    sleep(2);

    printf("MAIN - Cancelar a thread\n");
    status = pthread_cancel(thread);
    printf("MAIN - Retorno de pthread_cancel %d\n", status);

    pthread_join(thread, NULL);
    printf("MAIN - Main completed\n");

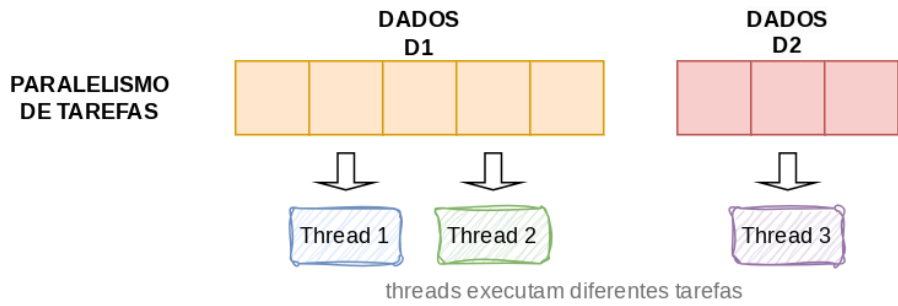
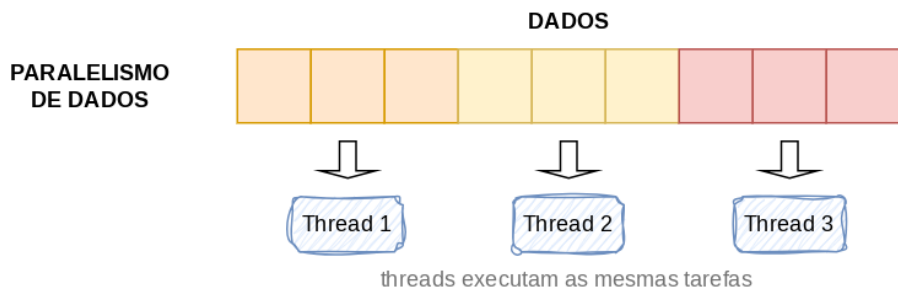
    pthread_exit(NULL);

    return 0;
}

```

6.7 Paralelismo com Threads

- **Paralelismo de Dados**
- Execução concorrente de uma mesma tarefa em diferentes elementos de dados.
- **Paralelismo de Tarefas**
- Execução concorrente de diferentes tarefas nos mesmos ou diferentes dados.



7 Atividades

7.1 Atividades

1. Execute os códigos dos exemplos da Aula.

8 Prática: Soma de Vetores Sequencial

- Execute o exemplo *vectoradd* versão sequencial (sem o uso de qualquer recurso de paralelismo)
- Código fonte em: [src/vectoradd](#)

```
float h_a[N];
float h_b[N];
float h_c[N];

int main(int argc, char **argv) {
    int i;
    /* Inicializacao dos vetores. */
    init_array();

    /* Calculo. */
    for (i = 0; i < N; i++) {
        h_c[i] = h_a[i] + h_b[i];
    }

    /* Resultados. */
    print_array();
    check_result();

    return 0;
}
```

9 Prática: Soma de Vetores Paralelo: Versão estática

- Analise as alterações que foram feitas no código do exemplo *vectoradd* para a utilização de *pthread*s ([books/daglib/0085131/](https://books.daglib.org/0085131/)), com particionamento estático das iterações do laço.
- Código fonte em: [src/vectoradd-pthreads-distribuicao-estatica](#)

```
void *vecadd(void *ptr) {
    int *id;
    int i, ii, ff;

    id = (int *) ptr;

    ii = (*id) * partition;
    ff = ii + partition; // ff = (ii + 1) * partition;

    printf(" Thread[%lu]: Particao prevista: %d [%d..%d]: %d.\n", (long int) pthread_self(), *id, ii, ff, (ff - ii));

    /* A sobra n - ff sempre sera maior que uma particao ate a penultima thread. Na última thread se sobrar
    algumas iterações no final, a ultima thread assume estendendo seu ff para n. */
    if((n - ff) < partition){
        ff = n;
    }

    printf(" Thread[%lu]: Executando sobre particao: %d [%d..%d]: %d.\n", (long int) pthread_self(),
    *id, ii, ff, (ff - ii));

    for (i = ii; i < ff; i++) {
        h_c[i] = h_a[i] + h_b[i];
    }

    printf(" Thread[%lu]: Terminando...\n", (long int) pthread_self());
    pthread_exit(0);
}

int main(int argc, char *argv[]) {
    int i, num_elementos, num_threads = 0;

    if(argc < 3){
        printf("Uso: %s <size> <num_threads>\n", argv[0]);
        exit(0);
    }

    num_elementos = atoi(argv[1]);
    num_threads = atoi(argv[2]);

    partition = num_elementos / num_threads;
```

```

n = num_elementos;

printf("num_elementos: %d num_threads: %d\n", num_elementos, num_threads);

h_a = (float *) malloc(num_elementos * sizeof(float));
h_b = (float *) malloc(num_elementos * sizeof(float));
h_c = (float *) malloc(num_elementos * sizeof(float));

init_array(num_elementos);

pthread_t *threads;

threads = (pthread_t *) malloc(num_threads * sizeof(pthread_t));

int *irets;
irets = (int *) malloc (num_threads * sizeof(int));

int *ids;
ids = (int *) malloc (num_threads * sizeof(int));

for(i=0; i<num_threads; i++){
    ids[i] = i;
    irets[i] = pthread_create(&threads[i], NULL, vecadd, (void *)&ids[i]);
    printf("Thread[%lu]: Criacao da Thread %d com o id: %lu [%s]\n", (long int) pthread_self(), i,
        threads[i], ((irets[i] == 0)? "OK" : "Erro"));
}

for(i=0; i<num_threads; i++){
    printf("Thread[%lu]: Aguardando o termino das threads...\n", (long int) pthread_self());
    pthread_join(threads[i], NULL);
}

printf("Thread[%lu]: Todas as threads terminaram...\n", (long int) pthread_self());

printf("Thread[%lu]: Imprimindo o resultado.\n", (long int) pthread_self());

printf("Thread[%lu]: Verificando o resultado.\n", (long int) pthread_self());
    check_result(num_elementos);

printf("Thread[%lu]: Fui, Tchau!\n", (long int) pthread_self());

return 0;
}

```

10 Prática: Soma de Vetores Paralelo: Versão dinâmica

- Analise as alterações que foram feitas no código do exemplo *vectoradd* para a utilização de *pthread*s ([books/daglib/0085131?](#)), com particionamento dinâmico das iterações do laço.
- Código fonte em: [src/vectoradd-pthreads-distribuicao-dinamica](#)

```
bool get_next_loop_partition(long *ii, long *ff) {
    bool ret = true;

    *ii = 0;
    *ff = 0;

    fprintf(stdout, "Thread[%lu]: Trying to get the next partition.\n", (long int) pthread_self());

    sem_wait(&mutex);

    if(work_finished){
        ret = false;
    }
    else{
        *ii = last_assigned;
        *ff = last_assigned + partition;

        if (*ff > total_of_iterations){
            *ff = total_of_iterations;
        }

        last_assigned = *ff;
        ret = true;

        work_finished = (last_assigned == total_of_iterations);
    }

    sem_post(&mutex);

    fprintf(stdout, "Thread[%lu]: Got the partition [%lu, %lu].\n", (long int) pthread_self(), *ii, *ff);

    return ret;
}

void *vecadd(void *ptr) {
    long *id;
    long i, ii, ff;

    id = (long *) ptr;
    bool have_work = false;
```

```

while ((have_work = get_next_loop_partition(&ii, &ff)) != false){
    fprintf(stdout, "    Thread[%lu,%lu]: Working on partition: [%d..%d]: %d.\n", *id, (long int) pthread_self(),
        ii, ff, (ff - ii));
    for (i = ii; i < ff; i++) {
        h_c[i] = h_a[i] + h_b[i];
    }
}
if(!have_work){
    fprintf(stdout, "    Thread[%lu]: No work to do.\n", (long int) pthread_self());
}
fprintf(stdout, "    Thread[%lu]: Exiting.\n", (long int) pthread_self());
pthread_exit(0);
}

int main(int argc, char *argv[]) {
    int i, num_elements, num_threads = 0;

    if(argc < 4){
        fprintf(stderr, "Usage: %s <num_elements> <num_threads> <partition_size>\n", argv[0]);
        exit(0);
    }

    num_elements = atoi(argv[1]);
    num_threads = atoi(argv[2]);
    partition = atoi(argv[3]);

    total_of_iterations = num_elements;

    fprintf(stdout, "Thread[%lu]: num_elements: %d num_threads: %d.\n", (long int) pthread_self(), num_elements,
        num_threads);

    fprintf(stdout, "Thread[%lu]: Allocating the arrays.\n", (long int) pthread_self());
    h_a = (float *) malloc(num_elements * sizeof(float));
    h_b = (float *) malloc(num_elements * sizeof(float));
    h_c = (float *) malloc(num_elements * sizeof(float));

    init_array(num_elements);

    pthread_t *threads;

    threads = (pthread_t *) malloc(num_threads * sizeof(pthread_t));

    int *irets;
    irets = (int *) malloc (num_threads * sizeof(int));

    long *ids;
    ids = (long *) malloc (num_threads * sizeof(long));

    sem_init(&mutex, 0, 1);          /* initialize mutex to 1 - binary semaphore */
                                   /* second param = 0 - semaphore is local */

    work_finished = false;

    fprintf(stdout, "Thread[%lu]: Creating the Threads.\n", (long int) pthread_self());

    for(i=0; i<num_threads; i++){
        ids[i] = i;
        irets[i] = pthread_create(&threads[i], NULL, vecadd, (void *)&ids[i]);
        fprintf(stdout, "Thread[%lu]: Create Thread %d with id: %lu [%s]\n", (long int) pthread_self(), i, threads[i],
            ((irets[i] == 0)? "OK" : "Error"));
    }
}

```

```

}

for(i=0; i<num_threads; i++){
    fprintf(stdout, "Thread[%lu]: Waiting for ending of threads execution.\n", (long int) pthread_self());
    pthread_join(threads[i], NULL);
}

fprintf(stdout, "Thread[%lu]: All threads were finished.\n", (long int) pthread_self());

fprintf(stdout, "Thread[%lu]: Printing the result.\n", (long int) pthread_self());

fprintf(stdout, "Thread[%lu]: Checking the result.\n", (long int) pthread_self());
    check_result(num_elements);

fprintf(stdout, "Thread[%lu]: Fui, Tchau!\n", (long int) pthread_self());

return 0;
}

```

11 Relatório

- Escreva um relatório sobre a prática, colocando códigos e figuras da execução. Quais alterações foram necessárias para a execução das versões paralelas do soma de vetores utilizando *pthread*s com particionamento estático e dinâmico (quantidade de código, desempenho das versões sequencial e paralelas (estática e dinâmica) para tamanhos de dados suficientemente grandes).

12 Leitura Recomendada

12.1 Leitura Recomendada

[alertblock]{Capítulo 2: Processos e Threads}

Seção 2.2. Threads

TANENBAUM, Andrew S. Sistemas operacionais modernos. 3. ed.~São Paulo, SP: Pearson, c2008. xvi, 654 p.~ISBN 9788576052371. (**Tanenbaum?**)

[/alertblock]

[exampleblock]{Capítulo 5: Implementação de tarefas}

Seções 5.4. Threads

MAZIERO, C. Sistemas Operacionais: Conceitos e Mecanismos. Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2. (**Maziero?**)

[/exampleblock]

13 Referências

13.1 Referências

13.2 Leitura Recomendada

[alertblock]{Capítulo 2: Processos e Threads}

Seção 2.2. Threads

TANENBAUM, Andrew S. Sistemas operacionais modernos. 3. ed.~São Paulo, SP: Pearson, c2008. xvi, 654 p.~ISBN 9788576052371. (**Tanenbaum?**)

[/alertblock]

[exampleblock]{Capítulo 5: Implementação de tarefas}

Seções 5.4. Threads

MAZIERO, C. Sistemas Operacionais: Conceitos e Mecanismos. Editora da UFPR, 2019. 456 p. ISBN 978-85-7335-340-2. (**Maziero?**)

[/exampleblock]

14 Próximas Aulas

14.1 Próximas Aulas

- Sincronização entre Processos.

14.2 Livros Texto

[columns]

[column=0.3]

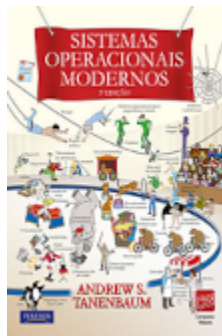


Figure 14.1: Livro

[column=0.7]

TANENBAUM, Andrew S. Sistemas operacionais modernos. 3. ed. São Paulo, SP: Pearson, c2008. xvi, 654 p. ISBN 9788576052371.

[/columns]

[framebreak]

[columns]

[column=0.3]



Figure 14.2: Livro

[column=0.7]

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Sistemas operacionais: conceitos e aplicações / Abraham Silberschatz, Peter Galvin, Greg Gagne; traduzido por Adriana Ceschin Rieche. [s. l.]: Campus, 2000.

[/columns]

[framebreak]

[columns]

[column=0.3]

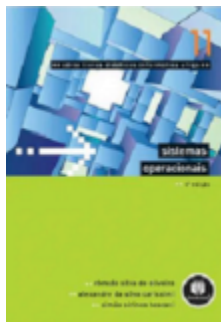


Figure 14.3: Livro

[column=0.7]

OLIVEIRA, Rômulo Silva de; CARISSIMI, Alexandre da Silva; TOSCANI, Simão Sirineo. Sistemas operacionais. 4. ed. Porto Alegre, RS: Bookman, Instituto de Informática da UFRGS, 2010. 374 p. (Livros didáticos, 11). ISBN 9788577805211.

[/columns]

[framebreak]

[columns]

[column=0.3]

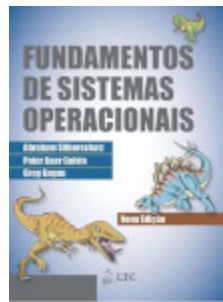


Figure 14.4: Livro

[column=0.7]

SILBERSCHATZ, Abraham; GALVIN, Peter Baer; GAGNE, Greg. Fundamentos de sistemas operacionais. 9. ed. Rio de Janeiro, RJ: LTC, c2015. 508 p. ISBN 9788521629399.

[/columns]

15 Referências

15.1 Referências

15.2 William

Threads, também conhecidas como processos leves, são fluxos de execução independentes dentro de um mesmo processo, compartilhando recursos como memória, arquivos abertos e espaço de endereçamento, mas mantendo estruturas próprias como pilha, registradores e contador de programa. Diferentemente dos processos tradicionais, que são criados com um conjunto completo de recursos pelo sistema operacional, as threads são mais leves e eficientes, permitindo maior paralelismo e desempenho.

Em sistemas com múltiplos núcleos de processamento, as threads podem ser executadas simultaneamente, aumentando a performance e a capacidade de resposta de aplicações, especialmente aquelas interativas. Conceitos fundamentais em programação multithread incluem escalonamento, sincronização, granularidade, regiões críticas e mecanismos de controle como locks, semáforos, mutexes, além da atenção com problemas como condições de corrida (race conditions) e impasses (deadlocks).

Sistemas operacionais modernos oferecem diferentes formas de suporte a threads. O Solaris adota um modelo híbrido com lightweight processes (LWPs), que atuam como intermediários entre threads de usuário e o kernel. O Windows 2000 utiliza um modelo com mapeamento direto entre threads de usuário e do kernel. Já o Linux trata threads como tarefas (tasks), criando-as por meio da chamada de sistema clone, que permite diferentes níveis de compartilhamento entre threads.

A programação com múltiplas threads oferece benefícios como economia de recursos, melhor aproveitamento dos processadores disponíveis e maior interatividade nas aplicações, sendo uma abordagem essencial para o desenvolvimento de sistemas concorrentes e responsivos.

15.3 Padrão POSIX

As POSIX Threads, ou simplesmente Pthreads, referem-se a uma API padronizada pelo IEEE no padrão POSIX 1003.1c, estabelecido em 1995. Esse padrão surgiu como uma resposta à fragmentação das implementações de threads disponíveis na época, quando diferentes

fornecedores de sistemas Unix adotavam soluções incompatíveis. Tal diversidade dificultava a portabilidade de aplicações multithread.

Com o objetivo de unificar essas abordagens, o padrão POSIX definiu uma interface uniforme e portátil para a criação, gerenciamento e sincronização de threads. Essa padronização abrange não apenas a sintaxe das funções, mas também o comportamento esperado, assegurando compatibilidade entre sistemas que implementam a API. A denominação “Pthreads” é comumente usada para referir-se às bibliotecas que seguem essa especificação, presentes em sistemas como Linux, BSD e Solaris.

O modelo proposto promove a programação concorrente com threads leves que compartilham o mesmo espaço de endereço, proporcionando desempenho superior em diversas aplicações.

16 Implementação de Threads

16.1 Implementação de Threads em Linguagens de Programação

- Basicamente em todas as linguagens que suportam *threads* a implementação segue a ideia de se ter uma função que é o código a ser executado pela *threads* quando ela for criada.
 - Na linguagem C, podemos utilizar a biblioteca *Pthreads* do **GNU/Linux**.
 - Implementa a API padrão POSIX (IEEE 1003.1c) para criação e sincronismo de *threads*.
 - A API especifica o comportamento da biblioteca de threads, a implementação fica para o desenvolvimento da biblioteca.
 - A especificação POSIX determina que os registradores do processador, a pilha e a máscara de sinal sejam mantidos individualmente para cada *thread*.
 - Detalhando como os sistemas operacionais devem emitir sinais a *Pthreads*, além de especificar diversos modos de cancelamento de *thread*.

- Básico de Pthreads:

[exampleblock]{Criar uma nova thread}

```
int pthread_create(pthread_t * thread, const pthread_attr_t * attr,  
void * (*start_routine)(void *), void *arg);
```

[/exampleblock]

[exampleblock]{Aguardar pelo término de outra thread}

```
int pthread_join(pthread_t th, void **thread_return);
```

[/exampleblock]

[block]{Terminar a thread}

```
void pthread_exit(void *retval);
```

[/block]

[alertblock]{Cancelar a execução da thread}

```
int pthread_cancel(pthread_t thread);
```

[/alertblock]

- Na linguagem C com *Pthreads*:

```
exemplo-thread.c X
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <pthread.h>
8
9  void *print_message_function( void *ptr );
10
11 int main() {
12     /* Declara duas threads */
13     pthread_t thread1, thread2;
14     char *message1 = "Olá eu sou a Thread 1";
15     char *message2 = "Olá eu sou a Thread 2";
16     int iret1, iret2;
17
18     /* Cria duas threads independentes, cada uma irá executar a função */
19     iret1 = pthread_create( &thread1, NULL, print_message_function, (void*) message1);
20     iret2 = pthread_create( &thread2, NULL, print_message_function, (void*) message2);
21
22     /* Aguarda até que todas as threads completem antes de continuar. */
23     /* Pode acontecer de executar algo que termine o processo/thread principal antes das threads terminarem */
24     pthread_join(thread1, NULL);
25     pthread_join(thread2, NULL);
26
27     printf("Thread 1 retornou: %d\n", iret1);
28     printf("Thread 2 retornou: %d\n", iret2);
29
30     exit(0);
31 }
32
33 void *print_message_function( void *ptr )
34 {
35     char *message;
36     message = (char *) ptr;
37     printf("%s \n", message);
38 }
39
```

```
gcc -lpthread exemplo-pthread.c -o exemplo-pthread.exe
```

Figure 16.1: Exemplo de Thread no Linux

[block]{Na linguagem C com Pthreads}

```
#include <pthread.h>

void *function( void *ptr ) {
    char *message;
    message = (char *) ptr;
    printf("%s \n", message);
}

int main() {
    /* Declara a thread */
    pthread_t thread1;
    char *message1 = "Ola eu sou a Thread 1";
    int iret1;

    /* Cria a thread para executar a funcao. */
    iret1 = pthread_create( &thread1, NULL, function, (void*) message1);

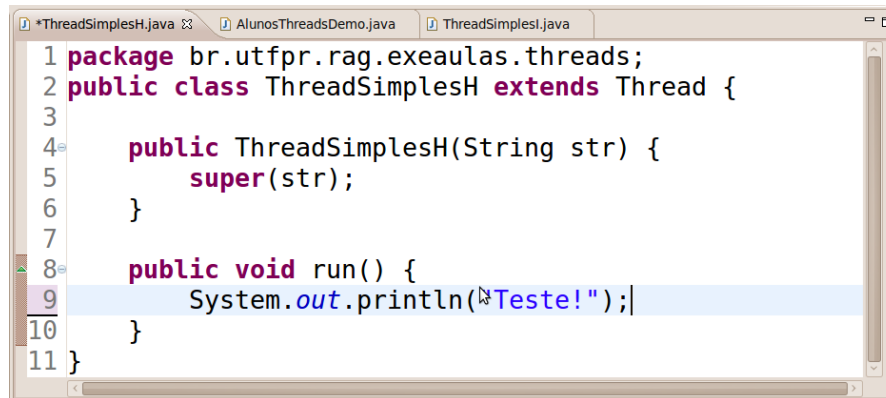
    /* Aguarda ate que todas as threads completem antes de continuar. */
    pthread_join(thread1, NULL);

    printf("Thread 1 retornou: %d\n", iret1);

    exit(0);
}
```

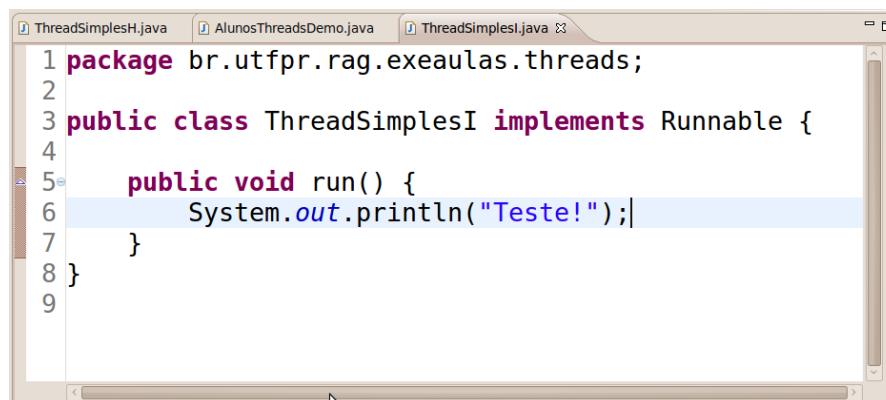
[/block]

- Em Java existem pelo menos duas formas de criarmos *threads*.

A screenshot of a Java IDE window titled 'ThreadSimpleH.java'. The code defines a class 'ThreadSimpleH' that extends 'Thread'. It has a constructor 'ThreadSimpleH(String str)' that calls 'super(str)'. The 'run()' method calls 'System.out.println("Teste!");'.

```
1 package br.utfpr.rag.exeaulas.threads;
2 public class ThreadSimpleH extends Thread {
3
4     public ThreadSimpleH(String str) {
5         super(str);
6     }
7
8     public void run() {
9         System.out.println("Teste!");
10    }
11 }
```

Figure 16.2: Criação por Herança

A screenshot of a Java IDE window titled 'ThreadSimpleI.java'. The code defines a class 'ThreadSimpleI' that implements 'Runnable'. It has a 'run()' method that calls 'System.out.println("Teste!");'.

```
1 package br.utfpr.rag.exeaulas.threads;
2
3 public class ThreadSimpleI implements Runnable {
4
5     public void run() {
6         System.out.println("Teste!");
7     }
8 }
9
```

Figure 16.3: Implementação por Interface

- Em *Python* também podemos criar *threads*:

[columns]

[column=0.5]

```
from threading import Thread
from time import sleep

class Hello(Thread):
    def run(self):
        sleep(3);
        print("Hello World")

thread = Hello()
thread.start()
```

Figure 16.4: Por Herança

[column=0.5]

```

from threading import Thread
from time import sleep

class Hello:
    def foo(self):
        sleep(3);
        print("Hello World")

hello = Hello()
thread = Thread(target=hello.foo)
thread.start()

```

Figure 16.5: Por Alvo

[/columns]

16.2 Implementação de Paralelismo com Cilk

- Paralelismo com a Extensão *Cilk*.

[columns]

[column=0.5]

```

#include <stdio.h>
#include <cilk/cilk.h>

static void hello(){
    int i=0;
    for(i=0;i<1000000;i++)
        printf("");
    printf("Hello ");
}

static void world(){
    int i=0;

```

```

    for(i=0;i<1000000;i++)
        printf("");
        printf("world! ");
}

int main(){
    cilk_spawn hello();
    cilk_spawn world();
    cilk_sync;
    printf("Done! ");
}

```

[column=0.5]

```

#include <stdio.h>
#include <pthread.h>

int main(){
    int sum = 0;
    int i = 0;
    pthread_mutex_t m;
    pthread_mutex_init(&m, NULL);

    cilk_for (i = 0; i <= 1000; i++){
        // lock.
        pthread_mutex_lock(&m);
        sum += i;
        // unlock.
        pthread_mutex_unlock(&m);
    }
    printf("%d\n",sum);
}

```

[/columns]

16.2.1 Primitivas principais da API Pthreads

16.2.1.1 1. Criação e finalização de threads

- `pthread_create`: cria uma nova thread, especificando a função que será executada.
- `pthread_exit`: encerra a thread atual e libera seus recursos.
- `pthread_join`: faz com que uma thread espere pela finalização de outra, permitindo sincronização e coleta de status.

16.2.1.2 2. Exclusão mútua (Mutexes)

Mutexes são utilizados para proteger seções críticas do código, garantindo que apenas uma thread por vez possa acessar determinados recursos compartilhados.

Principais funções: - `pthread_mutex_init`: inicializa um mutex. - `pthread_mutex_lock`: bloqueia o mutex, aguardando se ele já estiver em uso. - `pthread_mutex_unlock`: libera o mutex para que outras threads possam utilizá-lo. - `pthread_mutex_destroy`: destrói o mutex, liberando os recursos associados.

16.2.1.3 3. Variáveis de condição

Variáveis de condição (`pthread_cond_t`) permitem que threads aguardem por eventos ou estados específicos, sendo amplamente utilizadas em padrões como produtor-consumidor.

Funções principais: - `pthread_cond_init`: inicializa a variável de condição. - `pthread_cond_wait`: faz a thread esperar por um sinal, liberando temporariamente o mutex. - `pthread_cond_signal`: acorda uma thread em espera. - `pthread_cond_broadcast`: acorda todas as threads que estiverem aguardando. - `pthread_cond_destroy`: destrói a variável de condição.

16.2.1.4 4. Gerenciamento de atributos de threads

A biblioteca permite configurar atributos como: - Tipo de thread (`joinable` ou `detached`), - Prioridade, - Políticas de escalonamento.

Para isso, utilizam-se funções como: - `pthread_attr_init` - `pthread_attr_setdetachstate`

16.2.1.5 5. Cancelamento e tratamento de sinais

O padrão POSIX também prevê mecanismos para: - Cancelar threads (`pthread_cancel`), - Tratar sinais do sistema, contribuindo para a criação de aplicações robustas e responsivas.

16.3 Suporte do Hardware e Sistema Operacional

A API POSIX Threads (Pthreads) é amplamente suportada por sistemas operacionais compatíveis com o padrão POSIX, especialmente aqueles baseados em Unix, como Linux, macOS e BSD. Esses sistemas fornecem implementações robustas e eficientes da API, sendo frequentemente utilizadas em aplicações que demandam concorrência de alto desempenho.

Embora existam implementações de Pthreads para sistemas Windows, como a biblioteca Pthreads-w32, seu uso nesse ambiente é limitado. Isso se deve ao fato de que o Windows adota seu próprio modelo nativo de threads por meio da API Win32, o que torna o uso de Pthreads nesse contexto menos comum e, em alguns casos, redundante.

No nível de hardware, a utilização de threads POSIX é beneficiada por arquiteturas modernas de processadores que oferecem múltiplos núcleos e tecnologias como **Hyper-Threading**. Esse suporte permite que múltiplas threads sejam executadas em paralelo, seja em núcleos físicos distintos ou em unidades lógicas de execução.

No entanto, a responsabilidade pelo agendamento e pela distribuição das threads entre os núcleos disponíveis recai sobre o sistema operacional. Cabe a ele definir quando e onde cada thread será executada, considerando aspectos como prioridade, política de escalonamento e disponibilidade de recursos.

É importante destacar que as threads criadas por meio de Pthreads compartilham o mesmo espaço de memória do processo pai. Essa característica reduz a sobrecarga de criação e comunicação, quando comparada a processos independentes, mas impõe desafios adicionais em relação à sincronização. O acesso simultâneo a recursos compartilhados pode levar a condições de corrida e outros problemas de concorrência, exigindo o uso criterioso de mecanismos como mutexes e variáveis de condição.

16.4 Bibliotecas e Implementações

16.5 Exemplos Práticos com Pthreads

Nesta seção, apresentamos exemplos simples e didáticos para ilustrar o uso da biblioteca Pthreads em C.

16.5.1 Exemplo 1 – Hello World com threads

```
#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>

void *thread(void *vargp);

int main()
{
    pthread_t tid;
    printf("Thread[%lu]: Hello World da thread principal!\n", (long int) pthread_self());
    pthread_create(&tid, NULL, thread, NULL);
    pthread_join(tid, NULL);
    pthread_exit(NULL);
}

void *thread(void *vargp)
{
    printf("Thread[%lu]: Hello World da thread criada pela thread principal!\n", (long int) pthread_self());
}
```

```
pthread_exit(NULL);  
}
```

Preparamos um Makefile para facilitar a compilação dos exemplos.

```
all:    compile  
  
compile:    hello-world.c  
    gcc -lpthread hello-world.c -o hello-world.exe  
  
clean:  
    rm *.exe
```

O exemplo pode ser compilado utilizando o Makefile disponibilizado:

```
$ make  
gcc hello-world.c -o hello-world.exe  
$ ./hello-world.exe  
Thread[139899883358016]: Hello World da thread principal!  
Thread[139899883353792]: Hello World da thread criada pela thread principal!  
$
```

16.5.2 Exemplo 2 – Passagem de parâmetros

```
#include <pthread.h>  
#include <stdlib.h>  
#include <stdio.h>  
  
typedef struct {  
    int i;  
    int j;  
} thread_arg;  
  
void *thread(void *vargp);  
  
int main()  
{
```

```

pthread_t tid;
thread_arg a = {1, 2};

pthread_create(&tid, NULL, thread, (void *)&a);
pthread_join(tid, NULL);
pthread_exit(NULL);
}

void *thread(void *vargp)
{
    thread_arg *a = (thread_arg *)vargp;
    printf("Parâmetros recebidos: %d %d\n", a->i, a->j);
    pthread_exit(NULL);
}

```

16.5.3 Exemplo 3 – Execução paralela de múltiplas threads

```

#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>

typedef struct {
    int id;
} thread_arg;

void *thread(void *vargp);

int main()
{
    pthread_t tid[2];
    thread_arg a[2];

    for(int i = 0; i < 2; i++) {
        a[i].id = i;
        pthread_create(&tid[i], NULL, thread, &a[i]);
    }

    for(int i = 0; i < 2; i++) {

```

```

        pthread_join(tid[i], NULL);
    }

    pthread_exit(NULL);
}

void *thread(void *vargp)
{
    thread_arg *a = (thread_arg *)vargp;
    printf("Começou a thread %d\n", a->id);
    for(volatile int i = 0; i < 1000000; i++);
    printf("Terminou a thread %d\n", a->id);
    pthread_exit(NULL);
}

```

16.5.4 Exemplo 4 – Uso de mutex (trava)

```

#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>

typedef struct {
    int id;
} thread_arg;

pthread_mutex_t mutex;
int var = 0;

void *thread(void *vargp);

int main()
{
    pthread_t tid[2];
    thread_arg a[2];

    pthread_mutex_init(&mutex, NULL);

    for(int i = 0; i < 2; i++) {

```

```

        a[i].id = i;
        pthread_create(&tid[i], NULL, thread, &a[i]);
    }

    for(int i = 0; i < 2; i++) {
        pthread_join(tid[i], NULL);
    }

    pthread_mutex_destroy(&mutex);
    pthread_exit(NULL);
}

void *thread(void *vargp)
{
    thread_arg *a = (thread_arg *)vargp;

    pthread_mutex_lock(&mutex);
    printf("Thread %d: var antes = %d\n", a->id+1, var);
    var += a->id + 1;
    printf("Thread %d: var depois = %d\n", a->id+1, var);
    pthread_mutex_unlock(&mutex);

    pthread_exit(NULL);
}

```

16.5.5 Exemplo 5 – Variáveis condicionais

```

#include <pthread.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>

typedef struct {
    int id;
} thread_arg;

pthread_mutex_t mutex;
pthread_cond_t cond;

```

```

int ready = 0;

void *thread(void *vargp);

int main()
{
    pthread_t tid[2];
    thread_arg a[2];

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond, NULL);

    for(int i = 0; i < 2; i++) {
        a[i].id = i;
        pthread_create(&tid[i], NULL, thread, &a[i]);
    }

    for(int i = 0; i < 2; i++) {
        pthread_join(tid[i], NULL);
    }

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond);

    pthread_exit(NULL);
}

void *thread(void *vargp)
{
    thread_arg *a = (thread_arg *)vargp;

    if(a->id == 0) {
        pthread_mutex_lock(&mutex);
        while(!ready) {
            printf("Thread %d: esperando sinal...\n", a->id);
            pthread_cond_wait(&cond, &mutex);
        }
        printf("Thread %d: recebeu sinal\n", a->id);
        pthread_mutex_unlock(&mutex);
    } else {
        sleep(1);
        pthread_mutex_lock(&mutex);
    }
}

```

```

        ready = 1;
        pthread_cond_signal(&cond);
        printf("Thread %d: enviou sinal\n", a->id);
        pthread_mutex_unlock(&mutex);
    }

    pthread_exit(NULL);
}

```

16.6 Modelo de Programação e Execução

O modelo de programação com Pthreads é baseado na criação e controle explícito de threads a partir de um processo principal. Cada thread pode ser vista como uma função separada que será executada de forma concorrente com as demais. O ponto de entrada de uma thread é sempre uma função com a assinatura `void* função(void* argumento)`, permitindo grande flexibilidade para passagem de parâmetros via ponteiros genéricos. Funções como `pthread_create`, `pthread_join` e `pthread_exit` formam a base para o controle de ciclo de vida de uma thread. Além disso, mecanismos de sincronização como `pthread_mutex_lock` e `pthread_cond_wait` permitem o controle de acesso a recursos compartilhados, prevenindo condições de corrida (race conditions).

A sincronização se dá principalmente por meio de mutexes (para exclusão mútua) e variáveis de condição (para coordenação de estados). Essas estruturas permitem a construção de zonas críticas protegidas e a implementação de padrões clássicos de concorrência, como produtores e consumidores. Por exemplo, com `pthread_mutex_lock` e `pthread_mutex_unlock`, você garante que apenas uma thread possa acessar determinada seção crítica por vez. Já com `pthread_cond_wait` e `pthread_cond_signal`, você pode fazer com que uma thread aguarde até que uma certa condição seja atendida. O uso dessas primitivas exige disciplina, pois erros como deadlocks e starvation podem ocorrer se não forem bem planejadas.

Na prática, a modelagem com Pthreads requer um planejamento cuidadoso: é preciso definir claramente as tarefas que podem ser executadas em paralelo, decidir como compartilhar dados entre threads, e garantir que essas interações sejam seguras. Uma vantagem do modelo Pthreads é sua leveza: as threads compartilham o mesmo espaço de memória, permitindo comunicação mais eficiente do que processos independentes. No entanto, isso também exige maior atenção na programação, pois erros em uma thread podem afetar todo o processo.

```

int main(){

}

```

Nota

Nota

16.7 Considerações Finais

Atualmente, praticamente todo tipo de aplicação tende a ser paralelizada, já que o uso de processadores multicore está cada vez mais disseminado. Em pouco tempo, todos os processadores terão múltiplos núcleos. O padrão Pthreads é uma das opções que permite o uso maximizado desses processadores, não havendo um tipo específico de aplicação para Pthreads.

Embora seu uso seja mais óbvio em algumas aplicações, como servidores web que recebem múltiplas requisições simultâneas, quase toda aplicação pode ser paralelizada com o uso de Pthreads.

16.7.1 Prós e Contras

16.7.1.1 Vantagens de se programar utilizando threads:

- Utiliza melhor o potencial dos processadores multicore, que são cada vez mais comuns.
- O custo da troca de contexto entre threads é menor do que entre processos, devido à leveza das threads.
- Aplicações com paralelismo inerente, como grandes servidores web, se beneficiam do uso simples de threads para atender múltiplas requisições simultâneas.

16.7.1.2 Desvantagens de se programar utilizando threads:

- O modelo de programação com threads é mais complexo do que o modelo sequencial tradicional.
- Converter programas sequenciais em programas com threads não é trivial, muitas vezes exigindo reescrita significativa do código.

Bloco de Código padrão.

```
$ gcc -pthread programa.c -o programa
$ ./programa
```

Código 16.1 Terminal Comandos de compilação e execução do programa.

```
$ gcc -pthread programa.c -o programa
$ ./programa
Saída do programa aqui...
```

Filename

Conforme apresentado no Código 16.1, o programa é compilado utilizando a flag `-pthread`.

O Código 16.3 apresenta um exemplo de código que utiliza pthreads.

Código 16.2 Exemplo de Thread Código do arquivo teste.c.

i Nota

Bloco de Código com callout.

```
$ gcc -pthread programa.c -o programa
$ ./programa
Executando em paralelo...
```

Terminal

```
$ gcc -pthread programa.c -o programa
$ ./programa
```

Terminal

```
$ gcc -pthread programa.c -o programa
$ ./programa
```

Conforme apresentado na Precaução 1, bla

Precaução 1: Terminal

```
$ gcc -pthread programa.c -o programa
$ ./programa
```

16.8 Referências

- Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. 2007. *Using OpenMP: Portable Shared Memory Parallel Programming (Scientific and Engineering Computation)*. The MIT Press.
- Clang, Site. 2013. “Clang Site @ONLINE.” <http://clang.llvm.org/>.
- GCC. 2020. *GCC, the GNU Compiler Collection*. <http://gcc.gnu.org/onlinedocs>.
- GNU Libgomp. 2015. *GNU libgomp, GNU Offloading and Multi Processing Runtime Library documentation (Online manual)*. <https://gcc.gnu.org/onlinedocs/libgomp/>.
- Hirata, Hiroaki, Kozo Kimura, Satoshi Nagamine, et al. 1992. “An Elementary Processor Architecture with Simultaneous Instruction Issuing from Multiple Threads.” *Proceedings of the 19th Annual International Symposium on Computer Architecture* (New York, NY, USA), ISCA '92, 136–45. <https://doi.org/10.1145/139669.139710>.
- Intel. 2016. *Intel® OpenMP* Runtime Library Interface*. Intel. https://www.openmpRTL.org/sites/default/files/resources/libomp_20160322_manual.pdf.
- Intel. 2019. *Intel® c++ Compiler 19.1 Developer Guide and Reference*. <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>.
- Marr, Deborah T., Frank Binns, David L. Hill, et al. 2002. “Hyper-Threading Technology Architecture and Microarchitecture.” *Intel Technology Journal* 6 (1): 4–15.
- Olukotun, Kunle, Basem A. Nayfeh, Lance Hammond, Ken Wilson, and Kunyung Chang. 1996. “The Case for a Single-Chip Multiprocessor.” *SIGPLAN Not.* (New York, NY, USA) 31 (9): 2–11. <https://doi.org/10.1145/248209.237140>.
- OpenMP API Site. 2020. *The OpenMP(r) API Specification for Parallel Programming*. <http://openmp.org>.
- OpenMP-ARB. 2011. *OpenMP Application Program Interface Version 3.1*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP3.1.pdf>.
- OpenMP-ARB. 2013. *OpenMP Application Program Interface Version 4.0*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>.
- OpenMP-ARB. 2014. *OpenMP Application Program Interface Examples 4.0.1*. OpenMP Architecture Review Board (ARB). http://openmp.org/mp-documents/OpenMP_Example

s_4.0.1.pdf.

- OpenMP-ARB. 2015. *OpenMP Application Program Interface Version 4.5*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/openmp-4.5.pdf>.
- Smith, J. E., and G. S. Sohi. 1995. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE* 83 (12): 1609–24. <https://doi.org/10.1109/5.476078>.
- Spracklen, Lawrence, and Santosh G. Abraham. 2005. “Chip Multithreading: Opportunities and Challenges.” *Proceedings of the 11th International Symposium on High-Performance Computer Architecture (USA), HPCA '05*, 248–52. <https://doi.org/10.1109/HPCA.2005.10>.
- Standard, OpenACC. 2012. “OpenACC Directives for Accelerators Site @ONLINE.” March. <http://www.openacc-standard.org/>.
- Tomasulo, R. M. 1967. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units.” *IBM Journal of Research and Development* 11 (1): 25–33. <https://doi.org/10.1147/rd.111.0025>.
- Tullsen, Dean M., Susan J. Eggers, and Henry M. Levy. 1995. “Simultaneous Multithreading: Maximizing on-Chip Parallelism.” *SIGARCH Comput. Archit. News* (New York, NY, USA) 23 (2): 392–403. <https://doi.org/10.1145/225830.224449>.
- Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky. 1994. “Performance Estimation of Multistreamed, Superscalar Processors.” *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences* 1: 195–204. <https://doi.org/10.1109/HICSS.1994.323172>.

Código 16.3 exemplo-thread.c Código do arquivo exemplo-thread.c.

```
/* Para compilar use:
 * C compiler: gcc -lpthread exemplo-thread.c -o exemplo-thread
 * C++ compiler: g++ -lpthread exemplo-thread.c -o exemplo-thread
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *print_message_function( void *ptr );

int main() {
    /* Declara duas threads */
    pthread_t thread1, thread2;
    char *message1 = "Olá eu sou a Thread 1";
    char *message2 = "Olá eu sou a Thread 2";
    int  iret1, iret2;

    /* Cria duas threads independentes, cada uma irá executar a função */
    iret1 = pthread_create( &thread1, NULL, print_message_function, (void*) message1);
    iret2 = pthread_create( &thread2, NULL, print_message_function, (void*) message2);

    /* Aguarda até que todas as threads completem antes de continuar. */
    /* Pode acontecer de executar algo que termine o processo/thread principal antes das threads */
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Thread 1 retornou: %d\n", iret1);
    printf("Thread 2 retornou: %d\n", iret2);

    exit(0);
}

void *print_message_function(void *ptr) {
    char *message;
    message = (char *) ptr;
    printf("%s \n", message);
}
```

17 Introdução ao OpenMP: Diretivas de Compilação, Variáveis de Ambiente e Funções de Biblioteca

Resumo: Este capítulo apresenta conceitos fundamentais e introdutórios sobre o OpenMP, um padrão e API amplamente utilizados para explorar o potencial de processadores multicore em ambientes de memória compartilhada. Apresentamos o modelo de programação e modelo de execução `fork-join`, controle de `threads`, variáveis de ambiente e algumas funções das bibliotecas dos *runtimes* das implementações associadas aos principais compiladores atuais.

17.1 Introdução

Nas últimas décadas, a evolução da computação deixou de se concentrar apenas no aumento da frequência dos processadores e passou a investir na multiplicação de núcleos de processamento. Hoje, mesmo computadores pessoais e notebooks comuns contam com múltiplas cores, e servidores e supercomputadores chegam a ter dezenas ou centenas de núcleos trabalhando em conjunto. Essa mudança de paradigma exige que os programas sejam capazes de executar tarefas em paralelo, aproveitando ao máximo os recursos de hardware disponíveis.

O OpenMP (Open Multi-Processing) surge como uma solução prática e padronizada para explorar o paralelismo em sistemas de memória compartilhada. Trata-se de uma API amplamente utilizada, que permite adicionar paralelismo a programas escritos em C, C++ e Fortran de forma incremental, sem a necessidade de reescrever completamente o código. Seu modelo de programação é baseado no conceito `fork-join`, facilitando a adaptação de códigos sequenciais, permitindo ganhos de desempenho significativos com alterações relativamente pequenas.

O aprendizado do OpenMP é progressivo e incremental. Começando pela diretiva com o construtor de região paralela, depois pela divisão do trabalho entre as *threads* dentro dessa região paralela e na sequência paralelizando laços de repetição e avançando para outros construtores como os de tarefas independentes (tasks) e vetorização (SIMD).

17.2 Padrão OpenMP, compiladores, bibliotecas e Implementações

A abordagem do OpenMP é de mais alto nível, com uso de diretivas de compilação: instruções inseridas no código-fonte para indicar regiões paralelas. Funcionam como anotações que indicam seções paralelas do código sem a necessidade de reescrever toda a aplicação, permitindo testes e ajustes graduais. São implementadas usando-se as diretivas de pré-processamento `#pragma`, em C/C++, e sentinelas `!$`, no Fortran.

O modelo de programação do OpenMP trabalha em sistemas de memória compartilhada e é baseado no conceito `fork-join`: o programa inicia com uma única thread (master), que cria múltiplas **threads** para executar regiões paralelas (fork) explicitamente ou implicitamente. e, ao final, sincroniza todas elas (join).

O padrão OpenMP é suportado por praticamente todos os compiladores atuais: GCC (GNU Compiler Collection): suporte a OpenMP via `-fopenmp`; LLVM/Clang: suporte moderno com mais otimizações; Intel oneAPI / ICC: otimizações específicas para processadores Intel. IBM XL C/C++ e Fortran: suporte em sistemas de alto desempenho.

A utilização da biblioteca de OpenMP com a flag `-fopenmp` para C/C++ permite o acesso às seguintes funções:

- `int omp_get_thread_num()`: retorna o identificador da thread.
- `void omp_set_num_threads(int num_threads)`: indica o número de threads a executar na região paralela.
- `int omp_get_num_threads()`: retorna o número de threads que estão executando no momento.

Para a execução dos exemplos posteriores (construídos em C), será utilizado o compilador GCC com a flag `-fopenmp`, para facilitar as execuções, um exemplo de arquivo `makefile` é disponibilizado, onde `name` é a variável que define o nome do arquivo `.c`:

```
name=example

all:
    gcc -fopenmp ${name}.c -o ${name}.exe

clean:
    rm -rf *.o ${name}.exe
```

17.3 Diretivas de Compilação

As diretivas são instruções especiais que informam ao compilador como paralelizar o código. São formadas por construtores e cláusulas, os quais podem definir o escopo de variáveis, regras

de concorrência e algoritmos de escalonamento utilizado. Cada construtor possui diferentes cláusulas.

As principais diretivas são:

#pragma omp parallel: cria uma região paralela. Podendo conter cláusulas que definem o escopo de variáveis ou mesmo o número de **threads** para essa região.

Exemplo de utilização do construtor **parallel** com as funções da biblioteca omp, em uma região sem paralelismo, com paralelismo e paralelismo com número de **threads** definida:

```
#include <omp.h>
#include <stdio.h>

int main()
{
    int i;
    int omp_np; /* Número de Processadores. */
    int omp_nt; /* Número de threads OpenMP. */
    int omp_tid; /* Id da Thread OpenMP. */

    /* Teste 1 (Sem utilização de paralelismo) */
    printf("Teste 1 -----\\n");
    /* Traz os parâmetros do OpenMP utilizando as funções. */
    omp_np = omp_get_num_procs(); /* Traz o número de processadores. */
    omp_nt = omp_get_num_threads(); /* Traz o número de threads. */
    omp_tid = omp_get_thread_num(); /* Traz o id da thread OpenMP. */
    printf("Thread_id: %d #Processadores: %d #OpenMP Threads: %d\\n", omp_tid, omp_np, omp_nt);

    /* Teste 2 (Com utilização de paralelismo) */
    printf("Teste 2 -----\\n");
    #pragma omp parallel
    {
        omp_np = omp_get_num_procs(); /* Traz o número de processadores. */
        omp_nt = omp_get_num_threads(); /* Traz o número de threads. */
        omp_tid = omp_get_thread_num(); /* Traz o id da thread OpenMP. */
        printf("Thread_id: %d #Processadores: %d #OpenMP Threads: %d\\n", omp_tid, omp_np, omp_nt);
    }

    /* Teste 3 (Com utilização de paralelismo e controle do número de threads) */
    printf("Teste 3 -----\\n");
    #pragma omp parallel num_threads(8)
    {
        omp_np = omp_get_num_procs(); /* Traz o número de processadores. */
```

```
    omp_nt = omp_get_num_threads(); /* Traz o número de threads. */  
    omp_tid = omp_get_thread_num(); /* Traz o id da thread OpenMP. */  
    printf("Thread_id: %d #Processadores: %d #OpenMP Threads: %d\n", omp_tid, omp_np, omp_nt);  
}  
return 0;  
}
```

17.4 Objetivos

- Uma visão geral sobre OpenMP.
- Especificação x Implementações.
- Ferramentas: Compiladores.

18 Abordagem do OpenMP

18.1 Vantagens do uso de OpenMP

- Poucas alterações no código serial existente.
- Fácil compreensão e uso das diretivas.
- Suporte a paralelismo aninhado.
- Ajuste dinâmico do número de *threads*. Criação de *threads* é incremental.

18.2 Padrão OpenMP

- O OpenMP (Chapman et al. 2007) é um padrão bem conhecido e amplamente utilizado em aplicações para plataformas *multicore*.
- O uso de **Diretivas de Compilação**.
- O OpenMP implementa o modelo *fork-join*.
- Múltiplas *threads* executam tarefas definidas implicitamente ou explicitamente.
- As diretivas funcionam como anotações.
- São implementadas usando-se as diretivas de pré-processamento **#pragma**, em C/C++, e sentinelas **!\$**, no Fortran.
- Padrão OpenMP (*Open Multi-Processing*) [Chapman et al. (2007)](OpenMP API Site 2020)
 - É Conjunto de diretivas de compilação mais um pequeno conjunto de funções de biblioteca e variáveis de ambiente que usam como base as linguagens C/C++ e Fortran.
- As **diretivas de compilação** são usadas para anotar o código e guiar o compilador no processo de tradução e paralelização de código.
- Nas linguagens C e C++ as diretivas OpenMP são identificadas pelo *#pragma omp*, enquanto que na linguagem Fortran, as diretivas são identificadas pela sentinela *!\$omp*.
- O código anotado com diretivas de pré-processamento (**#pragma**) é tratado e então o código paralelo é gerado.

18.3 OpenMP

- O OpenMP implementa o modelo fork-join.

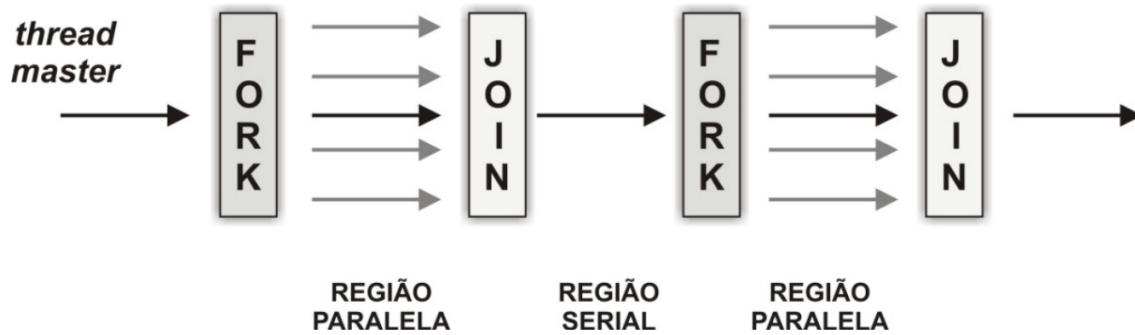


Figure 18.1: Modelo fork-join

[framebreak]

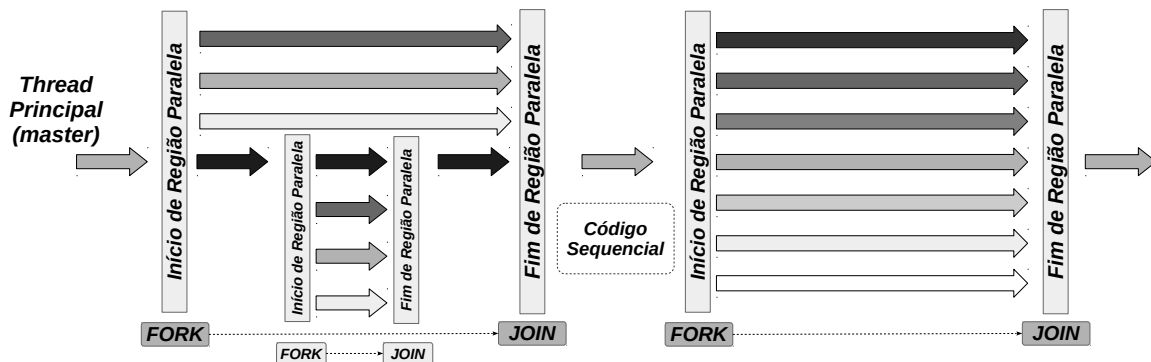


Figure 18.2: Modelo fork-join

- As *threads* são criadas a partir de blocos de código anotados.
- A *master thread* executa sequencialmente até encontrar a primeira construção que delimita uma região paralela.
- Uma operação similar a um *fork* acontece, fazendo com que um time de *threads* seja criado para executar o bloco que representa a região paralela (a *thread* mestre participa do time).

18.4 Expansão das Diretivas de Compilação

- As diretivas são substituídas pelo seu formato de código expandido com as chamadas para o runtime do OpenMP.
- Nosso estudo foi baseado nas diretivas de compilação da biblioteca `libgomp` (GNU Libgomp 2015) do GCC.
- A motivação desse estudo foi a necessidade de interceptar código de aplicações OpenMP para fazer offloading de código para aceleradores, como GPU.

18.5 Sintaxe dos Elementos de OpenMP para C/C++.

- *Variáveis de Ambiente:* `OMP_NOME`
 - São variáveis que controlam a execução do programa escrito com OpenMP.
- *Diretivas de Compilação:* `#pragma omp diretiva [cláusulas]`
 - Instrui o compilador para processar a seção de código após a diretiva.
- *Biblioteca e Rotinas:* `omp_serviço(...)`
 - Rotinas que manipulam e monitoram as *threads*, os processadores e as variáveis de ambiente. Existem rotinas para a sincronização das threads e para medir tempo.

18.6 Implementações OpenMP

- O OpenMP tem sido suportado por praticamente todos os compiladores atuais. [OpenMP Compilers & Tools](#).
- Compiladores como GCC (GCC 2020), Intel `icc` (Intel 2019) e LLVM `clang` (Clang 2013) tem implementações para OpenMP.
- Pelo menos duas implementações: GNU GCC `libgomp` (GNU Libgomp 2015) e a Intel `libomp` (*OpenMP* Runtime Library*) (Intel 2016).
- A especificação do OpenMP atualmente cobre *offloading* de código para aceleradores.
- A `libgomp` é capaz de fazer *offloading* usando o padrão OpenACC (Standard 2012).

19 Diretivas de Compilação

19.1 Diretivas de Compilação: Construtor Paralelo

- (exemplo: hello-world)
- *Construtor Paralelo*: `#pragma omp parallel`
- É a diretiva mais importante do OpenMP.
- É responsável pela indicação da região do código que será executada em paralelo.
- Se esse construtor não for especificado o programa será executado de forma sequencial.
- Quando a *thread* inicial encontra um construtor paralelo ela cria um grupo de *threads* que irão executar o código e torna-se a *thread* mestre desse grupo.
- Porém, esse construtor não divide o trabalho entre as *threads*, apenas cria a região paralela.

[framebreak]

[exampleblock]{Sintaxe}

```
#pragma omp parallel [clausula,...] novalinha
Instrucao ou Bloco de Comandos
```

[/exampleblock]

[alertblock]{Cláusulas que podem ser utilizadas com esse construtor}

```
if (expressao logica)
private (lista de variaveis)
shared (lista de variaveis)
firstprivate (lista de variaveis)
default (shared | none)
copyin (lista de variaveis)
reduction (operador: lista de variaveis)
num_threads (variavel inteira)
```

[/alertblock]

[framebreak]

- Hello World!!!

[columns]

[column=0.4]

```
#include <stdio.h>

int main() {
    printf("Hello World!!!\n");
    return 0;
}
```

[column=0.6]

```
#include <stdio.h>
#include <omp.h>

int main() {
#pragma omp parallel
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d, %lu]: Hello World!!!\n", id_omp, id_sys);
}
    return 0;
}
```

[/columns]

[framebreak]

- *Condicional de Compilação* - Para tornar possível a compilação de um programa **OpenMP** tanto na versão paralela quanto na versão sequencial.

```
#include <stdio.h>
#ifdef _OPENMP
#include <omp.h>
#else
#define omp_get_thread_num() 0
#endif

int main() {
#pragma omp parallel
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d, %lu]: Hello World!!!\n", id_omp, id_sys);
}
    return 0;
}
```

20 Variáveis de Ambiente

20.1 Variáveis de Ambiente do OpenMP

- *Exemplo:* [variaveis-omp](#)

```
int main() {
    int i;
    int omp_np; /* Número de Processadores. */
    int omp_nt; /* Número de threads OpenMP. */
    int omp_tid; /* Id da Thread OpenMP. */

    /* Teste 1 */
    printf("Teste 1: Sem Região Paralela -----\\n");
    /* Recupera os parâmetros do OpenMP utilizando as funções. */
    omp_np = omp_get_num_procs(); /* Recupera o número de processadores. */
    omp_nt = omp_get_num_threads(); /* Recupera o número de threads. */
    omp_tid = omp_get_thread_num(); /* recupera o id da thread OpenMP. */

    printf("OpenMP thread id: %2d, pthread id: %lu, Num. Processadores: %2d, Num. OpenMP Threads: %2d\\n", omp_tid, (long int) pthread_self(), omp_np, omp_nt);

    /* Teste 2 */
    printf("Teste 2: Região Paralela Padrão -----\\n");
    #pragma omp parallel
    {
        omp_np = omp_get_num_procs(); /* Recupera o número de processadores. */
        omp_nt = omp_get_num_threads(); /* Recupera o número de threads. */
        omp_tid = omp_get_thread_num(); /* recupera o id da thread OpenMP. */

        printf("OpenMP thread id: %2d, pthread id: %lu, Num. Processadores: %2d, Num. OpenMP Threads: %2d\\n", omp_tid, (long int) pthread_self(), omp_np, omp_nt);
    }
    /* Teste 3 */
    printf("Teste 3: Região Paralela com num_threads(8) -----\\n");
    #pragma omp parallel num_threads(8)
    {
        omp_np = omp_get_num_procs(); /* Recupera o número de processadores. */
        omp_nt = omp_get_num_threads(); /* Recupera o número de threads. */
        omp_tid = omp_get_thread_num(); /* recupera o id da thread OpenMP. */

        printf("OpenMP thread id: %2d, pthread id: %lu, Num. Processadores: %2d, Num. OpenMP Threads: %2d\\n", omp_tid, (long int) pthread_self(), omp_np, omp_nt);
    }

    printf("Fim -----\\n");

    return 0;
}
```

[framebreak]

```

[rag@backporting variaveis-omp]$ make
[rag@backporting variaveis-omp]$ ./variaveis-omp-03.exe
Teste 1: Sem Região Paralela -----
OpenMP thread id: 0, pthread id: 140371058636800, Num. Processadores: 16, Num. OpenMP Threads: 1
Teste 2: Região Paralela Padrão -----
OpenMP thread id: 0, pthread id: 140371058636800, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 9, pthread id: 140370987939520, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 2, pthread id: 140371046688448, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 7, pthread id: 140371004724928, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 4, pthread id: 140371029903040, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 10, pthread id: 140370979546816, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 15, pthread id: 140370937583296, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 12, pthread id: 140370962761408, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 8, pthread id: 140370996332224, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 8, pthread id: 140370971154112, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 1, pthread id: 140371055081152, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 1, pthread id: 140370954368704, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 6, pthread id: 140371013117632, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 14, pthread id: 140370945976000, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 3, pthread id: 140371038295744, Num. Processadores: 16, Num. OpenMP Threads: 16
OpenMP thread id: 5, pthread id: 140371021510336, Num. Processadores: 16, Num. OpenMP Threads: 16
Teste 3: Região Paralela com num_threads(8) -----
OpenMP thread id: 1, pthread id: 140371055081152, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 5, pthread id: 140371021510336, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 4, pthread id: 140371029903040, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 7, pthread id: 140371004724928, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 6, pthread id: 140371013117632, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 3, pthread id: 140371038295744, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 0, pthread id: 140371058636800, Num. Processadores: 16, Num. OpenMP Threads: 8
OpenMP thread id: 2, pthread id: 140371046688448, Num. Processadores: 16, Num. OpenMP Threads: 8
Fim -----
[rag@backporting variaveis-omp]$ █

```

Figure 20.1: Execução/Variáveis

21 Diretivas de Compilação

21.1 ...

Diretivas de Compilação e Código OpenMP Expandido

(Formato pós expansão das diretivas)

LibGOMP:

*GNU Offloading and Multi Processing Runtime Library –
(GNU OpenMP Runtime Library)*

21.2 Estudo das Diretivas de Compilação

- Verificou-se o formato de código gerado pelo GCC + libgomp para os construtores de regiões paralelas (*parallel region*) e compartilhamento de trabalho (*parallel for*).
- O código foi gerado nas versões do GCC (4.8, 4.9, 5.3, 6.1) para verificarmos o formato interceptável.
- Adotamos o GCC 4.8, porque esta versão apresentar um formato mais consistente e definido que possibilita a interceptação.

21.3 Regiões paralelas: construtor `parallel`

- Esta é uma das mais importantes diretivas, pois ela é responsável pela demarcação de regiões paralelas.

[exampleblock]{Regiões paralelas são criadas usando-se o construtor}

```
#pragma omp parallel [clause[ [,] clause] ... ] new-line
{
    /* Bloco estruturado. */
}
```

[/exampleblock]

- Quando uma região paralela é encontrada, é criado um time de threads para executar o código da região.
- Porém, esse construtor não divide o trabalho entre as threads.

[framebreak]

- Quando a diretiva de região paralela é utilizada:

```
#pragma omp parallel
{
    // body;
}
```

- A diretiva *parallel* é implementada com a criação de uma nova função (*outlined function*) usando o código contido em *body*.
- A `libgomp` usa funções para delimitar a região. Chamadas a essas funções são colocadas no código para indicar o início e o fim de uma região paralela:

[alertblock]{The `libgomp` ABI}

```
void GOMP_parallel_start(void (*fn)(void *), void *data,
unsigned num_threads)
void GOMP_parallel_end(void)
```

[/alertblock]

- O código expandido gerado assume o formato:

```
void subfunction (void *data){
    use data;
    body;
}

// replace the annotated parallel region.
setup data;
GOMP_parallel_start(subfunction, &data, num threads);
subfunction(&data);
GOMP_parallel_end();
```

[framebreak]

- *Exemplo:* [parallel-region](#)

```
#include <omp.h>

int main (void) {

#pragma omp parallel
{
    // body.
}

return 0;
}
```

[framebreak]

- Utilizando o GCC com a opção para manter os arquivos intermediários do OpenMP (*-fopenmp -fdump-tree-ompexp*):

```
[terminal] aula-intro-openmp/src/parallel-region$ gcc -O0 -fopenmp -fdump-tree-ompexp
parallel-region.c aula-intro-openmp/src/parallel-region$ [/terminal]
```

[framebreak]

- O código em GIMPLE, que é a representação intermediária do GCC:

```
main._omp_fn.0 (void * .omp_data_i) {
    return;
}

main () {
    int D.1803;

    <bb 2>:
    __builtin_GOMP_parallel_start (main._omp_fn.0, 0B, 0);
    main._omp_fn.0 (0B);
```

```

__builtin_GOMP_parallel_end ();
D.1803 = 0;

<LO>:
    return D.1803;
}

```

[framebreak]

- O código em *assembly* pode ser gerado com o parâmetros `-S`:

[terminal] aula-intro-openmp/src/parallel-region\$ gcc -O0 -fopenmp -fdump-tree-ompexp parallel-region.c -S -o parallel-region.asm aula-intro-openmp/src/parallel-region\$ [/terminal]

```

.file      "parallel-region.c"
.text
.globl     main
.type      main, @function

main:
    pushq   %rbp
    movq    %rsp, %rbp
    movl    $0, %edx
    movl    $0, %esi
    movl    $main._omp_fn.0, %edi
    call    GOMP_parallel_start
    movl    $0, %edi
    call    main._omp_fn.0
    call    GOMP_parallel_end
    movl    $0, %eax
    popq    %rbp
    ret
.size      main, .-main

.type      main._omp_fn.0, @function
main._omp_fn.0:
    pushq   %rbp
    movq    %rsp, %rbp
    movq    %rdi, -8(%rbp)
    popq    %rbp
    ret
.size      main._omp_fn.0, .-main._omp_fn.0
.ident     "GCC: (Debian 4.8.4-1) 4.8.4"
.section   .note.GNU-stack,"",@progbits

```

21.4 Outras Flags Úteis

Vimos que uma das vantagens do uso de **diretivas de compilação** é que o código anotado ao ser compilado sem a *flag* `-fopenmp` faz com que o compilador (GCC, clang ou icx) ignore as diretivas e o código sequencial seja gerado.

Pode acontecer também do compilador não ter aquela diretiva implementada, por exemplo o GCC em uma versão antiga poderia não conhecer uma data diretiva atual, como a `target` para aceleradores. O código seria gerado sem considerar a diretiva, não gerando a versão do código para GPU, deixando o usuário pensar que a versão paralela havia sido gerada e de fato não foi.

O GCC tem uma *flag* `-Wunknown-pragmas` que instrui o compilador a emitir uma mensagem caso encontre alguma diretiva de pré-processamento não conhecida.

O exemplo [unknown-directives](#) apresentado também no Código 21.1 mostra duas diretivas uma existente (`parallel`) e outra que não existe no OpenMP, a diretiva com o construtor `utfpr`.

Código 21.1 unknown-directives.c Verificando a flag -Wunknown-pragmas

```
#include <omp.h>

int main(int argc, char *argv[]) {

#pragma omp parallel num_threads(4)
{
    // body.
}

#pragma omp utfpr
{
    // body.
}

    return 0;
}
```

Para compilar o exemplo do Código 21.2, podemos utilizar o compilador `gcc` ou o `clang`, o programa deve ser compilado com as *flags* `-fopenmp` e `-Wunknown-pragmas`, o GCC acusará que não conhece a diretiva `omp` com o construtor `utfpr` e irá ignorá-la, enquanto que o `clang` irá acusar um erro, pois não conhece a diretiva.

Código 21.2 Terminal Diretivas desconhecidas

```
[unknown-directives]$ gcc -fopenmp -Wunknown-pragmas unknown-directives.c -o unknown-directives
unknown-directives.c: In function 'main':
unknown-directives.c:10: warning: ignoring '#pragma omp utfpr' [-Wunknown-pragmas]
    10 | #pragma omp utfpr
[unknown-directives]$ clang -fopenmp -Wunknown-pragmas unknown-directives.c -o unknown-directives
unknown-directives.c:10:13: error: expected an OpenMP directive
    10 | #pragma omp utfpr
        |               ^
1 error generated.
[unknown-directives]$
```

Outro ponto interessante é que sem a *flag* `-fopenmp` ligada, até mesmo as diretivas existentes no OpenMP tornam-se desconhecidas para o compilador, conforme apresentado no exemplo Código 21.3.

Código 21.3 Terminal Diretivas desconhecidas com a flag -fopenmp desligada

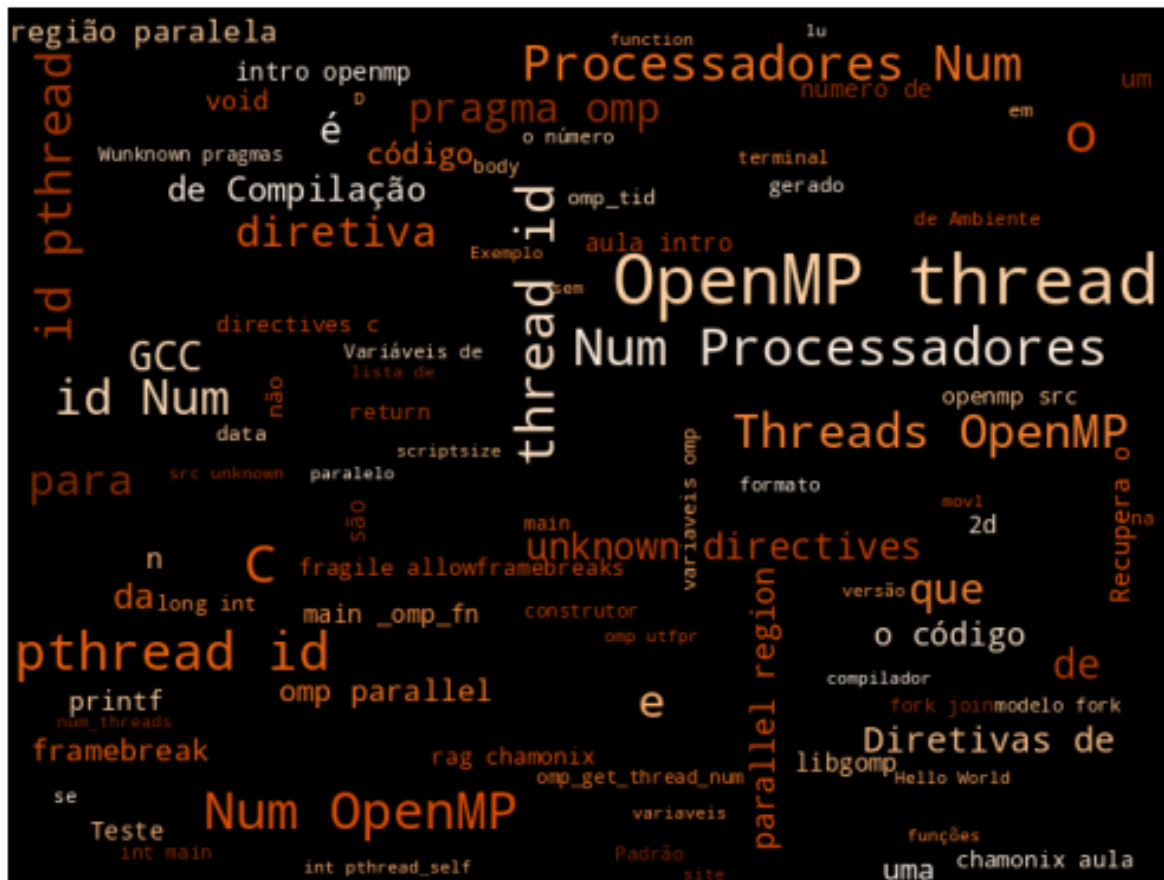
```
[unknown-directives]$ gcc -Wunknown-pragmas unknown-directives.c -o unknown-directives.exe
unknown-directives.c: In function 'main':
unknown-directives.c:5: warning: ignoring '#pragma omp parallel' [-Wunknown-pragmas]
    5 | #pragma omp parallel num_threads(4)
      | 
unknown-directives.c:10: warning: ignoring '#pragma omp utfpr' [-Wunknown-pragmas]
   10 | #pragma omp utfpr
      | 
[unknown-directives]$ clang -Wunknown-pragmas unknown-directives.c -o unknown-directives.exe
[unknown-directives]$
```

21.5 Considerações Finais

O OpenMP permite identificar oportunidades de paralelismo, dividir o trabalho de forma equilibrada, sincronizar apenas quando necessário e medir continuamente o impacto das mudanças dentro do código. Sendo flexível e portátil, ele permite que você comece paralelizando um único laço e avançando para arquiteturas mais complexas, ajustando **scheduling**, explorando tarefas independentes e combinando paralelismo com outras técnicas de otimização.

O próximo passo é continuar testando diferentes configurações, medindo resultados, identificando gargalos e refinando o código. A prática constante e a análise crítica são as chaves para dominar o OpenMP e, mais amplamente, a programação paralela.

21.6 Word Cloud



Referências

- Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. 2007. *Using OpenMP: Portable Shared Memory Parallel Programming (Scientific and Engineering Computation)*. The MIT Press.
- Clang, Site. 2013. “Clang Site @ONLINE.” <http://clang.llvm.org/>.
- GCC. 2020. *GCC, the GNU Compiler Collection*. <http://gcc.gnu.org/onlinedocs>.
- GNU Libgomp. 2015. *GNU libgomp, GNU Offloading and Multi Processing Runtime Library documentation (Online manual)*. <https://gcc.gnu.org/onlinedocs/libgomp/>.
- Hirata, Hiroaki, Kozo Kimura, Satoshi Nagamine, et al. 1992. “An Elementary Processor Architecture with Simultaneous Instruction Issuing from Multiple Threads.” *Proceedings of the 19th Annual International Symposium on Computer Architecture* (New York, NY, USA), ISCA '92, 136–45. <https://doi.org/10.1145/139669.139710>.
- Intel. 2016. *Intel® OpenMP* Runtime Library Interface*. Intel. https://www.openmpurl.org/sites/default/files/resources/libomp_20160322_manual.pdf.
- Intel. 2019. *Intel® c++ Compiler 19.1 Developer Guide and Reference*. <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>.
- Marr, Deborah T., Frank Binns, David L. Hill, et al. 2002. “Hyper-Threading Technology Architecture and Microarchitecture.” *Intel Technology Journal* 6 (1): 4–15.
- Olukotun, Kunle, Basem A. Nayfeh, Lance Hammond, Ken Wilson, and Kunyung Chang. 1996. “The Case for a Single-Chip Multiprocessor.” *SIGPLAN Not.* (New York, NY, USA) 31 (9): 2–11. <https://doi.org/10.1145/248209.237140>.
- OpenMP API Site. 2020. *The OpenMP(r) API Specification for Parallel Programming*. <http://openmp.org>.
- OpenMP-ARB. 2011. *OpenMP Application Program Interface Version 3.1*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP3.1.pdf>.
- OpenMP-ARB. 2013. *OpenMP Application Program Interface Version 4.0*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>.
- OpenMP-ARB. 2014. *OpenMP Application Program Interface Examples 4.0.1*. OpenMP Architecture Review Board (ARB). http://openmp.org/mp-documents/OpenMP_Examples_4.0.1.pdf.
- OpenMP-ARB. 2015. *OpenMP Application Program Interface Version 4.5*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/openmp-4.5.pdf>.
- Smith, J. E., and G. S. Sohi. 1995. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE* 83 (12): 1609–24. <https://doi.org/10.1109/5.476078>.
- Spracklen, Lawrence, and Santosh G. Abraham. 2005. “Chip Multithreading: Opportunities and Challenges.” *Proceedings of the 11th International Symposium on High-Performance Computer Architecture* (USA), HPCA '05, 248–52. <https://doi.org/10.1109/HPCA.2005.10>.
- Standard, OpenACC. 2012. “OpenACC Directives for Accelerators Site @ONLINE.” March. <http://www.openacc-standard.org/>.
- Tomasulo, R. M. 1967. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units.” *IBM Journal of Research and Development* 11 (1): 25–33. <https://doi.org/10.1147/rd.111.0025>.

Tullsen, Dean M., Susan J. Eggers, and Henry M. Levy. 1995. "Simultaneous Multithreading: Maximizing on-Chip Parallelism." *SIGARCH Comput. Archit. News* (New York, NY, USA) 23 (2): 392–403. <https://doi.org/10.1145/225830.224449>.

Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky. 1994. "Performance Estimation of Multistreamed, Superscalar Processors." *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences* 1: 195–204. <https://doi.org/10.1109/HICSS.1994.323172>.

Tutorial de OpenMP C/C++: http://www.lncc.br/pdf_consultar.php?id_arquivo=4286

Notas de Aulas.

22 Diretivas de Compilação para Regiões Paralelas

Resumo: Regiões Paralelas são fundamentais para a execução de aplicações no contexto de Computação Paralela. Nessas regiões que o fluxo de execução principal da aplicação se divide entre mais *threads* e o paralelismo na execução de tarefas pode ser alcançado. Neste material apresentamos a Diretiva de Compilação para Regiões Paralelas, a diretiva *parallel* e as cláusulas que podem ser utilizadas. O objetivo é testar os efeitos da utilização da diretiva na criação de regiões paralelas, nas quais um time de *threads* é criado para a execução do código da região, e verificar os efeitos causados com a aplicação das cláusulas. Na prática desse capítulo, o objetivo é percebermos a redução da quantidade de código com o uso de diretivas de compilação. O estudo do código desse exemplo de soma de vetores, um exemplo simples, ajuda a percebermos a quantidade de código necessária para a criação das *threads*.

22.1 Introdução

Vamos explorar a diretiva de compilação com o construtor para Regiões Paralelas (*parallel*) e as cláusulas admitidas por ele¹. Testando os efeitos das cláusulas.

Este é um dos construtores mais importantes do **OpenMP**, pois ele é responsável pela demarcação de regiões paralelas, indicando a região de código que será executada em paralelo pelas *threads*.

22.2 Diretivas de Compilação para Regiões paralelas: construtor *parallel*

Quando uma região paralela é encontrada pela *thread* principal (**master**), é criado um time de *threads* que irão executar o código da região paralela. Se esse construtor não for especificado o programa será executado de forma sequencial, isto é, não será criada a região paralela.

A *thread principal* do processo torna-se a *thread* mestre (**master thread**) para esse grupo.

Regiões paralelas são criadas usando-se o construtor e cláusulas conforme apresentado na Nota 2. A diretiva **parallel** é responsável pela criação de regiões paralelas, indicando a região de código que será executada em paralelo.

i Nota 2: Diretiva com construtor **parallel**

```
#pragma omp parallel [clause[ [,] clause] ... ] new-line
{
    /* Bloco estruturado. */
}
```

Quando a diretiva com o construtor de região paralela é inserida no código, conforme o Código 22.1.

O compilador irá expandir o código da região paralela

¹[OpenMP-ARB (2013)](OpenMP-ARB 2014)

Código 22.1 Hello World em OpenMP

```
#pragma omp parallel
{
    /* Body */
}
```

- A diretiva *parallel* é implementada com a criação de uma nova função (*outlined function*) usando o código contido em *body*.
- A *libgomp* usa funções para delimitar a região. Chamadas a essas funções são colocadas no código para indicar o início e o fim de uma região paralela:

! A libgomp ABI

```
void GOMP_parallel_start(void (*fn)(void *), void *data,
    unsigned num_threads)
void GOMP_parallel_end(void)
```

- O código expandido gerado assume o formato:

i Nota 2: Diretiva com construtor `parallel`

```
void subfunction (void *data){
    use data;
    body;
}

// replace the annotated parallel region.
setup data;

GOMP_parallel_start(subfunction, &data, num threads);
subfunction(&data);
GOMP_parallel_end();
```

[exampleblock]{Comando do GCC}

```
`${CC}` -fopenmp -fdump-tree-ompexp parallel-region.c
```

[/exampleblock]

- O código em GIMPLE, que é a representação intermediária do GCC:

```
main._omp_fn.0 (void * .omp_data_i) {
    return;
}

main () {
    int D.1803;
```

```

<bb 2>:
  __builtin_GOMP_parallel_start (main._omp_fn.0, 0B, 0);
  main._omp_fn.0 (0B);
  __builtin_GOMP_parallel_end ();
  D.1803 = 0;

<LO>:
  return D.1803;
}

```

- A representação gráfica do código GIMPLE pode ser gerada com o comando:

[exampleblock]{Comando do GCC}

```

${CC} -fopenmp -fdump-tree-ompexp-graph parallel-region.c

```

[/exampleblock]

- Gerar o código em *Assembly*:

i Comando do GCC

```

${CC} -fopenmp parallel-region.c -fno-asynchronous-unwind-tables -S -o
parallel-region.s

```

- Código em *Assembly*:

<pre> .file "parallel-region.c" .text .globl main .type main, @function main: pushq %rbp movq %rsp, %rbp movl \$0, %edx movl \$0, %esi movl \$main._omp_fn.0, %edi call GOMP_parallel_start movl \$0, %edi call main._omp_fn.0 call GOMP_parallel_end movl \$0, %eax popq %rbp ret .size main, .-main </pre>	<pre> .type main._omp_fn.0, @function main._omp_fn.0: pushq %rbp movq %rsp, %rbp movq %rdi, -8(%rbp) popq %rbp ret .size main._omp_fn.0, .-main._omp_fn.0 .ident "GCC: (Debian 4.8.4-1) 4.8.4" .section .note.GNU-stack,"",@progbits </pre>
---	--

Porém, esse construtor não divide o trabalho entre as *threads*, apenas cria a região paralela.

22.3 Cláusulas do Construtor *parallel*

O Construtor para a definição de **Regiões Paralelas** admite algumas cláusulas. A sintaxe para o construtor *parallel* com cláusulas:

💡 Sintaxe

```
#pragma omp parallel [clause[ [,] clause] ... ] new-line
{
    body;
}
```

- Serão apresentadas aqui as cláusulas que podem ser utilizadas com o construtor *parallel*.

i Cláusulas que podem ser utilizadas nesse construtor

```
if (scalar-expression)
num_threads (variavel inteira)
default (shared | none)
private (lista de variaveis)
firstprivate (lista de variaveis)
shared (lista de variaveis)
copyin (lista de variaveis)
reduction (operador: lista de variaveis)
proc_bind(master | close | spread)
```

Antes de testarmos cada uma das *cláusulas* do construtor *parallel*, vamos relembrar o exemplo básico **Hello World** que servirá de código de partida. O Código 22.2 na linguagem C imprime a famosa e conhecida mensagem **Hello World!!!** que todo programador iniciante em uma linguagem de programação escreve como primeiro exemplo.

Código 22.2 helloworld-seq.c Hello World em C

```
#include <stdio.h>

int main() {
    printf("Hello World!!!\n");

    return 0;
}
```

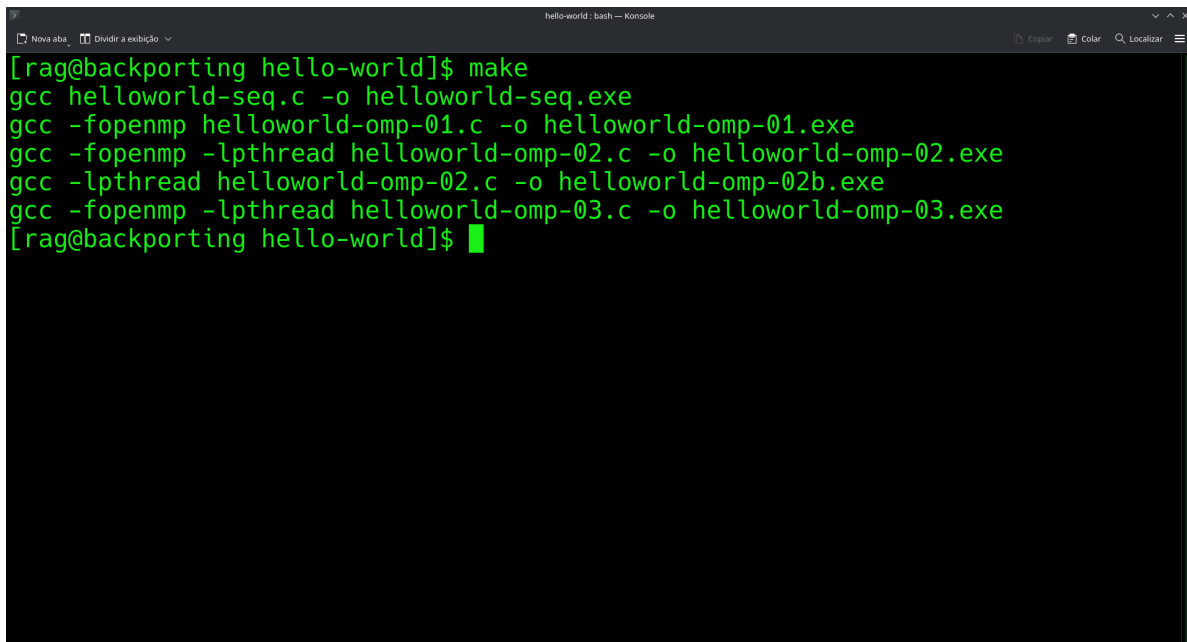
O mesmo programa em **OpenMP** ficaria em uma versão básica conforme o Código 22.3. A chamada à função **printf** foi colocada dentro de uma região paralela.

Notem que na versão apresentada acima, não foi especificada o número de threads que deverão ser criadas. O número de threads utilizadas será determinado pelo número de processadores disponíveis no sistema.

A Figure 22.1.

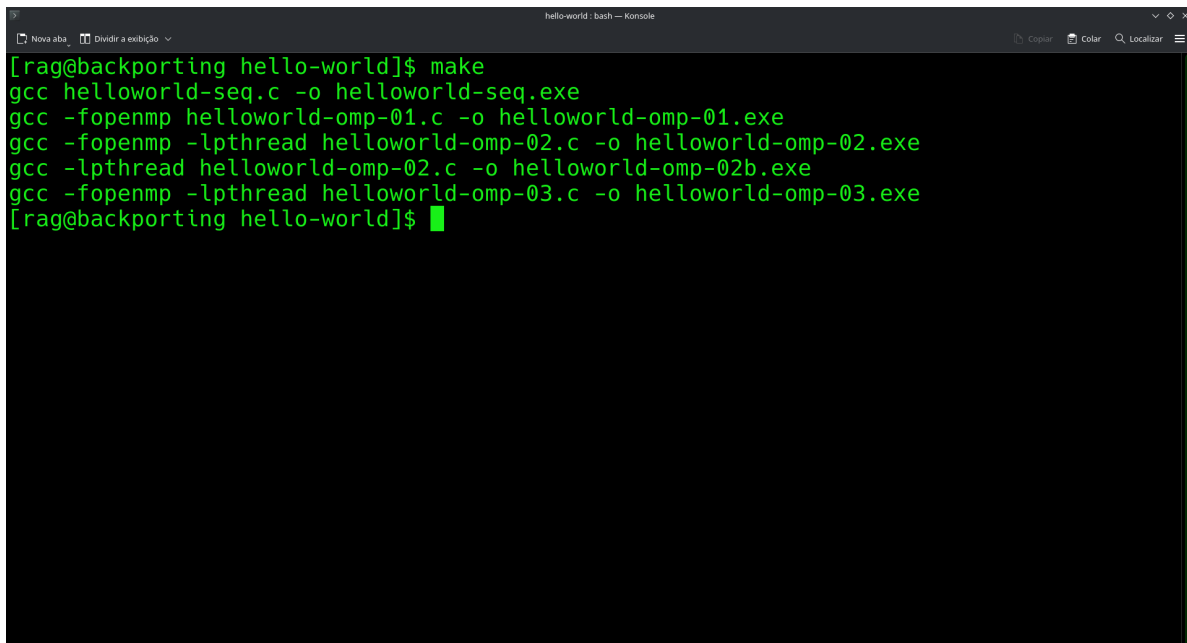
1.

- **Condicional de Compilação** - Para tornar possível a compilação de um programa **OpenMP** tanto na versão paralela quanto na versão sequencial.



```
hello-world : bash — Konsole
Nova aba  Dividir a exibição
[rag@backporting hello-world]$ make
gcc helloworld-seq.c -o helloworld-seq.exe
gcc -fopenmp helloworld-omp-01.c -o helloworld-omp-01.exe
gcc -fopenmp -lpthread helloworld-omp-02.c -o helloworld-omp-02.exe
gcc -lpthread helloworld-omp-02.c -o helloworld-omp-02b.exe
gcc -fopenmp -lpthread helloworld-omp-03.c -o helloworld-omp-03.exe
[rag@backporting hello-world]$
```

Figure 22.1: Execução do make



```
hello-world : bash — Konsole
Nova aba  Dividir a exibição
[rag@backporting hello-world]$ make
gcc helloworld-seq.c -o helloworld-seq.exe
gcc -fopenmp helloworld-omp-01.c -o helloworld-omp-01.exe
gcc -fopenmp -lpthread helloworld-omp-02.c -o helloworld-omp-02.exe
gcc -lpthread helloworld-omp-02.c -o helloworld-omp-02b.exe
gcc -fopenmp -lpthread helloworld-omp-03.c -o helloworld-omp-03.exe
[rag@backporting hello-world]$
```

Figure 22.2: Execução do hello world em OpenMP

Código 22.3 helloworld-omp-01.c Hello World em OpenMP

```
/* Compilar:
 * gcc -fopenmp helloworld-omp-01.c -o helloworld-omp-01.exe
 * ou
 * make omp-01
 */
#include <stdio.h>
#include <omp.h>

int main() {

#pragma omp parallel
{
    printf("Hello World!!!\n");
}

    return 0;
}
```

```
#ifdef _OPENMP
#include <omp.h>
#else
#define omp_get_thread_num() 0
#define omp_get_num_threads() 1
#define omp_get_num_procs() (*(system("cat /proc/cpuinfo | \
                                grep 'processor' | wc -l"))*)
#endif
```

- Hello World completo:

O Código 22.5 ilustra o funcionamento do Hello World.

```
#include <stdio.h>
#include <stdlib.h>
#ifdef _OPENMP
#include <omp.h>
#else
#define omp_get_thread_num() 0
#define omp_get_num_threads() 1
#define omp_get_num_procs() (*(system("cat /proc/cpuinfo | \
                                grep 'processor' | wc -l"))*)
#endif

int main() {
#pragma omp parallel
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
```

Código 22.4 helloworld-omp-02.c Hello World em OpenMP com cláusula `num_threads`

```
/* Compilar:
 * gcc -fopenmp helloworld-omp-02.c -o helloworld-omp-02.exe
 * ou
 * make omp-02
 */
#include <stdio.h>
#include <pthread.h>
#ifdef _OPENMP
#include <omp.h>
#else
#define omp_get_thread_num() 0
#endif

int main() {
    #pragma omp parallel
    {
        int id_omp = omp_get_thread_num();
        long int id_sys = (long int) pthread_self();
        printf("Thread[%2d, %lu]: Hello World!!!\n", id_omp, id_sys);
    }

    return 0;
}
```

```
printf("Thread[%d] [%lu]: Hello World!!!\n", id_omp, id_sys);
}
return 0;
}
```

22.4 Cláusula *num_threads* do Construtor *parallel*

- A cláusula `num_threads` é utilizada para definir a quantidade de *threads* a serem criadas na *região paralela*:

Sintaxe

```
#pragma omp parallel num_threads(valor inteiro)
{
    body;
}
```

Código 22.5 helloworld-omp-01.c Hello World

```
/* Compilar:
 * gcc -fopenmp helloworld-omp-01.c -o helloworld-omp-01.exe
 * ou
 * make omp-01
 */
#include <stdio.h>
#include <omp.h>

int main() {

#pragma omp parallel
{
    printf("Hello World!!!\n");
}

    return 0;
}
```

Definição

Define o número de *threads* que serão criadas na seção paralela.
A mesma definição pode ser feita pela variável de ambiente: `OMP_NUM_THREADS`

```
setenv OMP_NUM_THREADS 4,3,2
```

Seção 4.2 OMP_NUM_THREADS² da especificação.
ou chamando-se a função:

```
void omp_set_num_threads(int num_threads);
```

Seção 3.2.1 omp_set_num_threads³ do documento de especificação.

- Um exemplo de uso de `num_threads`. Quatro *threads* serão criadas para executarem o código da região paralela.

```
/*Codigo suprimido.*/
int main() {
#pragma omp parallel num_threads(4)
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d][%lu]: Hello World!!!\n", id_omp, id_sys);
}
```

³<http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf#M6.9.IDZ866058596>

³<http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf#G5.1001042>

```

return 0;
}

```

- O resultado apresentado no terminal é a execução feita pelas 4 *threads* criadas pelo *runtime* do OpenMP, com os *ids* de 0 a 3.

```

[parallel-num_threads-clause]$ ./example-parallel-num-threads.exe
Thread[0] [18446744072025257856]: Hello World!!!
Thread[1] [18446744072012752640]: Hello World!!!
Thread[3] [18446744071995967232]: Hello World!!!
Thread[2] [18446744072004359936]: Hello World!!!
[parallel-num_threads-clause]$ ./example-parallel-num-threads.exe

```

- Uma outra forma de definir o número de *threads* a serem criadas na região paralela é usar a função `omp_set_num_threads(int num_threads)` como pode ser visto no Código 22.4.

Código 22.4. Uso da função `omp_set_num_threads(int num_threads)`

```

1  /* Código suprimido. */
2  int main() {
3      omp_set_num_threads(7);
4      #pragma omp parallel
5      {
6          int id_omp = omp_get_thread_num();
7          long int id_sys = (long int) pthread_self();
8          printf("Thread[%d] [%lu]: Hello World!!!\n", id_omp, id_sys);
9      }
10     return 0;
11 }

```

```

[parallel-num_threads-clause]$ ./example-parallel-omp_set_num_threads-function.exe
Thread[0] [18446744073166374784]: Hello World!!!
Thread[6] [18446744073111906048]: Hello World!!!
Thread[3] [18446744073137084160]: Hello World!!!
Thread[4] [18446744073128691456]: Hello World!!!
Thread[2] [18446744073145476864]: Hello World!!!
Thread[1] [18446744073153869568]: Hello World!!!
Thread[5] [18446744073120298752]: Hello World!!!
[parallel-num_threads-clause]$

```

22.5 Cláusula *if* do Construtor *parallel*

Sintaxe

```

#pragma omp parallel if(expressao)
{
    body;
}

```

i Definição

Se a expressão retornar um valor verdadeiro o time de threads é criado conforme a definição de `num_threads`. Se o valor da expressão for falso apenas uma thread irá executar o código.

Seção 2.5.1 Determining the Number of Threads for a parallel Region do documento de especificação:

<http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf#G4.1027332>

Ver o *Algoritmo 2.1*.

- Um exemplo que **expression** retornará *false*.

```
/* Codigo suprimido. */
int main() {
    int n = 10;

#pragma omp parallel if(n>10) num_threads(4)
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d] [%lu]: Hello World!!!\n", id_omp, id_sys);
}
    return 0;
}
```

```
[parallel-if-clause]$ ./example-parallel-if-false.exe
Thread[0] [340465536]: Hello World!!!
[parallel-if-clause]$
```

- Um exemplo que **expression** retornará *true*.

```
/* Codigo suprimido. */
int main() {
    int n = 11;

#pragma omp parallel if(n>10) num_threads(4)
{
    int id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d] [%lu]: Hello World!!!\n", id_omp, id_sys);
}
    return 0;
}
```

```
[parallel-if-clause]$ ./example-parallel-if-true.exe
Thread[0] [1551517568]: Hello World!!!
Thread[2] [1530619648]: Hello World!!!
Thread[1] [1539012352]: Hello World!!!
Thread[3] [1522226944]: Hello World!!!
[parallel-if-clause]$
```

22.6 Cláusulas *default*, *private*, *shared* do Construtor *parallel*

Sintaxe

```
#pragma omp parallel default(none) private(lista) shared(lista)
{
    body;
}
```

Definição

A cláusula `default(none)` requer que cada variável referenciada no construtor tenha um atributo de compartilhamento determinado explicitamente.

Seção 33 *The default(none) Clause* do documento de especificação:

http://openmp.org/mp-documents/OpenMP_Examples_4.0.1.pdf#G3.1035623

- Um exemplo do uso das cláusulas de compartilhamento de dados.

```
/* Codigo suprimido. */
int main() {
    int id_omp, n = 0;

#pragma omp parallel default(none) private(id_omp) shared(n)
{
    id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d] [%lu]: Valor de n: %d antes.\n", id_omp, id_sys, n);
    n = n + id;
    printf("Thread[%d] [%lu]: Valor de n: %d depois.\n", id_omp, id_sys, n);
}

    printf("Thread[%d] [%lu]: Valor final de n: %d.\n", omp_get_thread_num(), (long int) pthread_self(), n);
    return 0;
}
```

```
[parallel-data-share-clauses]$ ./example-parallel-data-share-clauses.exe
Thread[0] [18446744073195390848]: Valor de n: 0 antes.
Thread[0] [18446744073195390848]: Valor de n: 0 depois.
Thread[3] [18446744073166100224]: Valor de n: 0 antes.
Thread[3] [18446744073166100224]: Valor de n: 3 depois.
Thread[2] [18446744073174492928]: Valor de n: 0 antes.
Thread[2] [18446744073174492928]: Valor de n: 5 depois.
Thread[4] [18446744073157707520]: Valor de n: 0 antes.
Thread[4] [18446744073157707520]: Valor de n: 9 depois.
Thread[5] [18446744073149314816]: Valor de n: 0 antes.
Thread[5] [18446744073149314816]: Valor de n: 14 depois.
Thread[6] [18446744073140922112]: Valor de n: 0 antes.
Thread[6] [18446744073140922112]: Valor de n: 20 depois.
Thread[7] [18446744073132529408]: Valor de n: 0 antes.
Thread[7] [18446744073132529408]: Valor de n: 27 depois.
Thread[1] [18446744073182885632]: Valor de n: 0 antes.
Thread[1] [18446744073182885632]: Valor de n: 28 depois.
Thread[0] [18446744073195390848]: Valor final de n: 28.
[parallel-data-share-clauses]$
```

22.7 Cláusula *reduction* do Construtor *parallel*

Sintaxe

```
#pragma omp parallel reduction(operador: variavel)
{
    body;
}
```

Definição

A cláusula `reduction(operador: variavel)` faz a operação de redução em uma variável com os valores parciais de cada uma das threads.

2.14.3.6 *reduction clause* do documento de especificação:

<http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf#G4.1014420>

- Um exemplo do uso da cláusula *reduction*.

```
/* Codigo suprimido. */
int main() {
    int id_omp, n = 0;

#pragma omp parallel default(none) private(id_omp) reduction(+: n)
{
    id_omp = omp_get_thread_num();
    long int id_sys = (long int) pthread_self();
    printf("Thread[%d] [%lu]: Valor de n: %d antes.\n", id_omp, id_sys, n);
    n = n + id_omp;
    printf("Thread[%d] [%lu]: Valor de n: %d depois.\n", id_omp, id_sys, n);
}

    printf("Thread[%d] [%lu]: Valor final de n: %d.\n", omp_get_thread_num(), (long int) pthread_self(), n);
    return 0;
}
```

```
[parallel-reduction-clause]$ ./example-parallel-reduction-clauses.exe
Thread[2] [1719039744]: Valor de n: 0 antes.
Thread[2] [1719039744]: Valor de n: 2 depois.
Thread[5] [1693861632]: Valor de n: 0 antes.
Thread[5] [1693861632]: Valor de n: 5 depois.
Thread[4] [1702254336]: Valor de n: 0 antes.
Thread[4] [1702254336]: Valor de n: 4 depois.
Thread[3] [1710647040]: Valor de n: 0 antes.
Thread[3] [1710647040]: Valor de n: 3 depois.
Thread[7] [1677076224]: Valor de n: 0 antes.
Thread[7] [1677076224]: Valor de n: 7 depois.
Thread[1] [1727432448]: Valor de n: 0 antes.
Thread[1] [1727432448]: Valor de n: 1 depois.
Thread[6] [1685468928]: Valor de n: 0 antes.
Thread[6] [1685468928]: Valor de n: 6 depois.
Thread[0] [1739937664]: Valor de n: 0 antes.
Thread[0] [1739937664]: Valor de n: 0 depois.
Thread[0] [1739937664]: Valor final de n: 28.
[parallel-reduction-clause]$
```

22.8 Prática: Exercício de Implementação

Soma de Vetores com Regiões Paralelas

1. Implementamos uma versão do exemplo **soma de vetores** utilizando processos com memória compartilhada e com **pthread**s (✓)
2. Implemente uma versão do exemplo **soma de vetores** utilizando a diretiva **parallel** e recursos vistos até o momento.
3. Execute os exemplos **vectoradd** utilizando o construtor **parallel** (OpenMP-ARB 2011) (OpenMP-ARB 2013) (OpenMP-ARB 2015), com particionamento de iterações **estática** e **dinâmica**.

22.9 Relatório

- Escreva sobre as alterações que foram necessárias para a execução das versões paralelas do soma de vetores utilizando **#pragma omp parallel** com particionamento estático e dinâmico. Complemente o relatório comparativo da prática anterior utilizando a versão sequencial e com **pthread**s.

22.10 Considerações Finais

O OpenMP permite identificar oportunidades de paralelismo, dividir o trabalho de forma equilibrada, sincronizar apenas quando necessário e medir continuamente o impacto das mudanças dentro do código. Sendo flexível e portátil, ele permite que você comece paralelizando um único laço e avançando para arquiteturas mais complexas, ajustando **scheduling**, explorando tarefas independentes e combinando paralelismo com outras técnicas de otimização.

O próximo passo é continuar testando diferentes configurações, medindo resultados, identificando gargalos e refinando o código. A prática constante e a análise crítica são as chaves para dominar o OpenMP e, mais amplamente, a programação paralela.

22.11 Word Cloud



Referências

- Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. 2007. *Using OpenMP: Portable Shared Memory Parallel Programming (Scientific and Engineering Computation)*. The MIT Press.
- Clang, Site. 2013. “Clang Site @ONLINE.” <http://clang.llvm.org/>.
- GCC. 2020. *GCC, the GNU Compiler Collection*. <http://gcc.gnu.org/onlinedocs>.
- GNU Libgomp. 2015. *GNU libgomp, GNU Offloading and Multi Processing Runtime Library documentation (Online manual)*. <https://gcc.gnu.org/onlinedocs/libgomp/>.
- Hirata, Hiroaki, Kozo Kimura, Satoshi Nagamine, et al. 1992. “An Elementary Processor Architecture with Simultaneous Instruction Issuing from Multiple Threads.” *Proceedings of the 19th Annual International Symposium on Computer Architecture* (New York, NY, USA), ISCA '92, 136–45. <https://doi.org/10.1145/139669.139710>.
- Intel. 2016. *Intel® OpenMP* Runtime Library Interface*. Intel. https://www.openmpurl.org/sites/default/files/resources/libomp_20160322_manual.pdf.
- Intel. 2019. *Intel® c++ Compiler 19.1 Developer Guide and Reference*. <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>.
- Marr, Deborah T., Frank Binns, David L. Hill, et al. 2002. “Hyper-Threading Technology Architecture and Microarchitecture.” *Intel Technology Journal* 6 (1): 4–15.
- Olukotun, Kunle, Basem A. Nayfeh, Lance Hammond, Ken Wilson, and Kunyung Chang. 1996. “The Case for a Single-Chip Multiprocessor.” *SIGPLAN Not.* (New York, NY, USA) 31 (9): 2–11. <https://doi.org/10.1145/248209.237140>.
- OpenMP API Site. 2020. *The OpenMP(r) API Specification for Parallel Programming*. <http://openmp.org>.
- OpenMP-ARB. 2011. *OpenMP Application Program Interface Version 3.1*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP3.1.pdf>.
- OpenMP-ARB. 2013. *OpenMP Application Program Interface Version 4.0*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>.
- OpenMP-ARB. 2014. *OpenMP Application Program Interface Examples 4.0.1*. OpenMP Architecture Review Board (ARB). http://openmp.org/mp-documents/OpenMP_Examples_4.0.1.pdf.
- OpenMP-ARB. 2015. *OpenMP Application Program Interface Version 4.5*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/openmp-4.5.pdf>.
- Smith, J. E., and G. S. Sohi. 1995. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE* 83 (12): 1609–24. <https://doi.org/10.1109/5.476078>.
- Spracklen, Lawrence, and Santosh G. Abraham. 2005. “Chip Multithreading: Opportunities and Challenges.” *Proceedings of the 11th International Symposium on High-Performance Computer Architecture* (USA), HPCA '05, 248–52. <https://doi.org/10.1109/HPCA.2005.10>.
- Standard, OpenACC. 2012. “OpenACC Directives for Accelerators Site @ONLINE.” March. <http://www.openacc-standard.org/>.
- Tomasulo, R. M. 1967. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units.” *IBM Journal of Research and Development* 11 (1): 25–33. <https://doi.org/10.1147/rd.111.0025>.

- Tullsen, Dean M., Susan J. Eggers, and Henry M. Levy. 1995. "Simultaneous Multithreading: Maximizing on-Chip Parallelism." *SIGARCH Comput. Archit. News* (New York, NY, USA) 23 (2): 392–403. <https://doi.org/10.1145/225830.224449>.
- Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky. 1994. "Performance Estimation of Multistreamed, Superscalar Processors." *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences* 1: 195–204. <https://doi.org/10.1109/HICSS.1994.323172>.

Referências

- Chapman, Barbara, Gabriele Jost, and Ruud van der Pas. 2007. *Using OpenMP: Portable Shared Memory Parallel Programming (Scientific and Engineering Computation)*. The MIT Press.
- Clang, Site. 2013. “Clang Site @ONLINE.” <http://clang.llvm.org/>.
- GCC. 2020. *GCC, the GNU Compiler Collection*. <http://gcc.gnu.org/onlinedocs>.
- GNU Libgomp. 2015. *GNU libgomp, GNU Offloading and Multi Processing Runtime Library documentation (Online manual)*. <https://gcc.gnu.org/onlinedocs/libgomp/>.
- Hirata, Hiroaki, Kozo Kimura, Satoshi Nagamine, et al. 1992. “An Elementary Processor Architecture with Simultaneous Instruction Issuing from Multiple Threads.” *Proceedings of the 19th Annual International Symposium on Computer Architecture* (New York, NY, USA), ISCA '92, 136–45. <https://doi.org/10.1145/139669.139710>.
- Intel. 2016. *Intel® OpenMP* Runtime Library Interface*. Intel. https://www.openmpurl.org/sites/default/files/resources/libomp_20160322_manual.pdf.
- Intel. 2019. *Intel® c++ Compiler 19.1 Developer Guide and Reference*. <https://software.intel.com/en-us/cpp-compiler-developer-guide-and-reference>.
- Marr, Deborah T., Frank Binns, David L. Hill, et al. 2002. “Hyper-Threading Technology Architecture and Microarchitecture.” *Intel Technology Journal* 6 (1): 4–15.
- Olukotun, Kunle, Basem A. Nayfeh, Lance Hammond, Ken Wilson, and Kunyung Chang. 1996. “The Case for a Single-Chip Multiprocessor.” *SIGPLAN Not.* (New York, NY, USA) 31 (9): 2–11. <https://doi.org/10.1145/248209.237140>.
- OpenMP API Site. 2020. *The OpenMP(r) API Specification for Parallel Programming*. <http://openmp.org>.
- OpenMP-ARB. 2011. *OpenMP Application Program Interface Version 3.1*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP3.1.pdf>.
- OpenMP-ARB. 2013. *OpenMP Application Program Interface Version 4.0*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/OpenMP4.0.0.pdf>.
- OpenMP-ARB. 2014. *OpenMP Application Program Interface Examples 4.0.1*. OpenMP Architecture Review Board (ARB). http://openmp.org/mp-documents/OpenMP_Examples_4.0.1.pdf.
- OpenMP-ARB. 2015. *OpenMP Application Program Interface Version 4.5*. OpenMP Architecture Review Board (ARB). <http://www.openmp.org/mp-documents/openmp-4.5.pdf>.
- Smith, J. E., and G. S. Sohi. 1995. “The Microarchitecture of Superscalar Processors.” *Proceedings of the IEEE* 83 (12): 1609–24. <https://doi.org/10.1109/5.476078>.
- Spracklen, Lawrence, and Santosh G. Abraham. 2005. “Chip Multithreading: Opportunities and Challenges.” *Proceedings of the 11th International Symposium on High-Performance Computer Architecture* (USA), HPCA '05, 248–52. <https://doi.org/10.1109/HPCA.2005.10>.
- Standard, OpenACC. 2012. “OpenACC Directives for Accelerators Site @ONLINE.” March. <http://www.openacc-standard.org/>.
- Tomasulo, R. M. 1967. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units.” *IBM Journal of Research and Development* 11 (1): 25–33. <https://doi.org/10.1147/rd.111.0025>.

- Tullsen, Dean M., Susan J. Eggers, and Henry M. Levy. 1995. "Simultaneous Multithreading: Maximizing on-Chip Parallelism." *SIGARCH Comput. Archit. News* (New York, NY, USA) 23 (2): 392–403. <https://doi.org/10.1145/225830.224449>.
- Yamamoto, W., M. J. Serrano, A. R. Talcott, R. C. Wood, and M. Nemirosky. 1994. "Performance Estimation of Multistreamed, Superscalar Processors." *1994 Proceedings of the Twenty-Seventh Hawaii International Conference on System Sciences* 1: 195–204. <https://doi.org/10.1109/HICSS.1994.323172>.